



Sponsoring Society:



Regular Papers

Effective Inversely Proportional Hypermutations for Unimodal and Multimodal Optimisation

Submission ID 192c6564-c140-4342-afef-6c9b3cd0ace8

Submission Version Revision 1

PDF Generation 21 Mar 2026 02:08:04 EST by Atypon ReX

Files for peer review

All files submitted by the author for peer review are listed below. Files that could not be converted to PDF are indicated; reviewers are able to access them online.

Name	Type of File	Size	Page
TEVCNew15PagesFlat.pdf	Anonymized Main Document - PDF	635.7 KB	Page 3
TEVCNewFullPaperFlat.pdf	Supplementary Material for Review	738.6 KB	Page 18
TEVCNew15PagesFlat.pdf	Main Document - Tracked Changes	635.7 KB	Page 40
Difference with conference version.docx	Previously Published - Statement	14.0 KB	Page 55
CorusOlivetoYazdaniGecco2019.pdf	Previously Published - Files	1.0 MB	Page 56
ReplyToReviewers.pdf	Author Response	167.0 KB	Page 65

Effective Inversely Proportional Hypermutations for Unimodal and Multimodal Optimisation

Anonymous authors

Abstract—Artificial Immune Systems (AIS) employing hypermutations with linear static mutation potential have recently been shown to be very effective at escaping local optima of combinatorial optimisation problems at the expense of being slower during the exploitation phase compared to standard evolutionary algorithms. In this paper, we examine the more commonly used dynamic inversely proportional hypermutations (IPH) with mutation potential for which previous theoretical analyses have shown disappointing performance. We first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations in the ideal situation where the optimum is known if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. Afterwards, we define a simple (1+1) Opt-IA that uses IPH and ageing for realistic applications where optimal solutions are unknown. The aim of this AIS is to approximate the ideal behaviour of the IPH better and better as the search space is explored and local optima of higher quality are identified. We present two variants, one that decreases the mutation rate proportionally until the local (global) optimum is identified, and another where the mutation rate may increase again when detecting that the algorithm is stuck on a large plateau of local optima. We show that the latter variant outperforms standard bit mutations by escaping the local optima on a multimodal function from the literature and that both variants outperform static hypermutations by escaping the local optima of the standard CLIFF benchmark function by accepting solutions of lower fitness.

Index Terms—Artificial immune systems, heavy-tailed mutations, inversely proportional hypermutations, theory, runtime analysis

I. INTRODUCTION

Three recent hot topics in the theory of randomised search heuristics for optimisation in general and in artificial intelligence in particular have been the benefits of mutation operators with higher mutation rates than traditionally employed [1]–[4] (or standard operators using higher parameter values than those previously recommended [5]–[10]), the automatic adaptation of the mutation rate [11]–[15] and the analysis of when non-elitist search heuristics are beneficial compared to elitist ones [16]–[24].

Artificial Immune Systems (AIS) are a class of bio-inspired algorithms which naturally use both high mutation rates via so-called hypermutation operators, often automatically adapt the mutation rate, and use non-elitism implicitly through the use of ageing operators, characteristics that are typical in the immune system of vertebrates from which AIS draw inspiration. In this paper we analyse the performance of an adaptive mutation operator called Inversely Proportional Hypermutation (IPH) that aims to increase the mutation rate with (an estimate of) the distance from the optimum, and propose a considerably improved one for efficient unimodal and multimodal optimisation.

AIS for optimisation [25], [26] are generally inspired by the clonal selection principle [27]. For this reason they are also often referred to as *clonal selection* algorithms [25]. Clonal selection algorithms are also *popular* in evolutionary multi-objective optimisation [28]–[32]. In the literature two key features of clonal selection algorithms have been identified [33]:

- 1) The proliferation rate of each immune cell is proportional to its affinity (similarity) with the selective antigen (pathogen): the higher the affinity, the higher the number of offspring generated (clonal selection and expansion).
- 2) The mutation suffered by each immune cell during reproduction is inversely proportional to the affinity of the cell receptor with the antigen: the higher the affinity, the smaller the mutation (affinity maturation).

Indeed, well-known clonal selection algorithms employ mutation operators applied to immune cells (i.e., candidate solutions) with a rate that decreases with their similarity to the antigen (i.e., global optima) during affinity maturation. Often such operators are referred to as *inversely proportional hypermutations* (IPH). Two of the most popular clonal selection algorithms, Clonalg [34] and the Opt-IA immune algorithm [35] (as well as their many variants [36]) both employ inversely proportional hypermutations. The ideal behaviour of the IPH operator is that the mutation rate is minimal in proximity of the global optimum and increases as the difference between the fitness of the global optimum and that of the candidate solution increases. However, achieving such behaviour in practice may be problematic because the fitness of the global optimum is usually unknown. As a result, in practical applications information about the problem is used to find bounds on (or estimates of) the value of the global optimum¹. Thus, the closer the estimate is to the actual value of the global optimum, the closer should the behaviour of the IPH operator be to the desired one. On the other hand, if the bound is much higher (e.g., for a maximisation problem) than the true value, then there is a risk that the mutation rate is too high in proximity of the global optimum i.e., the algorithm will struggle to identify the optimum.

While decreasing the step size as the optimum is approached is a common practice in continuous optimisation [37], previous theoretical analyses for combinatorial optimisation, though, have highlighted various problems with IPH operators from the literature even when the fitness value of the global optimum is known. Zarges [38] analysed the effects of mutating

¹Alternatively, the fitness of the best candidate solution is sometimes used and the mutation rate of the rest of the population is inversely proportional to the best.

candidate solutions with a rate that is inversely proportional to their fitness for the ONEMAX problem. She considered two different rates for the decrease of the mutation rate as the fitness increases: a linear decay (i.e., each bit is flipped with probability $\frac{\text{OPT}-\text{ONEMAX}(x)}{\text{OPT}}$, where OPT is the value of the optimum) and an exponential decay (i.e., each bit flips with probability $e^{-\rho \frac{\text{ONEMAX}(x)}{\text{OPT}}}$ where ρ is called the *decay* parameter). The motivation behind these choices are that the former operator flips in expectation exactly the number of bits that maximises the probability of reaching the optimum in the next step while the latter is the operator used in the Clonal algorithm. She showed that if the optimum of ONEMAX is known, then an algorithm employing such mutation operators will require exponential time to optimise ONEMAX with overwhelming probability (w.o.p.) in both cases of linear or exponential decays of the mutation rate. The reason is that the initial random solutions that have roughly half the fitness (i.e., $n/2$) of the global optimum (i.e., n) have very high mutation rates. Such rates do not allow the algorithm to make any progress with any reasonable probability even if the decay parameter ρ is chosen very carefully (i.e., $\rho = \ln n$ leads to reasonable mutation rates between $1/n$ and $1/2$ during the optimisation of ONEMAX). Hence, the behaviour of the mutation operator leads to very inefficient performance even for the simple OneMax function when the optimum is assumed to be known.

On the other hand, the Opt-IA clonal selection algorithm, that uses a mutation operator called 'hypermutations with mutation potential' which also has very high mutation rates has been proven to be efficient by employing a selection strategy called *stop at first constructive mutation* (FCM). The strategy evaluates the fitness after each bit is flipped and interrupts the hypermutation immediately if an improvement is detected. Otherwise it returns the bit-string obtained after it has flipped the maximum number of bits it is allowed to flip i.e., the mutation potential. Indeed, the operator using a static mutation potential (i.e., a linear number of bits are flipped unless an improvement is found along the way) has been proven to be very effective at escaping local optima of standard multimodal benchmark functions [39] and at finding arbitrarily good approximations for the Number Partitioning NP-Hard problem [40], [41] at the expense of being slower at hillclimbing during the exploitation phases - it is up to a linear factor slower than the standard bit mutation (SBM) used by evolutionary algorithms for easy unimodal functions such as OneMax and LeadingOnes. Hence, the first insight we provide in this paper is that, differently from other clonal selection algorithms (such as Clonalg and Opt-IA without FCM), hypermutations with mutation potential coupled with FCM can cope with the desired behaviour of inversely proportional mutation rates by being capable of making progress efficiently with high mutation rates. In this paper we first consider whether Opt-IA may become faster during the exploitation phases if inversely proportional mutation potentials are applied rather than static ones. The reason to believe this is that the hypermutations waste many fitness function evaluations during exploitation, since the probability of flipping bits that lead to an improvement decreases when the optimum is approached.

Hence, with high probability the operator flips wrong bits at the beginning of a hypermutation and ends up wasting a linear number of fitness function evaluations in each hypermutation. On the other hand, if the mutation potential decreases as the optimum is approached, then the amount of wasted evaluations should decrease accordingly.

A previous runtime analysis for OneMax of the inversely proportional hypermutations with mutation potential used by Opt-IA [42] has shown that the mutation rate is always in the range $[(c/2)n, cn]$ where $M = cn$ is the highest mutation potential. Hence it does not decrease proportionally with the decrease in distance to the optimum as desired and, as a result, no asymptotic speed-ups are achieved compared to static mutation potentials.

In this paper we first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations, if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. To show this, we analyse the IPH operators for standard unimodal benchmark functions where the performance of static mutation potentials is well-understood [39]. We first identify what speed-ups may be hoped for by analysing the operators in the ideal situation where the location of the optimum is known. A result of this first analysis is that a mutation potential that increases exponentially with the Hamming distance to the optimum provides the larger speed-ups for hillclimbing unimodal functions. Furthermore, using the Hamming distance rather than the fitness as the measure to quantify proximity to the optimum makes the operator robust to fitness function scaling. However, an exponential decay with the distance to the optimum also implies that high mutation rates would occur only when the current solution is prohibitively far away from the optimum, *thus, the algorithm would flip just one bit most of the time, i.e., very rare hyper-mutations*. For instance, a randomly generated solution with expected Hamming distance $n/2$ from the optimum flips $\sqrt{n}$ bits and this mutation will decrease exponentially fast with further improvements. For this reason we find that a linear decrease of the mutation rate is a better compromise in *practice* as it allows for longer phases where large mutations occur and it flips the number of bits that increase the probability of hitting the optimum in the next step.

Afterwards we propose a clonal selection algorithm that we call (1+1) Opt-IA that uses IPH and ageing² [35] to be applied in practical unimodal and multimodal applications where the optimum is not known. The algorithm uses the best solution it has encountered to estimate the mutation rates. In the literature it has been shown that ageing allows algorithms to escape from local optima either by identifying a new slope of increasing fitness or by completely restarting the optimisation process if it cannot escape [41], [42], [45]–[47]. The idea behind our proposed AIS is that the more the search space is explored, the better is the ideal behaviour of the IPH approximated through the discovery of better and better local optima. We propose two variants. The first one, called (1 + 1) Opt-IA_{LASTBEST}, updates the location of the best seen solution whenever a new solution with fitness value at least as good as the previous one

²The well-known Opt-IA AIS uses both operators in combination [35].

TABLE I: Expected runtimes of the different mutation operators when embedded in a (1+1) algorithmic framework for the benchmark functions considered in this paper. For all results the optimum is unknown to the algorithms. The * indicates that we do not find it worth the effort to perform a formal analysis, yet for reasons provided at the end of Section VI, the upper bound is correct. $\beta := \log_2 \frac{20}{c}$, where $c \leq 1$ is the constant factor in the M_{linHD} potential $M = \lfloor c \cdot H(x, \text{best}) \rfloor$.

(1+1) Algorithms	ONEMAX	LEADINGONES	CLIFF _d	LEADINGTRAPJUMP _{k, ρ}
EA (SBM)	$\Theta(n \log n)$ [43]	$\Theta(n^2)$ [43]	$\Theta(n^d)$ [44]	$n^{\Omega(\log n)}$
Opt-IA _{STATIC}	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]	$O(\frac{n^{d+1} \cdot e^d}{d^d})$ [39]	$O(n\tau + n^{2k+3} + n^{\beta+1})^*$
Opt-IA _{LASTBEST}	$\Theta(n \log n)$	$\Theta(n^2)$	$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$	$2^{\Omega(n)}$
Opt-IA _{FIRSTBEST}	$\Theta(n \log n)$	$\Theta(n^3)$	$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$	$O(n\tau + n^{2k+3} + n^{\beta+1})$

is identified. This guarantees that the mutation rate is minimal on the optimum once it is identified, exactly in the spirit of inversely proportional hypermutations, and allows for the best possible expected runtimes when hillclimbing unimodal functions such as ONEMAX and LEADINGONES. However, the Opt-IA using such an operator loses power compared to static hypermutations because it cannot escape from the best identified local optima via large mutations as its mutation rate will be minimal. To this end, we define a second variant called (1 + 1) Opt-IA_{FIRSTBEST}, that only updates the location of the best seen solution when a strictly better solution is identified. This algorithm is capable of increasing its mutation rate again when stuck on large plateaus of local optima by increasing its Hamming distance from the best seen solution.

We first use the CLIFF benchmark function to prove the effectiveness of the IPH over static hypermutations. In particular, we show that the IPH can escape local optima when coupled with ageing, because this operator allows the algorithm to escape the local optimum by accepting solutions of inferior fitness. We remark that static hypermutations cannot optimise the function efficiently because the operator only stops if a constructive mutation is identified, thus will not return inferior solutions that may survive the ageing operator and allow the algorithm to escape the local optima of the function class. This makes a difference between polynomial and superpolynomial runtimes for super-constant cliff distances, and an expected runtime of $O(n^2 \log n)$ versus exponential for linear cliffs. A different problem would occur if hypermutations inversely proportional to fitness were to be used. Since the difference in fitness between the local optima and the bottom of the Cliff is very large, the operator's mutation rate at the bottom of the cliff would be too high leading the algorithm back to the local optima with high probability.

We conclude the paper by providing an example benchmark function where it is necessary to escape from local optima using large mutations. Thus the (1 + 1) Opt-IA_{FIRSTBEST} is efficient while traditional standard bit mutations (SBM) that use lower mutation rates are not. The benchmark function, called LEADINGTRAPJUMP, is inspired from one previously used in the literature to highlight when high mutation rates are beneficial and when not [48]. Essentially, by switching the fitness values of the local optima and the global optimum one can provide examples where SBM outperforms high mutation

rates and viceversa. Herein we show that the (1 + 1) Opt-IA_{FIRSTBEST} can identify the global optimum in both cases.

A summary of the results obtained in the paper is provided in Table I highlighting the speed-ups achieved by the two IPH operators compared to the standard bit mutations traditionally used by evolutionary algorithms and to the static hypermutation operator with FCM. It can be noticed how the (1 + 1) Opt-IA_{FIRSTBEST} is a better compromise to hillclimb faster and allow escaping local optima both by accepting solutions of inferior fitness *and* by performing large jumps in the search space making a difference between polynomial and super-polynomial runtimes.

The original extended abstract [49] of this paper analysed the advantages of using an exponential decay of the mutation rate since this provides the largest speed-ups for unimodal functions in ideal conditions where the optimum is known (see Section III). However, a great disadvantage of the exponential decay is that large mutations hardly ever occur in practice. Indeed no superiority of IPH over a SBM operator was provided. To this end, in this paper we analyse the use of the linear decay operator and provide examples where the IPH indeed considerably outperforms SBM providing a difference between polynomial and superpolynomial expected runtimes. Due to space restrictions a self-contained paper presenting all proofs is provided as supplementary material.

II. INVERSELY PROPORTIONAL MUTATION POTENTIALS

Hypermutations with mutation potential using FCM flip at most a linear amount of bits (i.e., the mutation potential $M = cn$) chosen uniformly at random without replacement from the interval $[1..n]$. After each of the M bit flips, the fitness of the produced solution is evaluated and if a *constructive mutation* is identified, then the operator is stopped and the solution returned. A constructive mutation is a solution that is at least as good as that of the original parent. Otherwise the algorithm continues flipping the bits without setting them back until the potential M of bit flips is reached. If no constructive mutation has been identified earlier, then the M -th solution is returned. The original inversely proportional hypermutation operator (IPH) proposed for Opt-IA uses mutation potential $M = \lceil (1 - f_{\text{OPT}}/v)cn \rceil$ for minimisation problems, where $f_{\text{OPT}} \geq 0$ is the optimal fitness value, $v > 0$ is the fitness of the current individual, and $0 < c \leq 1$ is some constant [35],

[42]. In an analysis for $\text{ZEROMIN} := n + 1 - \text{ONEMAX}$ [42], such a mutation potential was shown not to decrease below $cn/2$, with cn being the highest possible mutation potential. As a result no speed-ups at hillclimbing are achieved compared to static hypermutations as the mutation potential is always at least linear. In this section, we introduce three different IPH operators that will be analysed in this paper and that provably speed-up the hillclimbing performance for ONEMAX. Two have been already considered in the literature while the third one is newly proposed by us based on analysed drawbacks of the first two at speeding up hillclimbing.

Hamming Distance Based Linear Decrease

Zarges analysed an inversely proportional hypermutation operator where the probability of flipping each bit increases linearly with the Hamming distance to the optimum or its best available estimate [38]. Precisely, this operator flips each bit with probability $H(x, \text{best})/n$ where n is the size of the problem and $H(x, \text{best})$ is the Hamming distance of the current point to the best individual (or an estimate if not available). As the expected number of bit-flips is $H(x, \text{best})$ during each execution of a mutation operator, a mutation potential inspired by this inversely proportional hypermutation is:

$$M_{\text{linHD}}(x) := H(x, \text{best}).$$

Fitness Difference Based Exponential Decrease

In Clonalg's IPH, the mutation probability decreases as an exponential function of the fitness of the current solution [34]. Precisely, each bit flips with probability $e^{-\rho \cdot v}$ where v is the normalised fitness value and ρ is a decay parameter that regulates the speed at which the mutation rate decreases. Since we consider maximisation problems, we use $v = \frac{f(x)}{f(\text{best})}$ as suggested by [38] where $f(\text{best})$ is the best known fitness value. Using the expected number of bits flipped by this mutation operator as a mutation potential gives $M = n \cdot e^{-\rho v}$. According to both experimental and theoretical analyses [38], a reasonable value for ρ is $\ln n$. We call this mutation potential $M_{\text{expoF}(x)}$ and define it as:

$$M_{\text{expoF}(x)}(x) := \left\lfloor n^{1 - \frac{f(x)}{f(\text{best})}} \right\rfloor.$$

Hamming Distance Based Exponential Decrease

It is well understood that fitness-proportional selection is sensitive to the difference between the fitness values of candidate solutions, i.e., it struggles to distinguish between solutions that have similar, yet different, fitness values [50]–[53]. For this reason, nowadays selection operators that rank individuals by fitness (e.g., tournament, linear ranking, comma selection) are generally preferred. We also consider a measure that avoids using the fitness values directly because, apart from the above consideration, the ideal amount of bits to be flipped clearly depends on the genotypic distance of the candidate solution to the optimum rather than on their difference in fitness values. To this end, we suggest a mutation potential which is similar to $M_{\text{expoF}(x)}$ with the exception that it uses the

normalised Hamming distance to the best estimate rather than the normalised fitness. We call this mutation potential M_{expoHD} and define it as $M_{\text{expoHD}} = \left\lfloor n \cdot e^{-\rho \frac{n - H(x, \text{best})}{n}} \right\rfloor$ where n is the problem size (the maximum Hamming distance between any two points), $H(x, \text{best})$ is the Hamming distance to the best known solution and ρ is the decay of the mutation potential. For the choice of $\rho = \ln n$, we get:

$$M_{\text{expoHD}}(x) := \left\lfloor n^{\frac{H(x, \text{best})}{n}} \right\rfloor.$$

III. PERFORMANCE GAINS IN IDEAL CONDITIONS

In this section, we evaluate the performance of the three different IPH operators, assuming the optimum is known (i.e., $\text{best} = \text{opt}$). Under this assumption, the operators exhibit their ideal behaviour (i.e., the mutation potential decreases with the desired rate as the optimum is approached). Our aim is to evaluate what speed-ups can be achieved in ideal conditions compared to the well-studied static mutation potentials. To achieve such comparisons we perform runtime analyses of the (1+1) IA using the IPH on the standard ONEMAX and LEADINGONES unimodal benchmark functions for which the performance of the same algorithm using static mutation potentials is known [39]. The *simple-to-define* ONEMAX function counts the number of 1-bits in a bit string and is normally used to evaluate the hill climbing ability of algorithms [5], [9], [54], [55]. On the other hand, LEADINGONES is a slightly harder unimodal problem which counts the consecutive number of 1-bits at the beginning of the bit string before the first 0-bit *or the end of the string*. Evolutionary algorithms using standard bit mutation (or bit flip local search mutation) optimise the functions respectively in $\Theta(n \log n)$ and $\Theta(n^2)$ expected fitness function evaluations [12], [13], [43], [56], [57], while static hypermutations are a linear factor slower [39].

While in practice both evolutionary algorithms and artificial immune systems use a population of solutions, in this paper we consider single trajectory algorithms to better highlight the performance of all the considered hypermutation operators. The pseudo-code of a simple AIS, the (1+1) Immune Algorithm ((1+1) IA), is given in Algorithm 1. It simply uses one candidate solution in each iteration to which inversely proportional hypermutations are applied. For the function $\text{Hypermutate}(x)$, IPH performs two steps: 1) it calculates the mutation potential M , and 2) it creates y by flipping at most M distinct bits of x selected uniformly at random without replacement one after another until a constructive mutation happens. Otherwise it returns the last constructed solution. A constructive mutation is a mutation step where the evaluated bit string is at least as fit as the parent x . The pseudo-code of the operator once the mutation potential M has been calculated is depicted in Algorithm 2. The results proven in this section and comparisons with static hypermutations and the IPH operators from the literature are summarised in Table II.

Before stating the main results, we first introduce two lemmata. The first is the Ballot theorem which is used throughout this paper to derive lower bounds on the expected runtime of the algorithms [39], [42], [58]. This theorem is essentially

TABLE II: Comparison of the expected runtimes obtained by different hypermutation schemes for ONEMAX and LEADINGONES. The original IPH operator of Opt-IA [35] has the same expected runtimes as the $(1+1)$ IA using static hypermutations [42]. ‘*’ indicates that the result can be easily drawn from the analysis for ONEMAX [38].

$(1+1)$ IA	ONEMAX	LEADINGONES
Using static hypermutation	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]
Clonalg with linear decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)*}$
Clonalg with expo decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)*}$
Traditional Opt-IA	$\Theta(n^2 \log n)$ [42]	$\Theta(n^3)$ [42]
FCM with M_{linHD}	$\Theta(n^2)$	$\Theta(n^3)$
FCM with $M_{\text{expoF}(x)}$	$\Theta(n^{3/2}/\log n)$	$\Theta(n^3/\log^2 n)$
FCM with M_{expoHD}	$\Theta(n^{3/2}/\log n)$	$\Theta(n^{5/2}/\log^2 n)$

Algorithm 1 $(1+1)$ IA

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: Evaluate  $f(x)$ ;
3: while termination condition is not satisfied do
4:   Update mutation potential  $M$ ;
5:    $y := \text{Hypermutate}(x, M)$ ;
6:   if  $f(y) \geq f(x)$  then
7:      $x := y$ .
8:   end if
9: end while

```

used to show the probability of picking more 1-bits than 0-bits during one hypermutation operation given that there are more 1-bits in the initial bit-string (or vice-versa).

Lemma 1 (Ballot Theorem [58]). *In a ballot, suppose that candidate P receives p votes and candidate Q receives q votes such that $p > q$. The probability that throughout the counting P is always ahead of Q is $(p - q)/(p + q)$.*

In the context of hypermutation operators, the vote counts are the initial number of 0 and 1-bits in the solutions, while counting the votes correspond to flipping them until the first constructive mutation. The probability that a constructive mutation is found is equivalent to the probability of the Q candidate having as many votes counted as the other during any time in the counting process.

Our second lemma will be used to transform inverse polynomial exponents that arise from the exponential potential.

Lemma 2. *For large enough n and any arbitrary constant ϵ , $n^{1/n^\epsilon} = (1 + \frac{\ln n}{n^\epsilon})(1 \pm o(1))$.*

Proof. $n^{\frac{1}{n^\epsilon}} = e^{\frac{\ln(n)}{n^\epsilon}} = 1 + \frac{\ln(n)}{n^\epsilon} + o(\frac{\ln(n)}{n^\epsilon})$ by the Taylor’s series of the exponent. $\square$

A. Performance for ONEMAX in Ideal Conditions

The following theorems derive the expected runtimes using each of the three IPH operators for the function

Algorithm 2 HYPERMUTATE(x, M)

```

1:  $y \leftarrow x$ ;
2: for  $i = 1$  to  $M$  do
3:   Flip one previously unflipped bit of  $y$  chosen uniformly at random;
4:   Evaluate  $f(y)$ ;
5:   if  $f(y) \geq f(x)$  then
6:     return  $y$ ;
7:   end if
8: end for
9: return  $y$ ;

```

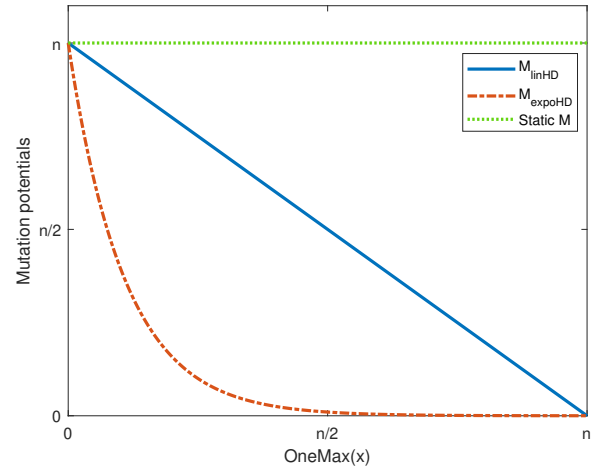


Fig. 1: Static and inversely proportional mutation potentials for ONEMAX.

$\text{ONEMAX}(x) := \sum_{i=1}^n x_i$. Figure 1 shows how the studied mutation potentials ideally decrease during the run of the algorithm when optimising the function. Notice that for ONEMAX the behaviours of $M_{\text{expoF}(x)}$ and M_{expoHD} are the same. By decreasing the potential linearly with the decrease of the Hamming distance, a logarithmic factor may be shaved off from the expected runtime of the $(1+1)$ IA compared to the expected runtime achieved by the same algorithm using static hypermutation potentials.

Theorem 1. *The $(1+1)$ IA using IPH with M_{linHD} optimises ONEMAX in $\Theta(n^2)$ expected fitness function evaluations.*

Proof. Considering i as the number of 0-bits in the candidate solution, the probability of improvement in the first step is i/n . Knowing that at most n improvements are needed to find the optimum and that in case of failure $H(x, \text{opt}) = i$ fitness function evaluations would be wasted, the total expected time to optimise ONEMAX is at most $\sum_{i=1}^n \frac{n}{i} \cdot i = O(n^2)$.

For the lower bound, we use the Ballot theorem (Lemma 1). We flip at most $M \leq n$ bits, and the probability that an improvement happens is the probability that at some moment there will be at least as many flipped zeros as there are flipped ones. So pessimistically we can assume that $M = n$ bits are always flipped, since it maximizes the chances of this unfortunate event.

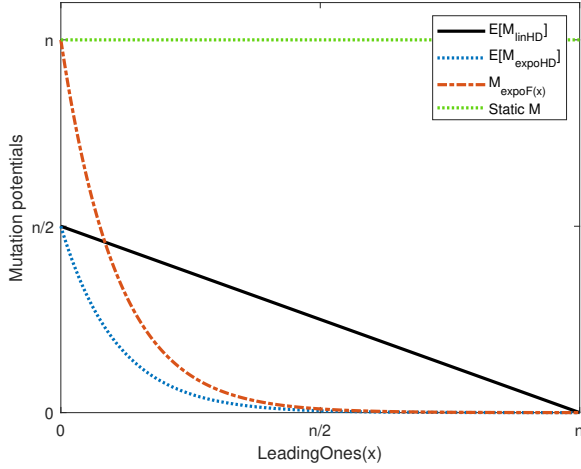


Fig. 2: Static and inversely proportional mutation potentials for LEADINGONES. Expected values are shown for those using Hamming distance.

By Chernoff bounds, the number of 0-bits in the initialised solution is at least $n/3$ w.o.p. Considering the number of 0-bits as $i = q$ and the number of 1-bits as $n - i = p$, the probability of an improvement is at most $1 - (p - q)/(p + q) = 1 - (n - 2i)/n = \frac{2i}{n}$ by the Ballot theorem, where $i = H(x, \text{opt})$. This means that we need to wait at least $\frac{n}{2i}$ iterations to see an improvement and each time the mutation operator fails to improve the fitness ($\frac{n}{2i} - 1$ times in expectation), i fitness function evaluations will be wasted. Considering that at least $n/3$ improvements are needed, the expected time to optimise ONEMAX is larger than $(1 - 2^{-\Omega(n)}) \sum_{i=1}^{n/3} (\frac{n}{2i} - 1) \cdot i = \Omega(n^2)$ since the operator cannot improve the ONEMAX function value by more than one in each application due to FCM. The factor $(1 - 2^{-\Omega(n)})$ accounts for the w.o.p. Chernoff event; since $2^{-\Omega(n)} = o(1)$, the bound holds unconditionally. $\square$

The following theorem shows that a greater speed up may be achieved if the potential decreases exponentially rather than linearly. Note that for ONEMAX the Hamming distance of a solution to the optimum and its difference in fitness are the same. Hence, there is no difference between the behaviours of the algorithm when using $M_{\text{expoF}(x)}$ and M_{expoHD} . Figure 1 highlights the reason why these operators waste fewer fitness evaluations.

Theorem 2. *The $(1 + 1)$ IA using IPH with either $M_{\text{expoF}(x)}$ or M_{expoHD} optimises ONEMAX in $\Theta(n^{3/2}/\log n)$ expected fitness function evaluations.*

B. Performance for LEADINGONES in Ideal Conditions

In the previous subsection, it was shown that decreasing the mutation rate linearly with the Hamming distance to the optimum allows for a logarithmic factor speed-up for ONEMAX compared to using static hypermutations. The following theorem shows that no asymptotic improvement over static mutation potentials is achieved for LEADINGONES: $\sum_{i=1}^n \prod_{j=1}^i x_i$. The main reason for the lack of improvement

is that a linear mutation potential is sustained until at least a linear number of improvements are achieved.

Theorem 3. *The $(1 + 1)$ IA using IPH with M_{linHD} optimises LEADINGONES in $\Theta(n^3)$ expected fitness function evaluations.*

We can now proceed with the analysis of the exponential-based potentials. Theorem 4 shows that the fitness difference based exponential mutation potential provides a $\Theta(\log^2 n)$ factor speed-up compared to static mutation potentials.

Theorem 4. *The $(1 + 1)$ IA using IPH with $M_{\text{expoF}(x)}$ optimises LEADINGONES in $\Theta(n^3/\log^2 n)$ expected fitness function evaluations.*

An advantage of Hamming distance-based exponential decays of the mutation potential compared to fitness-based ones is provided by the following theorem for LEADINGONES. The reason can be seen in Figure 2. While the initial fitness is very low, the potential is very high, but the actual number of bits that have to be flipped to reach the optimum (i.e., the Hamming distance) is much smaller. More precisely, fitness-based potentials suggest to flip n bits when only $n/2$ bits have to be flipped in expectation to reach the optimum in one step. M_{expoHD} exploits this property, thus wastes fewer fitness evaluations than $M_{\text{expoF}(x)}$.

Theorem 5. *The $(1 + 1)$ IA using IPH with M_{expoHD} optimises LEADINGONES in $\Theta(n^{5/2}/\log^2 n)$ expected fitness function evaluations.*

Overall, in this section, we have shown that considerable speed-ups can be achieved in the ideal condition where the optimum is known by using inversely proportional hypermutations compared to static ones. We conclude that M_{expoHD} provides larger hill-climbing speed-ups compared to the other mutation potentials and is stable to the scaling of fitness functions. However, M_{expoHD} only allows to flip at most $\sqrt{n}$ bits when far away from the optimum (e.g., at initialisation for ONEMAX), which means that the algorithm hardly ever performs large mutations except at initialisation defeating the purpose of having a hypermutation operator at all. Thus, we believe that M_{linHD} , while being slower during the hillclimbing phases, provides a better balance between high and low mutation rates for hard multimodal functions when the total runtime is reasonably dominated by the time required to escape from local optima. Notice from Table II that using a linear decay operator with FCM rather than in Clonalg still makes a difference between polynomial and exponential time for the functions when the optimum is known. In the following section we will highlight how being able to perform large mutations more often allows the algorithm to be effective in practice when the optimum is unknown for multimodal functions with very different characteristics.

IV. BEHAVIOUR OF IPH IN PRACTICE

In this section, we consider the usage of IPH in realistic applications where the optimum is unknown. To this end, the mutation rate of the IPH operator will be inversely proportional to the best seen solution rather than to the unknown optimum.

We combine IPH using M_{linHD} with hybrid ageing, as in the Opt-IA AIS [35], and embed it in a $(1+1)$ Opt-IA shown in Algorithms 3 and 4. To use an ageing operator, an age $\alpha(x)$, is assigned to the individual x . After the Hypermutate(x) operator, the ageing operator is applied. Among the different variants of ageing, *hybrid* ageing has been shown to be very efficient at escaping local optima [39], [40], [45], [59]. Using this operator, individuals are assigned with initial age zero. During each iteration of the algorithm, the age increases by 1 and is passed to the offspring if it does not improve over its parent's fitness. If the offspring is fitter than the parent, then its age is set to 0. At the end of each iteration, any individual with age larger than a threshold τ is removed with probability $p_{\text{die}} = 1/2$. In case there is no other individual left, a new individual is initialised uniformly at random³.

Ageing has been shown to enable algorithms to escape from local optima either by identifying a gradient leading away from it or by restarting the whole optimisation process. Our aim behind the algorithm design is that by escaping local optima via ageing, the number of local optima that are identified by the algorithm increases over time. Thus as time goes by the IPH should approximate its ideal behaviour better and better.

Concerning the IPH operator for practical applications we reintroduce the parameter $c \leq 1$ for determining the mutation potential when it grows linearly with the Hamming distance as in the original Opt-IA [42]:

$$M_{\text{linHD}} = \max(\lfloor c \cdot H(x, \text{best}) \rfloor, 1)$$

The parameter will be useful to avoid always ending up in the same basins of attraction as we will see in the analyses of both multimodal function classes we consider in this paper.

We also have a design choice regarding the IPH operator of whether to update the *best* seen solution only when a strictly better solution is identified or whether to update it when the algorithm finds a new solution that is as good as the previous best one. This design choice leads to considerably different algorithmic behaviours. If the current *best* is updated to the new solution when individuals of equal fitness are encountered, then the algorithm will be extremely fast on unimodal functions because the mutation rate will always be the smallest possible (i.e., 1-bit flips). So if the aim is to optimise easy unimodal functions then this version, which we call $(1+1)$ Opt-IA_{LASTBEST} is the preferred choice, as it can be proved to run in optimal asymptotic runtime on benchmark functions such as ONEMAX (i.e., $\Theta(n \log n)$) and LEADINGONES (i.e., $\Theta(n^2)$). However, this choice would mean that the algorithm would keep low mutation rates when trapped in the best found local optima if they are encountered again and again with high probability after the algorithm is restarted.

The other choice, that of only updating the *best* when a strictly better solution is encountered, allows individuals trapped on a large plateau of local optima to increase their Hamming distance to the best solution, thus increases the chances of escaping via higher mutation rates. This version,

³When a population is used, the required number of individuals are re-initialised to reach the population size again after ageing is applied.

Algorithm 3 $(1+1)$ Opt-IA_{LASTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ ;
3: evaluate  $f(x)$ ;
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ ;
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ ;
9:   else
10:     $\alpha(y) := \alpha(x)$ ;
11:   end if
12:   if  $f(y) \geq f(\text{best})$  then
13:     then set  $\text{best} := y$ ;
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
19:       if  $f(w) \geq f(\text{best})$  then
20:          $\text{best} := w$ 
21:       end if
22:     end if
23:   end for
24:   if  $f(y) \geq f(x)$  then
25:      $x := y$ 
26:   end if
27: end while

```

which we call $(1+1)$ Opt-IA_{FIRSTBEST}, allows to use high mutation rates more often at the expense of being slower on some unimodal functions like LEADINGONES as we will see. With this operator we remember to update the *best* to the last seen (best) individual of highest fitness (i.e. the current solution) when ageing is triggered because this limits the Hamming distance between the new solution and the current best when a local optima is escaped by accepting solutions of lower fitness, thus decreases the chances of jumping back to the previous local optimum.

We start with a general reduction. When the mutation potential remains 1 and ageing is unlikely to trigger, both algorithm variants execute single-bit flips with accept-if- $\geq$ and thus behave identically to RLS. Lemma 3 below formalises this observation; we then apply it to show that both algorithms optimise ONEMAX in $\Theta(n \log n)$ and that $(1+1)$ Opt-IA_{LASTBEST} optimises LEADINGONES in $\Theta(n^2)$.

Lemma 3 (RLS Reduction). *Let A denote the event that ageing triggers before the optimum is found and set $p := \Pr[A]$. If (i) $M = 1$ throughout every run in which A does not occur, and (ii) $p = o(1)$ and $\tau p = o(1)$, then $\mathbb{E}[T'] = (1 \pm o(1)) T_{\text{RLS}}$, where T_{RLS} is the expected runtime of RLS on the same function.*

Theorem 6. *The expected runtime of $(1+1)$ Opt-IA_{FIRSTBEST} and $(1+1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising ONEMAX is $\Theta(n \log n)$ with $\tau \geq C n \ln n$ for*

Algorithm 4 $(1 + 1)$ Opt-IA_{FIRSTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ 
3: evaluate  $f(x)$ 
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ 
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ 
9:   else
10:     $\alpha(y) := \alpha(x)$ 
11:   end if
12:   if  $f(y) > f(\text{best})$  then
13:      $\text{best} := y$ 
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         if  $f(w) = f(\text{best})$  then
19:            $\text{best} := w$ 
20:         end if
21:       reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
22:       if  $f(w) > f(\text{best})$  then
23:          $\text{best} := w$ 
24:       end if
25:     end if
26:   end for
27:   if  $f(y) \geq f(x)$  then
28:      $x := y$ 
29:   end if
30: end while

```

a sufficiently large constant C .

Proof. Immediately after initialisation $\text{best} = x$, so $H(x, \text{best}) = 0$ and $M = c \cdot H = 0$; since the potential is at least 1 by convention, $M = 1$. On ONEMAX, every bit flip changes fitness by ± 1 , so no neutral moves exist; every accepted solution strictly improves fitness and triggers $\text{best} \leftarrow x$ in both variants, maintaining $H = 0$ and $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), at any level $k \geq 1$ the improvement probability is $k/n \geq 1/n$, so the probability of τ consecutive non-improvements is at most $(1 - 1/n)^\tau \leq e^{-\tau/n} \leq e^{-C \ln n} = n^{-C}$. A union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$ for $C > 1$, and $\tau \cdot p \leq Cn \ln n \cdot n^{1-C} = Cn^{2-C} \ln n = o(1)$ for $C > 2$. Thus both conditions of Lemma 3 hold for $C > 2$.

By Lemma 3, $\mathbb{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. Since $T_{\text{RLS}} = \sum_{k=1}^n n/k = \Theta(n \log n)$ (lower bound: at least $n/3$ zeros initially w.o.p. by Chernoff, giving $\sum_{k=1}^{n/3} n/k = \Omega(n \log n)$), the result follows. $\square$

In the following two theorems we show how $(1 + 1)$ Opt-IA_{LASTBEST} achieves a linear factor speed-up over $(1 + 1)$ Opt-IA_{FIRSTBEST} for LEADINGONES, because the latter increases

its mutation rate on plateaus of equal fitness before finding each improvement.

Theorem 7. *The expected runtime of $(1 + 1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^2)$ with $\tau \geq Cn \ln n$ for a sufficiently large constant C .*

Proof. Since $(1 + 1)$ Opt-IA_{LASTBEST} accepts equal-or-better solutions and updates best on any acceptance, every accepted move (whether improving or neutral) restores $H(x, \text{best}) = 0$, so $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), the improvement probability at every level is exactly $1/n$ (flip the leftmost 0-bit), so $(1 - 1/n)^\tau \leq n^{-C}$ and a union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$. Moreover $\tau \cdot p \leq Cn^{2-C} \ln n = o(1)$ for $C > 2$, verifying condition (ii).

By Lemma 3, $\mathbb{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. For RLS on LEADINGONES, the unique improving move at each of the n levels is the leftmost 0-bit flip, with probability $1/n$; neutral tail flips are accepted but do not advance the level. Hence the expected time per level is n and $T_{\text{RLS}} = n^2$, giving $\mathbb{E}[T'] = \Theta(n^2)$. $\square$

Theorem 8. *The expected runtime of the $(1 + 1)$ Opt-IA_{FIRSTBEST} with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^3)$ with $\tau = \Omega(n^{1+\epsilon})$ for any arbitrary constant $\epsilon > 0$.*

Proof. Fix a level ℓ with $n/3 \leq \ell < 2n/3$. For the current individual x with $\text{LO}(x) = \ell$, we have $x_1 = \dots = x_\ell = 1$ and $x_{\ell+1} = 0$, and the remaining $m := n - \ell - 1$ positions form the tail. In each iteration, the first flipped bit is uniformly distributed over $\{1, \dots, n\}$; therefore the probability that the first flip hits the improving bit $\ell + 1$ is $1/n$, the probability that it hits the prefix is ℓ/n , and the probability that it hits the tail is m/n . Let T be the iteration on which the improving bit is first hit. Then T is geometrically distributed with parameter $1/n$, so $\mathbb{E}[T] = n$. Before this improving iteration occurs, the expected number of tail-first iterations is $\frac{m}{n-1}(\mathbb{E}[T] - 1) = \frac{m(n-1)}{n-1} = m = \Theta(n)$ (using that, conditional on not hitting the improving bit $\ell + 1$, the first flip is uniform over the remaining $n - 1$ positions, of which m are tail positions). Each such iteration flips a uniformly random tail bit. By standard coupon-collector arguments, after $\Theta(n)$ such tail flips a constant fraction of the m tail positions has been flipped exactly once with constant probability (since each hypermutation samples bits without replacement, each position is visited at most once per call). Thus, with constant probability, the Hamming distance between x and best is $\Theta(m) = \Theta(n)$, and therefore the IPH mutation potential satisfies $M = \Theta(n)$ at this level.

Consider now a subsequent iteration at the same level while $M \geq \alpha n$ for some constant $\alpha > 0$. If the first flip hits the prefix (an event of probability at least $\ell/n \geq 1/3$), then, because mutation proceeds without replacement, the lost prefix bit cannot be repaired within the same iteration. Consequently, no acceptance can occur in that iteration, and the algorithm executes the entire mutation budget M , rejecting at the end. Hence the expected number of fitness evaluations in such an

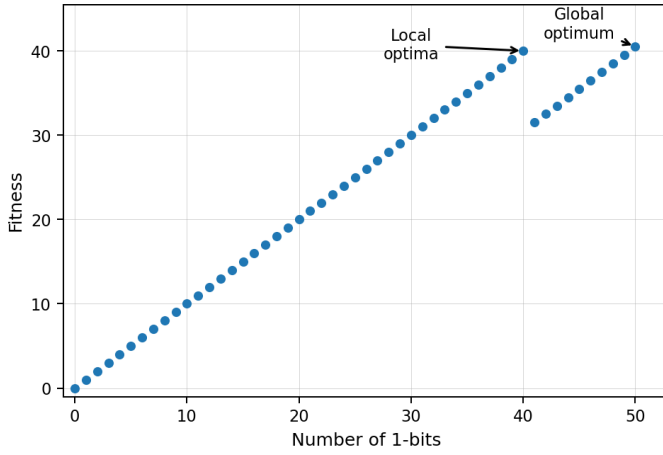


Fig. 3: The CLIFF_d function for $n = 50$, $d = 10$. The local optimum (40 one-bits, fitness 40) and global optimum (50 one-bits, fitness 40.5) are labelled.

iteration is at least αn . Since this happens with probability at least $1/3$, the unconditional expected cost of one iteration while $M = \Theta(n)$ is $\Omega(n)$.

The probability that an iteration yields an improvement is exactly $1/n$, because only a first flip that hits position $\ell + 1$ increases the LEADINGONES value. Therefore the expected number of iterations required to advance from level ℓ to $\ell + 1$ is $1/(1/n) = n$. Combining this with the previous observation, the expected number of fitness evaluations spent at level ℓ satisfies $\Omega(n) \cdot n = \Omega(n^2)$. There are $\Theta(n)$ levels ℓ in the interval $[n/3, 2n/3]$, and each such level contributes $\Omega(n^2)$ expected evaluations. Thus the total expected optimisation time is $\Omega(n^3)$. Finally, stochastic ageing can only increase the number of evaluations and therefore cannot invalidate the lower bound. Since the function has n levels, $1/n$ probability of improvement and $M \leq n$, the expected runtime is bounded from above by $O(n^3)$ as well. $\square$

In this section, we have proven the results for M_{linHD} . Naturally the same [positive](#) results follow for the M_{expoHD} operator which always has at most the same mutation rate or lower.

V. WHEN IPH OUTPERFORMS TOO HIGH MUTATION RATES

In this section we show that M_{linHD} combined with ageing can efficiently escape from local optima. We will use the well known CLIFF_d function for this purpose where static hypermutations are inefficient [39]:

$$\text{CLIFF}_d(x) := \begin{cases} \sum_{i=1}^n x_i & \text{if } \sum_{i=1}^n x_i \leq n - d, \\ \sum_{i=1}^n x_i - d + 1/2 & \text{otherwise.} \end{cases} \quad (1)$$

The class of CLIFF_d functions, illustrated in Figure 3, is usually used to assess the performance of non-elitist algorithms. It was originally designed as a problem instance where non-elitist EAs outperform elitist EAs [44]. This function has a ONEMAX slope with length $n - d$ which leads the algorithms

towards the local optima. The local optima are followed by a second ONEMAX slope (of length d) of lower fitness which leads to the global optimum. While elitist algorithms need to make a jump of size d to find the global optimum, [some](#) non-elitist algorithms can easily find the second slope by accepting inferior solutions, and then hillclimb up to the global optimum [3], [24], [39], [60], [61]. Due to the high mutation rates of static hypermutations, even if they were to escape the local optima of CLIFF_d (which is very unlikely due to FCM and linear mutation potentials), they would still jump back with high probability.

Recently [fast AISs](#) have been introduced [3] that simply flip the required number of bits independently according to the mutation potential (rather than sequentially and using FCM) in a similar fashion to power law mutation operators [4]. These have been shown to optimise CLIFF_d as long as the gap size is linear in expected time $O(n^{5/2} \log n)$.

Now we show that differently from static hypermutations, M_{linHD} combined with ageing can escape from the local optima of CLIFF_d [also for small cliff gaps](#), and [faster than the fast AISs for sublinear gaps](#), hence optimises the function efficiently. The condition that d is not prohibitively large is necessary to avoid that the reinitialised solutions due to ageing have larger fitness than a solution with $n - d + 1$ 1-bits, which would cause the latter to be removed from the population. We believe that this assumption is realistic for practical applications.

The following theorem is the main result of this section.

Theorem 9. *The $(1 + 1)$ $\text{Opt-IA}_{\text{FIRSTBEST}}$ with M_{linHD} (factor $c < \frac{1+4\epsilon}{3-4\epsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon'})$ optimises CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected*

$$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$$

fitness function evaluations.

We now present the proof sketch of our main result on the CLIFF_d function, together with all supporting lemmata. All proofs are deferred to the supplementary materials. The argument proceeds in two major stages: (i) obtaining a canonical entry point onto the second slope immediately after an ageing event at the cliff-top; and (ii) showing that from this configuration the algorithm reaches the optimum without dangerous increases in mutation potential.

For stage (ii) we will use the generalization of Markov's Inequality by Ville [62](Ch. VII) which will allow us to show that the mutation potential stays bounded with constant probability.

Theorem 10 (Ville's inequality for nonnegative supermartingales). *Let $(M_t)_{t \geq 0}$ be a nonnegative supermartingale adapted to $(\mathcal{F}_t)_{t \geq 0}$. Then for every $a > 0$,*

$$\Pr\left(\sup_{t \geq 0} M_t \geq a\right) \leq \frac{E[M_0]}{a}.$$

To establish the result, we first analyse how the algorithm reaches a highly structured “canonical” state shortly after ageing triggers at the cliff-top. At the cliff-top the algorithm

cannot improve except by sampling the global optimum, and so ageing will eventually be triggered. The key insight is that the iteration of ageing followed by two additional fitness improvements can place the algorithm into a **canonical starting configuration on the second slope**. Throughout the **CLIFF_d** analysis we index the successful fitness improvements on the second slope by $r = r_0, r_0 + 1, \dots$, and write $\text{HD}(r)$ for the Hamming distance between the current solution and *best* after r such improvements. With this notation, the canonical start corresponds to $r_0 = 2$ (two improvements completed from the cliff bottom) and $\text{HD}(r_0) = 2$ (the current solution differs from the locked *best* in exactly two positions). The following lemma formalises this.

Lemma 4. *Suppose ageing is triggered at the cliff-top (d zero-bits). The **(1 + 1) Opt-IA_{FIRSTBEST}** with M_{linHD} with $c \in (0, 1)$ reaches within at most three steps the configuration*

$$r_0 = 2, \quad \text{HD}(r_0) = 2,$$

with probability at least

$$\frac{3}{256} \left(\frac{d}{n} \right)^3.$$

The canonical condition $(r_0, \text{HD}(r_0)) = (2, 2)$ means that we begin the drift analysis two successful improvements into the second slope, at a point differing from the reference in exactly two bit positions.

Repeated ascents to the cliff-top therefore form a geometric waiting-time process. It takes at most $\sum_{i=d}^n n/i = O(n \log(n/d))$ for **RLS** to climb the first slope and the mutation potential is at most linear. Since each ascent takes $O(n^2 \log(n/d))$, the total time to obtain the canonical start is provided by the following corollary.

Corollary 1. *The expected evaluation count until the first occurrence of $(r_0, \text{HD}(r_0)) = (2, 2)$ is*

$$O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

From the canonical starting point, the key difficulty is showing that the mutation potential does not increase too much. With only few zero-bits remaining, successful steps on the second slope require flipping one of these rare zeros, whereas each $1 \rightarrow 0$ flip increases the Hamming distance and thus the mutation potential. If this potential grows too large, hypermutations become long enough that, under **FCM**($\geq$), they are increasingly likely to wander back to the cliff-top; once there, the stopping rule causes the process to lock onto the plateau and erase all progress. Hence the analysis must show that with constant probability the mutation potential stays small throughout the entire second-slope climb. This requires controlling tightly how many detrimental flips a hypermutation can accumulate before recovering to parent-level fitness. The next lemma provides the crucial starting point: an exponential tail bound for this damaging prefix.

Lemma 5. *Let the current solution on the second slope have $z \leq d$ zero-bits and define $p = z/n$. Consider a single application of the hypermutation operator; let ξ_z be the*

number of $1 \rightarrow 0$ flips made within that one hypermutation before the fitness first returns to $\geq$ the parent's fitness (i.e., before the first catch-up). Then for all $k \geq 1$,

$$\Pr(\xi_z \geq k) \leq (2p)^k.$$

Consequently, the probability that the Hamming distance increases by at least $2k$ in a single hypermutation is also at most $(2p)^k$.

A whole phase consists of all attempts at the same zero-bit count until the first success. Using the tail bound above, we obtain uniform exponential moment bounds for entire phases.

Lemma 6. *Consider a phase at zero-bit count z and let Y_z denote the total Hamming-distance increase from all failed attempts in the phase. Let $\rho = 2d/n$. For all $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$,*

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda).$$

To track the safety of the mutation potential across phases, we introduce a linear *buffer* $B_r = \theta r - s_0 - E_r$ capturing the invariant $c \text{HD}(r) \leq r$.

Lemma 7. *Let successful phases be indexed by $r = r_0, r_0 + 1, \dots$ and define*

$$s_0 = \text{HD}(r_0) - r_0, \quad \theta = \frac{1}{c} - 1, \quad E_r = \sum_{j=r_0+1}^r Y_{z_j},$$

and $B_r = \theta r - s_0 - E_r$. Then for all $r \geq r_0$:

- 1) $B_r \geq 0$ if and only if $c \cdot \text{HD}(r) \leq r$;
- 2) $B_{r+1} - B_r = \theta - Y_{z_{r+1}}$;
- 3) $B_{r_0} = \theta r_0 - s_0$.

Combining the **MGF of the buffer growth** with the buffer update yields an exponential drift envelope suitable for applying Ville's inequality for non-negative supermartingales (Theorem 10).

Lemma 8. *Let $\rho = 2d/n$. For any $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ define*

$$\frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda), \quad \alpha = e^{-\lambda \theta} K(\lambda).$$

Then for all $t \geq 0$,

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq \alpha.$$

We now apply Ville's inequality to turn such a drift envelope into a bound on barrier crossing.

Corollary 2. *If $B_0 = b > 0$ and there exist $\lambda > 0$ and $\alpha < 1$ with*

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq \alpha,$$

then

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda b}.$$

To guarantee the drift negativity required above, we identify the admissible range of the mutation-potential factor c .

Lemma 9. *Let $\rho = 2d/n$ and $\theta = \frac{1}{c} - 1$. Define*

$$c_*(n, d) = \frac{1 - \rho}{1 + \rho}.$$

If $c < c_*(n, d)$, then there exists $\lambda > 0$ such that

$$e^{-\lambda\theta} K(\lambda) < 1,$$

and hence the buffer satisfies the Ville barrier bound.

A uniform instantiation over all permitted d shows that the feasible c range has a nontrivial lower bound.

Corollary 3. If $d \leq (\frac{1}{4} - \varepsilon)n$, then

$$c_*(n, d) \geq \frac{1 + 4\varepsilon}{3 - 4\varepsilon} \geq \frac{1}{3}.$$

Finally, combining the threshold on c with the initial buffer value at $(r_0, \text{HD}(r_0)) = (2, 2)$ yields a uniform constant bound on the probability of avoiding jumping back.

Lemma 10. If $c < c_*(n, d)$ and $b = B_{r_0} = \frac{r_0}{c} - \text{HD}(r_0) > 0$, then

$$\Pr(\inf_{t \geq 0} B_t \leq 0) \leq e^{-\lambda b}$$

for some $\lambda > 0$ depending only on c and d/n . For fixed ε , this yields a uniform constant < 1 independent of n, d .

Once the mutation potential remains safe throughout, progress on the second slope is monotone and its runtime is easily bounded.

Lemma 11. If $B_t > 0$ for all t , then the second-slope ascent completes in expected

$$\Theta(n \log d)$$

iterations. If $\tau = \Omega(n^{1+\varepsilon})$, then $\Theta(n \log d) = o(\tau)$.

The expected time to reach the canonical start is $O(n^5 \log(n/d)/d^3)$. Once there, with constant probability the buffer remains positive for all phases; conditioned on this event, the second-slope ascent costs $O(n^2 \log d)$ evaluations. Since the success probability is constant, this remains the unconditional expectation. Summing the two contributions establishes the main runtime bound.

Proof [of Theorem 9.]

From a uniformly random start, the expected iterations to reach the cliff-top ($n - d$ ones) are $\sum_{i=d}^n \frac{n}{i} = \Theta(n \log \frac{n}{d})$, which at $\Theta(n)$ evaluations/iteration costs $\Theta(n^2 \log \frac{n}{d})$ evaluations per ascent to the cliff-top. At the cliff-top, the only fitter point is 1^n , so ageing triggers unless the optimum is found within τ iterations.

By Lemma 4, within at most three iterations after an ageing kick the process reaches the canonical second-slope start

$$r_0 = 2, \quad \text{HD}(r_0) = 2$$

with probability

$$p_{\text{can}}(n, d) \geq \frac{3}{256} \left(\frac{d}{n}\right)^3.$$

Hence the number of ageing epochs needed to obtain this start is geometric with mean $1/p_{\text{can}}$, and the expected evaluation cost up to the canonical start (including restarts) is

$$\Theta\left(\frac{1}{p_{\text{can}}(n, d)} \cdot n^2 \log \frac{n}{d}\right) = O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

Under the reference-update rule the start on the second slope is $r_0 = 2$, $\text{HD}(r_0) = 2$, so $s_0 = 0$ and with $\theta = \frac{1}{c} - 1$ define

$$B_r := \theta r - \sum_{j=r_0+1}^r Y_{z_j} \quad (r \geq r_0),$$

so $B_{r_0} = 2\theta$ and $B_{t+1} - B_t = \theta - Y_{z_{t+1}}$ by Lemma 7. By the refined phase MGF (Lemma 6), for every $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ with $\rho := 2d/n$,

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda).$$

Therefore, by Lemma 8,

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq e^{-\lambda\theta} K(\lambda) \quad \text{for all } t \geq 0.$$

Since $d \leq (\frac{1}{4} - \varepsilon)n$ and the theorem assumes

$$c < \frac{1 + 4\varepsilon}{3 - 4\varepsilon} \leq \frac{1 - 2d/n}{1 + 2d/n} =: c_*(n, d),$$

Lemma 9 guarantees an admissible $\lambda \in (0, \frac{1}{2} \ln \frac{n}{4d})$ with $e^{-\lambda\theta} K(\lambda) < 1$. With $B_{r_0} = 2\theta$, Corollary 2 yields

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda 2\theta}.$$

Moreover, since $\frac{1}{2} \ln \frac{n}{4d} \geq \frac{1}{2} \ln \frac{1}{1-4\varepsilon}$ for all admissible d , we may fix a uniform $\lambda_\varepsilon \in (0, \frac{1}{2} \ln \frac{1}{1-4\varepsilon})$ and conclude

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda_\varepsilon 2\theta}.$$

Because λ_ε depends only on ε and $\theta = \frac{1}{c} - 1$ depends only on c , this right-hand side is a constant in $(0, 1)$, independent of n and d .

On the event $\{\inf_{t \geq 0} B_t > 0\}$ (no jump-back), the second-slope ascent is monotone and completes in expected

$$\sum_{z=1}^d \frac{n}{z} = \Theta(n \log d)$$

iterations, which is $o(\tau)$ since $\tau = \Omega(n^{1+\varepsilon})$. Since the mutation potential satisfies $M \leq cn$ (where c is the constant from the M_{linHD} potential), each iteration evaluates at most $cn = \Theta(n)$ individuals; one second-slope attempt therefore costs $\Theta(n^2 \log d)$ evaluations, and the constant no-jump-back probability implies a constant expected number of such attempts.

Summing the (restarted) approach to the canonical start and the second-slope cost gives

$$\mathbb{E}[\text{evaluations}] = O\left(\frac{n^5 \log(n/d)}{d^3}\right) + O(n^2 \log d),$$

as claimed. $\square$

The analysis for $(1 + 1)$ Opt-IA_{LASTBEST} differs only in Lemma 4 and actually yields a d/n factor better probability bound since the mutation potential is 1 at the cliff-top and thus, creating a solution at the bottom of the cliff is immediately possible when any cliff top individual dies, whereas for $(1 + 1)$ Opt-IA_{FIRSTBEST} to work we have to first accept a neutral mutation to get the mutation potential

to 1, and then mutate into a cliff bottom solution in the next iteration. Consequently the runtime for $(1+1)$ Opt-IA_{LASTBEST} is $O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$.

Corollary 4. *The $(1+1)$ Opt-IA_{LASTBEST} with M_{linHD} (factor $c < \frac{1+4\epsilon}{3-4\epsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon'})$ optimises CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected*

$$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$$

fitness function evaluations.

The following section will highlight the advantages of using FIRST-BEST over using LAST-BEST.

VI. WHEN IPH OUTPERFORMS TOO LOW MUTATION RATES

In this section we introduce a function class where inversely proportional hypermutations outperform the lower mutation rates of SBM, and the higher mutation rate on local optima of the $(1+1)$ Opt-IA_{FIRSTBEST} will be essential. The function class is inspired by the one previously used to show the advantages and disadvantages of using high mutation rates in the context of rank-based mutation in population-based algorithms [48]. In rank-based mutation algorithms [63] each individual has a different mutation rate based on their rank in the population i.e., the best individual has the lowest mutation rate while the worst individual has a very high one. The LEADINGTRAPJUMP function was introduced as an example where the use of high mutation rates is detrimental while low mutation rates are efficient. However, if the locations of the local and global optima are swapped, then the opposite result can be shown (i.e. high mutation rates are effective while low ones are ineffective). We have slightly modified the location of the trap compared to [48] for our objectives.

Formally, for constants $k \geq 1$ and $\rho \in (0.8, 0.9)$, let $r := 2k + 1$, $L := \lfloor \log_2 n \rfloor$, $\mathcal{P} := \{x : \text{LO}(x) \geq 0.9n\}$, $\mathcal{B}_k := \{0^r 1^{(0.9n-r)*}\}$, and $\mathcal{D}_\rho := \{0^L * : |x|_1 \geq \rho n\}$. The function $\text{LTJ}_{k,\rho} : \{0, 1\}^n \rightarrow \mathbb{R}$ is

$$\text{LTJ}_{k,\rho}(x) = \begin{cases} 0, & x \in \mathcal{P}, \\ \text{LO}(x_{i>r}) + r, & x \in \mathcal{B}_k, \\ n + 1, & x \in \mathcal{D}_\rho, \\ \text{LO}(x), & \text{otherwise.} \end{cases}$$

Here $x_{i>r}$ denotes the suffix of x at positions $i > r$, i.e., $x_{i>r} := (x_{r+1}, x_{r+2}, \dots, x_n)$. Note that $r := 2k + 1$ is an odd number, as required for the block-jump prefix $0^r 1^{(0.9n-r)*}$ to force an even number of bit-flips to reach $\mathcal{B}_k$, which is the key parameter controlling the per-iteration success probability $\Theta(n^{-(r+1)}) = \Theta(n^{-(2k+2)})$. We introduce r as shorthand to avoid writing $2k + 1$ in every exponent throughout the proof.

The landscape consists of a long LeadingOnes slope rising from fitness $\text{LO}(x)$ up to level $0.9n - 1$, followed by a large penalised region. Two off-slope structures contain the two sets of optima but are inaccessible to standard hill-climbers. The set $\mathcal{B}_k$ begins with the prefix $0^r 1^{(0.9n-r)*}$ and forms a short

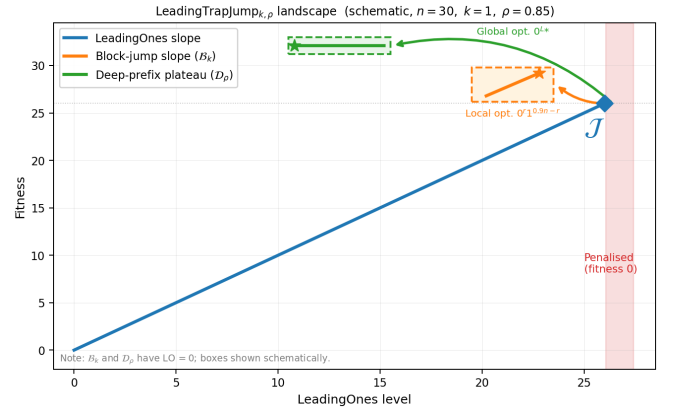


Fig. 4: The $\text{LEADINGTRAPJUMP}_{k,\rho}$ landscape (schematic, $n = 30, k = 1, \rho = 0.85$). The x-axis is the LEADINGONES level of the current solution. From the junction J (end of the main slope), large mutations can reach either the block-jump slope $\mathcal{B}_k$ (leading to the local optimum) or the deep-prefix plateau $\mathcal{D}_\rho$ (global optimum). Both off-slope structures have $\text{LO}(x) = 0$; their positions on the y-axis reflect their fitness values, not their LEADINGONES level.

slope, called block-jump slope, that deterministically increases toward its unique local optimum $0^r 1^{(0.9n-r)*}$; reaching this prefix is therefore equivalent to entering the leading ones path to the local optimum $0^r 1^{(0.9n-r)*}$. In contrast, the set $\mathcal{D}_\rho$ consists of solutions with prefix $0^L *$ whose suffix has ones-density at least ρ i.e., ρn ones, and forms a wide plateau of globally optimal fitness, which we call deep-prefix plateau. Both structures are hidden behind the penalised barrier (illustrated in Figure 4), and the only reliable way to reach them is via large hypermutation events. The junction $J := \{x : \text{LO}(x) = 0.9n - 1\}$ acts as a central gateway: each ageing epoch that restarts the search will, with high probability, ascend the LeadingOnes slope back to J before any off-slope structure is encountered. From J , the $(1+1)$ Opt-IA using IPH with M_{linHD} can perform jumps of the appropriate magnitude to reach either the entrance of the block-jump slope $\mathcal{B}_k$ or the plateau $\mathcal{D}_\rho$ i.e., via ageing restarts it will eventually find both optima efficiently. On the other hand, algorithms with lower mutation rates, such as those using SBM will with high probability not be able to flip the logarithmic number of bits required to reach the global optimum, thus have a runtime of $n^{\Omega(\log n)}$ with high probability.

Theorem 11. *The $(1+1)$ Opt-IA_{FIRSTBEST} with ageing threshold $\tau = \Omega(n^{1+\epsilon})$ using M_{linHD} with $c < 5/8$ identifies both local and global optima of $\text{LTJ}_{k,\rho}$ in $O(n\tau + n^{2k+3} + n^{\beta+1})$ fitness function evaluations where $\beta := \log_2 \frac{20}{c}$.*

To establish Theorem 11 we rely on three ingredients. First, every ageing epoch returns to the junction J with high probability. Second, an epoch beginning at J performs sufficiently long jumps to reach the beginning of the block-jump slope with probability $\Theta(n^{-(2k+2)})$ per iteration. Third, an epoch beginning at J performs a logarithmic jump onto the deep-prefix plateau with probability $\Theta(n^{-\beta})$ per iteration.

Since each event corresponds to reaching either the start of a short slope leading deterministically to its optimum (for $\mathcal{B}_k$) or directly a plateau of global optima (for $\mathcal{D}_\rho$), these per-epoch probabilities govern the entire runtime.

We first need to bound from below the probability that the algorithm visits the junction point after restarts in order to later argue that from the junction point both sets of optima will be accessed.

Lemma 12. *Starting from any random individual, the $(1 + 1)$ Opt-IA_{FIRSTBEST} using IPH with M_{linHD} and $c < 5/8$ reaches the junction $J = \{x : \text{LO}(x) = 0.9n - 1\}$ before accepting any individual in $\mathcal{B}_k$ or $\mathcal{D}_\rho$ with high probability.*

Within n^2 iterations, the algorithm performs at least $0.9n$ LeadingOnes improvements with high probability. Before reaching J , deep-prefix entrances require flipping the L -bit prefix to 0^L , an event whose probability per iteration is at most $(M(x)/n)^L$. Since under M_{linHD} the mutation potential satisfies $M(x) \leq q^*n$ for some constant $q^* < 1$ (determined solely by c and ρ), this becomes $(q^*)^L = n^{-\alpha^*}$ for $\alpha^* := \log_2(1/q^*) > 2$, yielding a total probability of $o(1)$ over n^2 iterations. Thus the slope is climbed to J with high probability.

The following two lemmata will condition throughout on the event that during an ageing epoch the junction J is reached. This event happens with high probability by Lemma 12.

Lemma 13. *Let an ageing epoch reach the junction J . Then, with probability $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$, the epoch performs a jump placing the individual at the beginning of the block-jump slope.*

The suffix of the individual at J is essentially random relative to the best individual stored from the previous epoch. With high probability their Hamming distance is at least $n/20$, guaranteeing $M(x) = cn/20$ throughout the epoch. Under this event, the probability that the first $r + 1$ flips of the hypermutation hit exactly the positions defining the prefix $0^r 1^{(0.9n-r)*}$ is $\Theta(n^{-(r+1)})$. Evaluating this over τ iterations yields the stated bound.

Lemma 14. *Let an ageing epoch reach the junction J . Then, with probability $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, the epoch performs a jump placing the individual onto the deep-prefix plateau.*

At J , the ones-density of the suffix already exceeds ρ with high probability. Thus it suffices to flip the prefix to 0^L . Under the same high-Hamming-distance event ensuring $M(x) \geq cn/20$, the probability that the first L flips hit positions $\{1, \dots, L\}$ is at least $(c/20)^L = n^{-\beta}$. Evaluating over τ iterations gives the lemma.

Each ageing epoch begins with a uniformly random reinitialisation and by Lemma 12 it ascends to the junction J with high probability before reaching any off-slope structure. We henceforth condition on epochs reaching J . Each ageing epoch evaluates $\Theta(n\tau)$ individuals. By Lemma 12, epochs reach J with high probability, and by lemmata 13 and 14, an epoch discovers each off-slope structure with probabilities p_B and p_D , respectively. Since B (entering $\mathcal{B}_k$) and D (entering $\mathcal{D}_\rho$) are mutually exclusive within a single epoch, the algorithm

must discover them in *different* epochs. By a coupon-collector argument (we need to “collect” both coupons B and D , which arrive independently across epochs with probabilities p_B and p_D respectively), the expected number of epochs until both optima have been seen is $O(1/p_B + 1/p_D)$. Multiplying by the cost per epoch yields the bound stated in Theorem 11.

Proof [of Theorem 11.]

By Lemma 12 we reach the junction J with probability $1 - o(1)$. Starting an epoch in the junction, the two lemmata give $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$ and $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, and B and D are mutually exclusive within one epoch. Thus the expected number of epochs to sample from both sets of optima is:

$$O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right).$$

Multiplying by the cost of each epoch we get,

$$\begin{aligned} \mathbb{E}[\text{evals}] &\leq \Theta(n\tau) O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right) \\ &= O(n\tau + n^{2k+3} + n^{\beta+1}). \end{aligned}$$

□

The static hypermutation would yield a same upper bound with a similar analysis except for the value of β which would be smaller and for the range of c which would be more restricted. Both modifications are necessary due to the larger mutation size of the static hypermutation compared to IPH for a given c value. The $(1 + 1)$ Opt-IA_{LASTBEST} IPH cannot make the jumps from the junction since any current junction solution would be the last best seen, yielding a mutation potential of one 1. $\mathcal{B}_k$ and $\mathcal{D}_\rho$ sets cannot be accessed from the junction via mutation but can still be sampled at any reinitialization with $2^{-\Omega(n)}$ probability. We conjecture that power-law mutation operators such as those employed by the Fast-IA [3] and the Fast-EA [4] would also succeed on $\text{LTJ}_{k,\rho}$ if equipped with an ageing operator such as the algorithms introduced in [3], and the same should hold for the Rank-GA [63]. However, we believe that the Rank-GA with ageing would fail on CLIFF_d because of its high mutation rate at the bottom of the second slope. On the other hand, the Fast-IA and the Fast-EA, which have been shown to be parameterisable such that they behave equivalently, can quite efficiently (a $\sqrt{n}$ factor slower than the $(1 + 1)$ Opt-IA_{FIRSTBEST} and the $(1 + 1)$ Opt-IA_{LASTBEST} IPH i.e., in expected time $O(n^{5/2} \log n)$) optimise CLIFF_d functions with gaps of linear size [3]. However, they have not been shown to be efficient for CLIFF_d functions with sublinear gap sizes. Stagnation detection algorithms [64]–[66] may also be efficient for $\text{LTJ}_{k,\rho}$ if equipped with the ageing operator. It is not obvious, though, how the operator would be incorporated in such an algorithm. Thus, to date the $(1 + 1)$ Opt-IA_{FIRSTBEST} IPH is the best known algorithm to optimise both Cliff-like functions while at the same time optimise functions where large mutations are necessary to escape from local optima. An open question is whether it may be possible to avoid the use of ageing altogether for $\text{LTJ}_{k,\rho}$ by using the recently introduced population-based EAs with low selective pressure [67], [68]. Low selective pressure may

avoid the entire population converging on one of the two local optima shortly after it is identified.

VII. CONCLUSION

We have presented a thorough analysis of Inversely Proportional Hypermutations (IPH). Previous theoretical studies have shown disappointing results concerning the IPH operators from the literature. In this paper we have shown their effectiveness in isolation for unimodal functions compared to static hypermutations in the ideal conditions when the optimum is known. Furthermore, we have provided an analysis of the operator embedded in the complete Opt-IA AIS to be used in practical applications where the optimum is usually unknown. We have proved its effectiveness for two well-studied multimodal functions with considerably different characteristics showing how it harmonises together with the AIS operator better than static hypermutations. In particular the $(1+1)$ Opt-IA_{FIRSTBEST} can escape local optima in three different ways: by mutating into far away solutions (see LEADINGTRAPJUMP), by accepting solutions of inferior quality (see CLIFF) and naturally by automatically reinitialising the trapped solutions via ageing as understood regarding ageing for a long time [45], [47], [69], [70]. In particular, compared to the conference version of this paper, we have now proposed an Opt-IA algorithm with IPH that has the capability to escape from local optima via large mutations, something that was not possible with the previous version since the mutation rate remained lowest on the local optima.

Although our analysis used standard benchmark functions capturing specific structures that may appear in combinatorial optimisation problems, we point out that the Opt-IA AIS with our proposed IPH can be shown to efficiently find arbitrarily good approximations to the NP-Hard Number Partitioning problem by following the same proof strategy of [41] that uses that local optima are escaped from thanks the ageing operator that simply restarts the algorithm once trapped on a local optimum. Concerning escaping local optima via hypermutation, IPH would have an insufficient mutation potential to find an approximation in the first run until reinitialisation. However establishing whether after restarts due to ageing triggering on local optima occur, subsequent runs with higher mutation potential can approximate arbitrarily well without relying on a good starting position require further analysis of the problem landscape.

Future work should evaluate the performance of the proposed algorithms and inversely proportional mutation operators both theoretically and experimentally for other combinatorial optimisation problems with practical applications.

REFERENCES

- [1] T. Friedrich, F. Quinzan, and M. Wagner, "Escaping large deceptive basins of attraction with heavy-tailed mutation operators," in *Proc. of GECCO 2018*, 2018, pp. 293–300.
- [2] T. Friedrich, A. Göbel, F. Quinzan, and M. Wagner, "Heavy-tailed mutation operators in single-objective combinatorial optimization," in *Proc. of PPSN XV*, 2018, pp. 134–145.
- [3] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast immune system inspired hypermutation operators for combinatorial optimisation," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 956–970, 2021.
- [4] B. Doerr, H. P. Le, R. Makhmara, and T. D. Nguyen, "Fast genetic algorithms," in *Proc. of GECCO 2017*, 2017, pp. 777–784.
- [5] D. Corus and P. S. Oliveto, "Standard steady state genetic algorithms can hillclimb faster than mutation-only evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 720–732, 2018.
- [6] D.-C. Dang, T. Friedrich, T. Kötzing, M. S. Krejca, P. K. Lehre, P. S. Oliveto, D. Sudholt, and A. M. Sutton, "Escaping local optima using crossover with emergent diversity," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 3, pp. 484–497, 2018.
- [7] D. Corus and P. S. Oliveto, "On the benefits of populations on the exploitation speed of standard steady-state genetic algorithms," in *Proc. of GECCO 2019*, 2019, pp. 1452–1460.
- [8] P. S. Oliveto, D. Sudholt, and C. Witt, "A tight lower bound on the expected runtime of standard steady state genetic algorithms," in *Proc. of GECCO 2020*, 2020, pp. 1323–1331.
- [9] B. Doerr, C. Doerr, and F. Ebel, "From black-box complexity to designing new genetic algorithms," *Theoretical Computer Science*, vol. 567, pp. 87–104, 2015.
- [10] J. Lengler, "A general dichotomy of evolutionary algorithms on monotone functions," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 6, pp. 995–1009, 2019.
- [11] B. Doerr and C. Doerr, "Theory of parameter control for discrete black-box optimization: Provable performance gains through dynamic parameter choices," in *Theory of Evolutionary Computation - Recent Developments in Discrete Optimization*, B. Doerr and F. Neumann, Eds. Springer, 2020, pp. 271–321.
- [12] B. Doerr, A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "On the runtime analysis of selection hyper-heuristics with adaptive learning periods," in *Proc. of GECCO 2018*, 2018, pp. 1015–1022.
- [13] B. Doerr, C. Doerr, and J. Yang, "k-bit mutation with self-adjusting k outperforms standard bit mutation," in *Proc. of PPSN'16*. Springer, 2016, pp. 824–834.
- [14] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "Simple Hyper-Heuristics Control the Neighbourhood Size of Randomised Local Search Optimally for LeadingOnes," *Evolutionary Computation*, vol. 28, no. 3, pp. 437–461, 09 2020.
- [15] —, "How the duration of the learning period affects the performance of random gradient selection hyper-heuristics," in *Proc. of AAAI-20*, 2020, pp. 2376–2383.
- [16] P. S. Oliveto, T. Paixão, J. Pérez Heredia, D. Sudholt, and B. Trubenová, "How to escape local optima in black box optimisation: when non-elitism outperforms elitism," *Algorithmica*, vol. 80, pp. 1604–1633, 2018.
- [17] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "When move acceptance selection hyper-heuristics outperform metropolis and elitist evolutionary algorithms and when not," *Artificial Intelligence*, vol. 314, p. 103804, 2023.
- [18] D.-C. Dang, A. V. Eremeev, and P. K. Lehre, "Escaping local optima with non-elitist evolutionary algorithms," in *Proc. of AAAI 2021*, 2021.
- [19] B. Doerr, "Does comma selection help to cope with local optima?" in *Proc. of GECCO '20*. ACM, 2020, p. 1304–1313.
- [20] B. Case and P. K. Lehre, "Self-adaptation in non-elitist evolutionary algorithms on discrete problems with unknown structure," *IEEE Transactions on Evolutionary Computation (To appear - arXiv:2004.00327)*, vol. 24, no. 4, pp. 650–663, 2020.
- [21] D.-C. Dang, T. Jansen, and P. K. Lehre, "Populations can be essential in dynamic optimisation," in *Proc. of GECCO 2015*, 2015, pp. 1407–1414.
- [22] D.-C. Dang and P. K. Lehre, "Efficient optimisation of noisy fitness functions with population-based evolutionary algorithms," in *Proc. of FOGA 2015*, 2015, pp. 62–68.
- [23] J. Lengler, L. Schiller, and O. Sieberling, "Plus strategies are exponentially slower for planted optima of random height," in *Proc. of GECCO 2024*, 2024, pp. 1587–1595.
- [24] M. A. Hevia-Fajardo and D. Sudholt, "Self-adjusting population sizes for non-elitist evolutionary algorithms: Why success rates matter?," *Algorithmica*, vol. 86, no. 2, pp. 526–565, 2024.
- [25] L. R. de Castro and J. Timmis, *Artificial Immune Systems: A New Computational Intelligence Paradigm*. Springer-Verlag, 2002.
- [26] Z. Huang and Y. Zhou, "Runtime analysis of somatic contiguous hypermutation operators in MOEA/D framework," in *Proc. of AAAI*, 2020, pp. 2359–2366.
- [27] F. M. Burnet, *The Clonal Selection Theory of Acquired Immunity*. Cambridge University Press, 1959.
- [28] E. A. Aguilar Arroyo and C. A. Coello Coello, "Moaisdx: A new multi-objective artificial immune system based on decomposition," in *Proc. of EMO 2025*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 46–59.

- [29] L. Li, Q. Lin, Z. Ming, M. Gong, and C. A. Coello Coello, "An immune-inspired resource allocation strategy for many-objective optimization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 6, pp. 3284–3297, 2022.
- [30] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast contiguous somatic hypermutations for single-objective optimisation and multi-objective optimisation via decomposition," in *Proc. of AAAI'25*, 2025, pp. 26922–26930.
- [31] Z. Huang and Y. Zhou, "Runtime analysis of somatic contiguous hypermutation operators in moea/d framework," in *Proc. of AAAI 2020*, 2020, pp. 2359–2366.
- [32] L. Li, Q. Lin, and Z. Ming, "A survey of artificial immune algorithms for multi-objective optimization," *Neurocomputing*, vol. 489, pp. 211–229, 2022.
- [33] J. Brownlee, "Clonal selection algorithms," Complex Intelligent Systems Laboratory (CIS), Swinburne University of Technology, Tech. Rep., 2007.
- [34] L. N. de Castro and F. J. V. Zuben, "Learning and optimization using the clonal selection principle," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 3, pp. 239–251, 2002.
- [35] V. Cutello, G. Nicosia, M. Pavone, and J. Timmis, "An immune algorithm for protein structure prediction on lattice models," *IEEE Transactions on Evolutionary Computation*, vol. 11, pp. 101–117, 2007.
- [36] B. H. Ulutas and S. Kulturel-Konak, "A review of clonal selection algorithm and its applications," *Artificial Intelligence Review*, vol. 36, pp. 117–138, 2011.
- [37] A. Auger and N. Hansen, "Theory of evolution strategies: a new perspective," in *Theory of Randomized Search Heuristics: Foundations and Recent Developments*, A. Auger and B. Doerr, Eds. World Scientific Publishing Co., Inc., 2011, pp. 289–325.
- [38] C. Zarges, "Rigorous runtime analysis of inversely fitness proportional mutation rates," in *Proc. of PPSN*, 2008, pp. 112–122.
- [39] D. Corus, P. S. Oliveto, and D. Yazdani, "When hypermutations and ageing enable artificial immune systems to outperform evolutionary algorithms," *Theoretical Computer Science*, vol. 832, pp. 166–185, 2019.
- [40] —, "Artificial immune systems can find arbitrarily good approximations for the NP-hard partition problem," in *Proc. of PPSN 2018*, 2018, pp. 16–28.
- [41] —, "Artificial immune systems can find arbitrarily good approximations for the NP-hard number partitioning problem," *Artificial Intelligence*, vol. 247, pp. 180–196, 2019.
- [42] T. Jansen and C. Zarges, "Variation in artificial immune systems: hypermutations with mutation potential," in *Proc. of ICARIS 2011*, 2011, pp. 132–145.
- [43] S. Droste, T. Jansen, and I. Wegener, "On the analysis of the $(1 + 1)$ evolutionary algorithm," *Theoretical Computer Science*, vol. 276, no. 1–2, pp. 51–81, 2002.
- [44] J. Jägersküpper and T. Storch, "When the plus strategy outperforms the comma strategy and when not," in *Proc. of FOCI 2007*, 2007, pp. 25–32.
- [45] P. S. Oliveto and D. Sudholt, "On the runtime analysis of stochastic ageing mechanisms," in *Proc. of GECCO 2014*, 2014, pp. 113–120.
- [46] D. Corus, P. S. Oliveto, and D. Yazdani, "On the runtime analysis of the Opt-IA artificial immune system," in *Proc. of GECCO 2017*, 2017, pp. 83–90.
- [47] C. Horoba, T. Jansen, and C. Zarges, "Maximal age in randomized search heuristics with aging," in *Proc. of GECCO 2009*, 2009, pp. 803–810.
- [48] P. S. Oliveto, P. K. Lehre, and F. Neumann, "Theoretical analysis of rank-based mutation - combining exploration and exploitation," in *Proc. of CEC 2009*, 2009, pp. 1455–1462.
- [49] D. Corus, P. S. Oliveto, and D. Yazdani, "On inversely proportional hypermutations with mutation potential," in *Proc. of GECCO 2019*, 2019, p. 215–223.
- [50] P. S. Oliveto and C. Witt, "On the runtime analysis of the simple genetic algorithm," *Theoretical Computer Science*, vol. 545, pp. 2–19, 2014.
- [51] —, "Improved time complexity analysis of the simple genetic algorithm," *Theoretical Computer Science*, vol. 605, pp. 21–41, 2015.
- [52] D. Whitley, "The genitor algorithm and selection pressure: Why rank-based allocation of reproductive trials is best," in *Proc. of the Third International Conference on Genetic Algorithms*, 1989, pp. 116–121.
- [53] F. Neumann, P. S. Oliveto, and C. Witt, "Theoretical analysis of fitness-proportional selection: landscapes and efficiency," in *Proc. of GECCO 2009*, 2009, pp. 835–842.
- [54] D. Corus, D.-C. Dang, A. V. Eremeev, and P. K. Lehre, "Level-based analysis of genetic algorithms and other search processes," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 707–719, 2017.
- [55] D. Corus and P. S. Oliveto, "On the benefits of populations for the exploitation speed of standard steady-state genetic algorithms," *Algorithmica*, vol. 82, pp. 3676–3706, 2020.
- [56] S. Böttcher, B. Doerr, and F. Neumann, "Optimal fixed and adaptive mutation rates for the leadingones problem," in *Proc. of PPSN'10*. Springer, 2010, pp. 1–10.
- [57] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "How the duration of the learning period affects the performance of random gradient selection hyper-heuristics," in *Proc. of AAAI 2020*, 2020, pp. 2376–2383.
- [58] W. Feller, *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, 1968.
- [59] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast artificial immune systems," in *Proc. of PPSN 2018*, 2018, pp. 67–78.
- [60] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "On the time complexity of algorithm selection hyper-heuristics for multimodal optimisation," in *Proc. of AAAI 2019*, 2019, pp. 2322–2329.
- [61] A. Bendahi, B. Doerr, A. Fradin, and J. F. Lutzeyer, "Speeding up hyper-heuristics with markov-chain operator selection and the only-worsening acceptance operator," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, ser. IJCAI '25, 2025.
- [62] J. L. Doob, *Stochastic Processes*. John Wiley & Sons, 1953.
- [63] J. Cervantes and C. R. Stephens, "Rank based variation operators for genetic algorithms," in *Proc. of GECCO 2008*, 2008, pp. 905–912.
- [64] M. S. Krejca and C. Witt, "A flexible evolutionary algorithm with dynamic mutation rate archive," in *Proc. of GECCO '24*, 2024, p. 1578–1586.
- [65] A. Rajabi and C. Witt, "Self-adjusting evolutionary algorithms for multimodal optimization," *Algorithmica*, vol. 84, no. 6, pp. 1694–1723, 2022.
- [66] —, "Stagnation detection with randomized local search," *Evolutionary Computation*, vol. 31, no. 1, pp. 1–29, 2023.
- [67] D. Corus, A. Lissovoi, P. S. Oliveto, and C. Witt, "On steady-state evolutionary algorithms and selective pressure: Why inverse rank-based allocation of reproductive trials is best," *ACM Transactions on Evolutionary Learning and Optimization*, vol. 1, no. 1, 2021.
- [68] D. Corus, P. S. Oliveto, and F. Zheng, "Hybrid selection allows steady-state evolutionary algorithmsto control the selective pressure in multimodal optimisation," in *Proc. of GECCO 2025*, 2025, pp. 881–889.
- [69] T. Jansen and C. Zarges, "On benefits and drawbacks of aging strategies for randomized search heuristics," *Theoretical Computer Science*, vol. 412, no. 6, pp. 543–559, 2011.
- [70] —, "Aging beyond restarts," in *Proc. of GECCO 2010*, 2010, pp. 705–712.

VIII. BIOGRAPHY SECTION

If you include a photo:



Michael Shell Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

If you will not include a photo:

John Doe Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.

Effective Inversely Proportional Hypermutations for Unimodal and Multimodal Optimisation

- Supplementary Material -

Anonymous authors

Abstract—Artificial Immune Systems (AIS) employing hypermutations with linear static mutation potential have recently been shown to be very effective at escaping local optima of combinatorial optimisation problems at the expense of being slower during the exploitation phase compared to standard evolutionary algorithms. In this paper, we examine the more commonly used dynamic inversely proportional hypermutations (IPH) with mutation potential for which previous theoretical analyses have shown disappointing performance. We first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations in the ideal situation where the optimum is known if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. Afterwards, we define a simple (1+1) Opt-IA that uses IPH and ageing for realistic applications where optimal solutions are unknown. The aim of this AIS is to approximate the ideal behaviour of the IPH better and better as the search space is explored and local optima of higher quality are identified. We present two variants, one that decreases the mutation rate proportionally until the local (global) optimum is identified, and another where the mutation rate may increase again when detecting that the algorithm is stuck on a large plateau of local optima. We show that the latter variant outperforms standard bit mutations by escaping the local optima on a multimodal function from the literature and that both variants outperform static hypermutations by escaping the local optima of the standard CLIFF benchmark function by accepting solutions of lower fitness.

Index Terms—Artificial immune systems, heavy-tailed mutations, inversely proportional hypermutations, theory, runtime analysis

I. INTRODUCTION

Three recent hot topics in the theory of randomised search heuristics for optimisation in general and in artificial intelligence in particular have been the benefits of mutation operators with higher mutation rates than traditionally employed [1]–[4] (or standard operators using higher parameter values than those previously recommended [5]–[10]), the automatic adaptation of the mutation rate [11]–[15] and the analysis of when non-elitist search heuristics are beneficial compared to elitist ones [16]–[24].

Artificial Immune Systems (AIS) are a class of bio-inspired algorithms which naturally use both high mutation rates via so-called hypermutation operators, often automatically adapt the mutation rate, and use non-elitism implicitly through the use of ageing operators, characteristics that are typical in the immune system of vertebrates from which AIS draw inspiration. In this paper we analyse the performance of an adaptive mutation operator called Inversely Proportional Hypermutation (IPH) that aims to increase the mutation rate

with (an estimate of) the distance from the optimum, and propose a considerably improved one for efficient unimodal and multimodal optimisation.

AIS for optimisation [25], [26] are generally inspired by the clonal selection principle [27]. For this reason they are also often referred to as *clonal selection* algorithms [25]. Clonal selection algorithms are also popular in evolutionary multi-objective optimisation [28]–[32]. In the literature two key features of clonal selection algorithms have been identified [33]:

- 1) The proliferation rate of each immune cell is proportional to its affinity (similarity) with the selective antigen (pathogen): the higher the affinity, the higher the number of offspring generated (clonal selection and expansion).
- 2) The mutation suffered by each immune cell during reproduction is inversely proportional to the affinity of the cell receptor with the antigen: the higher the affinity, the smaller the mutation (affinity maturation).

Indeed, well-known clonal selection algorithms employ mutation operators applied to immune cells (i.e., candidate solutions) with a rate that decreases with their similarity to the antigen (i.e., global optima) during affinity maturation. Often such operators are referred to as *inversely proportional hypermutations* (IPH). Two of the most popular clonal selection algorithms, Clonalg [34] and the Opt-IA immune algorithm [35] (as well as their many variants [36]) both employ inversely proportional hypermutations. The ideal behaviour of the IPH operator is that the mutation rate is minimal in proximity of the global optimum and increases as the difference between the fitness of the global optimum and that of the candidate solution increases. However, achieving such behaviour in practice may be problematic because the fitness of the global optimum is usually unknown. As a result, in practical applications information about the problem is used to find bounds on (or estimates of) the value of the global optimum¹. Thus, the closer the estimate is to the actual value of the global optimum, the closer should the behaviour of the IPH operator be to the desired one. On the other hand, if the bound is much higher (e.g., for a maximisation problem) than the true value, then there is a risk that the mutation rate is too high in proximity of the global optimum i.e., the algorithm will struggle to identify the optimum.

¹Alternatively, the fitness of the best candidate solution is sometimes used and the mutation rate of the rest of the population is inversely proportional to the best.

While decreasing the step size as the optimum is approached is a common practice in continuous optimisation [37], previous theoretical analyses for combinatorial optimisation, though, have highlighted various problems with IPH operators from the literature even when the fitness value of the global optimum is known. Zarges [38] analysed the effects of mutating candidate solutions with a rate that is inversely proportional to their fitness for the ONEMAX problem. She considered two different rates for the decrease of the mutation rate as the fitness increases: a linear decay (i.e., each bit is flipped with probability $\frac{\text{OPT}-\text{ONEMAX}(x)}{\text{OPT}}$, where OPT is the value of the optimum) and an exponential decay (i.e., each bit flips with probability $e^{-\rho \frac{\text{ONEMAX}(x)}{\text{OPT}}}$ where ρ is called the *decay* parameter). The motivation behind these choices are that the former operator flips in expectation exactly the number of bits that maximises the probability of reaching the optimum in the next step while the latter is the operator used in the Clonal algorithm. She showed that if the optimum of ONEMAX is known, then an algorithm employing such mutation operators will require exponential time to optimise ONEMAX with overwhelming probability (w.o.p.) in both cases of linear or exponential decays of the mutation rate. The reason is that the initial random solutions that have roughly half the fitness (i.e., $n/2$) of the global optimum (i.e., n) have very high mutation rates. Such rates do not allow the algorithm to make any progress with any reasonable probability even if the decay parameter ρ is chosen very carefully (i.e., $\rho = \ln n$ leads to reasonable mutation rates between $1/n$ and $1/2$ during the optimisation of ONEMAX). Hence, the behaviour of the mutation operator leads to very inefficient performance even for the simple OneMax function when the optimum is assumed to be known.

On the other hand, the Opt-IA clonal selection algorithm, that uses a mutation operator called 'hypermutations with mutation potential' which also has very high mutation rates has been proven to be efficient by employing a selection strategy called *stop at first constructive mutation* (FCM). The strategy evaluates the fitness after each bit is flipped and interrupts the hypermutation immediately if an improvement is detected. Otherwise it returns the bit-string obtained after it has flipped the maximum number of bits it is allowed to flip i.e., the mutation potential. Indeed, the operator using a static mutation potential (i.e., a linear number of bits are flipped unless an improvement is found along the way) has been proven to be very effective at escaping local optima of standard multimodal benchmark functions [39] and at finding arbitrarily good approximations for the Number Partitioning NP-Hard problem [40], [41] at the expense of being slower at hillclimbing during the exploitation phases - it is up to a linear factor slower than the standard bit mutation (SBM) used by evolutionary algorithms for easy unimodal functions such as OneMax and LeadingOnes. Hence, the first insight we provide in this paper is that, differently from other clonal selection algorithms (such as Clonal and Opt-IA without FCM), hypermutations with mutation potential coupled with FCM can cope with the desired behaviour of inversely proportional mutation rates by being capable of making progress efficiently with high mutation rates. In this paper we first consider

whether Opt-IA may become faster during the exploitation phases if inversely proportional mutation potentials are applied rather than static ones. The reason to believe this is that the hypermutations waste many fitness function evaluations during exploitation, since the probability of flipping bits that lead to an improvement decreases when the optimum is approached. Hence, with high probability the operator flips wrong bits at the beginning of a hypermutation and ends up wasting a linear number of fitness function evaluations in each hypermutation. On the other hand, if the mutation potential decreases as the optimum is approached, then the amount of wasted evaluations should decrease accordingly.

A previous runtime analysis for OneMax of the inversely proportional hypermutations with mutation potential used by Opt-IA [42] has shown that the mutation rate is always in the range $[(c/2)n, cn]$ where $M = cn$ is the highest mutation potential. Hence it does not decrease proportionally with the decrease in distance to the optimum as desired and, as a result, no asymptotic speed-ups are achieved compared to static mutation potentials.

In this paper we first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations, if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. To show this, we analyse the IPH operators for standard unimodal benchmark functions where the performance of static mutation potentials is well-understood [39]. We first identify what speed-ups may be hoped for by analysing the operators in the ideal situation where the location of the optimum is known. A result of this first analysis is that a mutation potential that increases exponentially with the Hamming distance to the optimum provides the larger speed-ups for hillclimbing unimodal functions. Furthermore, using the Hamming distance rather than the fitness as the measure to quantify proximity to the optimum makes the operator robust to fitness function scaling. However, an exponential decay with the distance to the optimum also implies that high mutation rates would occur only when the current solution is prohibitively far away from the optimum, *thus, the algorithm would flip just one bit most of the time, i.e., very rare hyper-mutations*. For instance, a randomly generated solution with expected Hamming distance $n/2$ from the optimum flips $\sqrt{n}$ bits and this mutation will decrease exponentially fast with further improvements. For this reason we find that a linear decrease of the mutation rate is a better compromise in *practice* as it allows for longer phases where large mutations occur and it flips the number of bits that increase the probability of hitting the optimum in the next step.

Afterwards we propose a clonal selection algorithm that we call (1+1) Opt-IA that uses IPH and ageing² [35] to be applied in practical unimodal and multimodal applications where the optimum is not known. The algorithm uses the best solution it has encountered to estimate the mutation rates. In the literature it has been shown that ageing allows algorithms to escape from local optima either by identifying a new slope of increasing fitness or by completely restarting the optimisation process if it cannot escape [41], [42], [45]–[47]. The idea behind our

²The well-known Opt-IA AIS uses both operators in combination [35].

TABLE I

EXPECTED RUNTIMES OF THE DIFFERENT MUTATION OPERATORS WHEN EMBEDDED IN A $(1+1)$ ALGORITHMIC FRAMEWORK FOR THE BENCHMARK FUNCTIONS CONSIDERED IN THIS PAPER. FOR ALL RESULTS THE OPTIMUM IS UNKNOWN TO THE ALGORITHMS. **THE * INDICATES THAT WE DO NOT FIND IT WORTH THE EFFORT TO PERFORM A FORMAL ANALYSIS, YET FOR REASONS PROVIDED AT THE END OF SECTION VI, THE UPPER BOUND IS CORRECT.** $\beta := \log_2 \frac{20}{c}$, WHERE $c \leq 1$ IS THE CONSTANT FACTOR IN THE M_{LINHD} POTENTIAL $M = \lfloor c \cdot H(x, \text{best}) \rfloor$.

(1+1) Algorithms	ONEMAX	LEADINGONES	CLIFF _d	LEADINGTRAPJUMP _{k, ρ}
EA (SBM)	$\Theta(n \log n)$ [43]	$\Theta(n^2)$ [43]	$\Theta(n^d)$ [44]	$n^{\Omega(\log n)}$
Opt-IA _{STATIC}	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]	$O(\frac{n^{d+1} \cdot e^d}{d^d})$ [39]	$O(n\tau + n^{2k+3} + n^{\beta+1})^*$
Opt-IA _{LASTBEST}	$\Theta(n \log n)$	$\Theta(n^2)$	$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$	$2^{\Omega(n)}$
Opt-IA _{FIRSTBEST}	$\Theta(n \log n)$	$\Theta(n^3)$	$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$	$O(n\tau + n^{2k+3} + n^{\beta+1})$

proposed AIS is that the more the search space is explored, the better is the ideal behaviour of the IPH approximated through the discovery of better and better local optima. We propose two variants. The first one, called $(1+1)$ Opt-IA_{LASTBEST}, updates the location of the best seen solution whenever a new solution with fitness value at least as good as the previous one is identified. This guarantees that the mutation rate is minimal on the optimum once it is identified, exactly in the spirit of inversely proportional hypermutations, and allows for the best possible expected runtimes when hillclimbing unimodal functions such as ONEMAX and LEADINGONES. However, the Opt-IA using such an operator loses power compared to static hypermutations because it cannot escape from the best identified local optima via large mutations as its mutation rate will be minimal. To this end, we define a second variant called $(1+1)$ Opt-IA_{FIRSTBEST}, that only updates the location of the best seen solution when a strictly better solution is identified. This algorithm is capable of increasing its mutation rate again when stuck on large plateaus of local optima by increasing its Hamming distance from the best seen solution.

We first use the CLIFF benchmark function to prove the effectiveness of the IPH over static hypermutations. In particular, we show that the IPH can escape local optima when coupled with ageing, because this operator allows the algorithm to escape the local optimum by accepting solutions of inferior fitness. We remark that static hypermutations cannot optimise the function efficiently because the operator only stops if a constructive mutation is identified, thus will not return inferior solutions that may survive the ageing operator and allow the algorithm to escape the local optima of the function class. This makes a difference between polynomial and superpolynomial runtimes for super-constant cliff distances, and an expected runtime of $O(n^2 \log n)$ versus exponential for linear cliffs. A different problem would occur if hypermutations inversely proportional to fitness were to be used. Since the difference in fitness between the local optima and the bottom of the Cliff is very large, the operator's mutation rate at the bottom of the cliff would be too high leading the algorithm back to the local optima with high probability.

We conclude the paper by providing an example benchmark function where it is necessary to escape from local optima using large mutations. Thus the $(1+1)$ Opt-IA_{FIRSTBEST} is efficient while traditional standard bit mutations (SBM) that

use lower mutation rates are not. The benchmark function, called LEADINGTRAPJUMP, is inspired from one previously used in the literature to highlight when high mutation rates are beneficial and when not [48]. Essentially, by switching the fitness values of the local optima and the global optimum one can provide examples where SBM outperforms high mutation rates and viceversa. Herein we show that the $(1+1)$ Opt-IA_{FIRSTBEST} can identify the global optimum in both cases.

A summary of the results obtained in the paper is provided in Table I highlighting the speed-ups achieved by the two IPH operators compared to the standard bit mutations traditionally used by evolutionary algorithms and to the static hypermutation operator with FCM. It can be noticed how the $(1+1)$ Opt-IA_{FIRSTBEST} is a better compromise to hillclimb faster and allow escaping local optima both by accepting solutions of inferior fitness *and* by performing large jumps in the search space making a difference between polynomial and super-polynomial runtimes.

The original extended abstract [49] of this paper analysed the advantages of using an exponential decay of the mutation rate since this provides the largest speed-ups for unimodal functions in ideal conditions where the optimum is known (see Section III). However, a great disadvantage of the exponential decay is that large mutations hardly ever occur in practice. Indeed no superiority of IPH over a SBM operator was provided. To this end, in this paper we analyse the use of the linear decay operator and provide examples where the IPH indeed considerably outperforms SBM providing a difference between polynomial and superpolynomial expected runtimes.

II. INVERSELY PROPORTIONAL MUTATION POTENTIALS

Hypermutations with mutation potential using FCM flip at most a linear amount of bits (i.e., the mutation potential $M = cn$) chosen uniformly at random without replacement from the interval $[1..n]$. After each of the M bit flips, the fitness of the produced solution is evaluated and if a *constructive mutation* is identified, then the operator is stopped and the solution returned. A constructive mutation is a solution that is at least as good as that of the original parent. Otherwise the algorithm continues flipping the bits without setting them back until the potential M of bit flips is reached. If no constructive mutation has been identified earlier, then the M -th solution is returned. The original inversely proportional hypermutation

operator (IPH) proposed for Opt-IA uses mutation potential $M = \lceil (1 - f_{\text{OPT}}/v)cn \rceil$ for minimisation problems, where $f_{\text{OPT}} \geq 0$ is the optimal fitness value, $v > 0$ is the fitness of the current individual, and $0 < c \leq 1$ is some constant [35], [42]. In an analysis for $\text{ZEROMIN} := n + 1 - \text{ONEMAX}$ [42], such a mutation potential was shown not to decrease below $cn/2$, with cn being the highest possible mutation potential. As a result no speed-ups at hillclimbing are achieved compared to static hypermutations as the mutation potential is always at least linear. In this section, we introduce three different IPH operators that will be analysed in this paper and that provably speed-up the hillclimbing performance for ONEMAX. Two have been already considered in the literature while the third one is newly proposed by us based on analysed drawbacks of the first two at speeding up hillclimbing.

Hamming Distance Based Linear Decrease

Zarges analysed an inversely proportional hypermutation operator where the probability of flipping each bit increases linearly with the Hamming distance to the optimum or its best available estimate [38]. Precisely, this operator flips each bit with probability $H(x, \text{best})/n$ where n is the size of the problem and $H(x, \text{best})$ is the Hamming distance of the current point to the best individual (or an estimate if not available). As the expected number of bit-flips is $H(x, \text{best})$ during each execution of a mutation operator, a mutation potential inspired by this inversely proportional hypermutation is:

$$M_{\text{linHD}}(x) := H(x, \text{best}).$$

Fitness Difference Based Exponential Decrease

In Clonalg's IPH, the mutation probability decreases as an exponential function of the fitness of the current solution [34]. Precisely, each bit flips with probability $e^{-\rho \cdot v}$ where v is the normalised fitness value and ρ is a decay parameter that regulates the speed at which the mutation rate decreases. Since we consider maximisation problems, we use $v = \frac{f(x)}{f(\text{best})}$ as suggested by [38] where $f(\text{best})$ is the best known fitness value. Using the expected number of bits flipped by this mutation operator as a mutation potential gives $M = n \cdot e^{-\rho v}$. According to both experimental and theoretical analyses [38], a reasonable value for ρ is $\ln n$. We call this mutation potential $M_{\text{expoF}(x)}$ and define it as:

$$M_{\text{expoF}(x)}(x) := \left\lfloor n^{1 - \frac{f(x)}{f(\text{best})}} \right\rfloor.$$

Hamming Distance Based Exponential Decrease

It is well understood that fitness-proportional selection is sensitive to the difference between the fitness values of candidate solutions, i.e., it struggles to distinguish between solutions that have similar, yet different, fitness values [50]–[53]. For this reason, nowadays selection operators that rank individuals by fitness (e.g., tournament, linear ranking, comma selection) are generally preferred. We also consider a measure that avoids using the fitness values directly because, apart from the above consideration, the ideal amount of bits to be flipped

clearly depends on the genotypic distance of the candidate solution to the optimum rather than on their difference in fitness values. To this end, we suggest a mutation potential which is similar to $M_{\text{expoF}(x)}$ with the exception that it uses the normalised Hamming distance to the best estimate rather than the normalised fitness. We call this mutation potential M_{expoHD} and define it as $M_{\text{expoHD}} = \left\lfloor n \cdot e^{-\rho \frac{n - H(x, \text{best})}{n}} \right\rfloor$ where n is the problem size (the maximum Hamming distance between any two points), $H(x, \text{best})$ is the Hamming distance to the best known solution and ρ is the decay of the mutation potential. For the choice of $\rho = \ln n$, we get:

$$M_{\text{expoHD}}(x) := \left\lfloor n^{\frac{H(x, \text{best})}{n}} \right\rfloor.$$

III. PERFORMANCE GAINS IN IDEAL CONDITIONS

In this section, we evaluate the performance of the three different IPH operators, assuming the optimum is known (i.e., $\text{best} = \text{opt}$). Under this assumption, the operators exhibit their ideal behaviour (i.e., the mutation potential decreases with the desired rate as the optimum is approached). Our aim is to evaluate what speed-ups can be achieved in ideal conditions compared to the well-studied static mutation potentials. To achieve such comparisons we perform runtime analyses of the (1+1) IA using the IPH on the standard ONEMAX and LEADINGONES unimodal benchmark functions for which the performance of the same algorithm using static mutation potentials is known [39]. The *simple-to-define* ONEMAX function counts the number of 1-bits in a bit string and is normally used to evaluate the hill climbing ability of algorithms [5], [9], [54], [55]. On the other hand, LEADINGONES is a slightly harder unimodal problem which counts the consecutive number of 1-bits at the beginning of the bit string before the first 0-bit *or the end of the string*. Evolutionary algorithms using standard bit mutation (or bit flip local search mutation) optimise the functions respectively in $\Theta(n \log n)$ and $\Theta(n^2)$ expected fitness function evaluations [12], [13], [43], [56], [57], while static hypermutations are a linear factor slower [39].

While in practice both evolutionary algorithms and artificial immune systems use a population of solutions, in this paper we consider single trajectory algorithms to better highlight the performance of all the considered hypermutation operators. The pseudo-code of a simple AIS, the (1+1) Immune Algorithm ((1+1) IA), is given in Algorithm 1. It simply uses one candidate solution in each iteration to which inversely proportional hypermutations are applied. For the function $\text{Hypermutate}(x)$, IPH performs two steps: 1) it calculates the mutation potential M , and 2) it creates y by flipping at most M distinct bits of x selected uniformly at random without replacement one after another until a constructive mutation happens. Otherwise it returns the last constructed solution. A constructive mutation is a mutation step where the evaluated bit string is at least as fit as the parent x . The pseudo-code of the operator once the mutation potential M has been calculated is depicted in Algorithm 2. The results proven in this section and comparisons with static hypermutations and the IPH operators from the literature are summarised in Table II.

TABLE II

COMPARISON OF THE EXPECTED RUNTIMES OBTAINED BY DIFFERENT HYPERMUTATION SCHEMES FOR ONEMAX AND LEADINGONES. THE ORIGINAL IPH OPERATOR OF OPT-IA [35] HAS THE SAME EXPECTED RUNTIMES AS THE $(1 + 1)$ IA USING STATIC HYPERMUTATIONS [42]. ** INDICATES THAT THE RESULT CAN BE EASILY DRAWN FROM THE ANALYSIS FOR ONEMAX [38].

$(1 + 1)$ IA	ONEMAX	LEADINGONES
Using static hypermutation	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]
Clonalg with linear decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)^*}$
Clonalg with expo decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)^*}$
Traditional Opt-IA	$\Theta(n^2 \log n)$ [42]	$\Theta(n^3)$ [42]
FCM with M_{linHD}	$\Theta(n^2)$	$\Theta(n^3)$
FCM with $M_{\text{expoF}(x)}$	$\Theta(n^{3/2} / \log n)$	$\Theta(n^3 / \log^2 n)$
FCM with M_{expoHD}	$\Theta(n^{3/2} / \log n)$	$\Theta(n^{5/2} / \log^2 n)$

Algorithm 1 $(1 + 1)$ IA

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: Evaluate  $f(x)$ ;
3: while termination condition is not satisfied do
4:   Update mutation potential  $M$ ;
5:    $y := \text{Hypermutate}(x, M)$ ;
6:   if  $f(y) \geq f(x)$  then
7:      $x := y$ .
8:   end if
9: end while

```

Before stating the main results, we first introduce two lemmata. The first is the Ballot theorem which is used throughout this paper to derive lower bounds on the expected runtime of the algorithms [39], [42], [58]. This theorem is essentially used to show the probability of picking more 1-bits than 0-bits during one hypermutation operation given that there are more 1-bits in the initial bit-string (or vice-versa).

Lemma 1 (Ballot Theorem [58]). *In a ballot, suppose that candidate P receives p votes and candidate Q receives q votes such that $p > q$. The probability that throughout the counting P is always ahead of Q is $(p - q)/(p + q)$.*

In the context of hypermutation operators, the vote counts are the initial number of 0 and 1-bits in the solutions, while counting the votes correspond to flipping them until the first constructive mutation. The probability that a constructive mutation is found is equivalent to the probability of the Q candidate having as many votes counted as the other during any time in the counting process.

Our second lemma will be used to transform inverse polynomial exponents that arise from the exponential potential.

Lemma 2. *For large enough n and any arbitrary constant ϵ , $n^{1/n^\epsilon} = (1 + \frac{\ln n}{n^\epsilon})(1 \pm o(1))$.*

Proof. $n^{\frac{1}{n^\epsilon}} = e^{\frac{\ln(n)}{n^\epsilon}} = 1 + \frac{\ln(n)}{n^\epsilon} + o(\frac{\ln(n)}{n^\epsilon})$ by the Taylor's series of the exponent. $\square$

Algorithm 2 HYPERMUTATE(x, M)

```

1:  $y \leftarrow x$ ;
2: for  $i = 1$  to  $M$  do
3:   Flip one previously unflipped bit of  $y$  chosen uniformly at random;
4:   Evaluate  $f(y)$ ;
5:   if  $f(y) \geq f(x)$  then
6:     return  $y$ ;
7:   end if
8: end for
9: return  $y$ ;

```

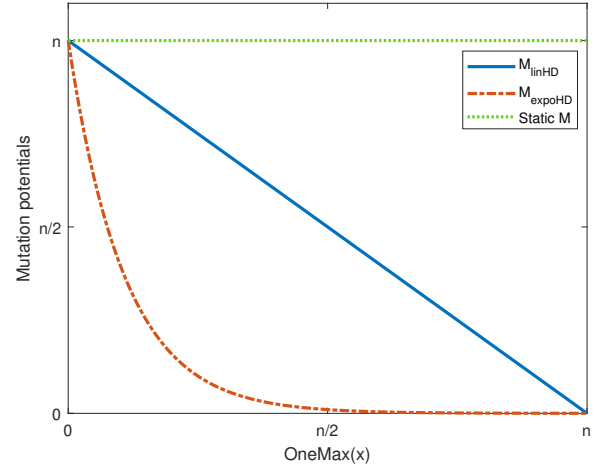


Fig. 1. Static and inversely proportional mutation potentials for ONEMAX.

A. Performance for ONEMAX in Ideal Conditions

The following theorems derive the expected runtimes using each of the three IPH operators for the function $\text{ONEMAX}(x) := \sum_{i=1}^n x_i$. Figure 1 shows how the studied mutation potentials ideally decrease during the run of the algorithm when optimising the function. Notice that for ONEMAX the behaviours of $M_{\text{expoF}(x)}$ and M_{expoHD} are the same. By decreasing the potential linearly with the decrease of the Hamming distance, a logarithmic factor may be shaved off from the expected runtime of the $(1 + 1)$ IA compared to the expected runtime achieved by the same algorithm using static hypermutation potentials.

Theorem 1. *The $(1 + 1)$ IA using IPH with M_{linHD} optimises ONEMAX in $\Theta(n^2)$ expected fitness function evaluations.*

Proof. Considering i as the number of 0-bits in the candidate solution, the probability of improvement in the first step is i/n . Knowing that at most n improvements are needed to find the optimum and that in case of failure $H(x, \text{opt}) = i$ fitness function evaluations would be wasted, the total expected time to optimise ONEMAX is at most $\sum_{i=1}^n \frac{n}{i} \cdot i = O(n^2)$.

For the lower bound, we use the Ballot theorem (Lemma 1). We flip at most $M \leq n$ bits, and the probability that an improvement happens is the probability that at some moment there will be at least as many flipped zeros as there are flipped ones. So pessimistically we can assume that $M = n$ bits

are always flipped, since it maximizes the chances of this unfortunate event.

By Chernoff bounds, the number of 0-bits in the initialised solution is at least $n/3$ w.o.p. Considering the number of 0-bits as $i = q$ and the number of 1-bits as $n - i = p$, the probability of an improvement is at most $1 - (p - q)/(p + q) = 1 - (n - 2i)/n = \frac{2i}{n}$ by the Ballot theorem, where $i = H(x, \text{opt})$. This means that we need to wait at least $\frac{n}{2i}$ iterations to see an improvement and each time the mutation operator fails to improve the fitness ($\frac{n}{2i} - 1$ times in expectation), i fitness function evaluations will be wasted. Considering that at least $n/3$ improvements are needed, the expected time to optimise ONEMAX is larger than $(1 - 2^{-\Omega(n)}) \sum_{i=1}^{n/3} (\frac{n}{2i} - 1) \cdot i = \Omega(n^2)$ since the operator cannot improve the ONEMAX function value by more than one in each application due to FCM. The factor $(1 - 2^{-\Omega(n)})$ accounts for the w.o.p. Chernoff event; since $2^{-\Omega(n)} = o(1)$, the bound holds unconditionally. $\square$

The following theorem shows that a greater speed up may be achieved if the potential decreases exponentially rather than linearly. Note that for ONEMAX the Hamming distance of a solution to the optimum and its difference in fitness are the same. Hence, there is no difference between the behaviours of the algorithm when using $M_{\text{expoF}(x)}$ and M_{expoHD} . Figure 1 highlights the reason why these operators waste fewer fitness evaluations.

Theorem 2. *The $(1 + 1)$ IA using IPH with either $M_{\text{expoF}(x)}$ or M_{expoHD} optimises ONEMAX in $\Theta(n^{3/2}/\log n)$ expected fitness function evaluations.*

Proof. We prove the results for $M_{\text{expoF}(x)}$ which applies to M_{expoHD} as well, since for ONEMAX the Hamming distance to the optimum equals the number of 0-bits, giving $M_{\text{expoF}(x)} = M_{\text{expoHD}}$.

Upper bound. By Chernoff bounds, the initialised solution has at most $n/2 + n^{2/3}$ zero-bits with overwhelming probability (w.o.p.). If this event fails, we assume $M = n$ throughout, which still yields polynomial expected runtime and contributes $o(1)$ to the total expected time. Let i denote the current number of 0-bits. The probability of improvement in the first bit-flip is at least i/n , at most $n/2 + n^{2/3}$ improvements are needed, and each failed mutation wastes at most $n^{i/n}$ fitness evaluations. Hence,

$$E(T) \leq o(1) + \sum_{i=1}^{n/2+n^{2/3}} \frac{n}{i} \cdot n^{i/n} = n \sum_{i=1}^{n/2+n^{2/3}} \frac{n^{i/n}}{i}.$$

We split the sum into three ranges. Fix $\delta = 1/4$.

Range 1: $i \in [1, n/\ln n]$. For $i \leq n/\ln n$ we have $n^{i/n} \leq n^{1/\ln n} = e$. Therefore,

$$n \sum_{i=1}^{\lfloor n/\ln n \rfloor} \frac{n^{i/n}}{i} \leq n \cdot e \cdot H_{\lfloor n/\ln n \rfloor} = O(n \log \log n)$$

Range 2: $i \in [n/\ln n, \delta n]$. For $i \leq \delta n$ we have $n^{i/n} \leq n^\delta$.

Therefore,

$$n \sum_{i=\lceil n/\ln n \rceil}^{\lfloor \delta n \rfloor} \frac{n^{i/n}}{i} \leq n \cdot n^\delta \sum_{i=\lceil n/\ln n \rceil}^{\lfloor \delta n \rfloor} \frac{1}{i} \leq n^{1+\delta} \cdot O(\ln(\delta \ln n))$$

since $\delta < 1/2$.

Range 3: $i \in [\delta n, n/2 + n^{2/3}]$. For $i \geq \delta n$ we have $1/i \leq 1/(\delta n) = O(1/n)$. Therefore,

$$n \sum_{i=\lceil \delta n \rceil}^{n/2+n^{2/3}} \frac{n^{i/n}}{i} \leq \frac{1}{\delta} \sum_{i=\lceil \delta n \rceil}^{n/2+n^{2/3}} n^{i/n}.$$

This is a partial geometric series with common ratio $r = n^{1/n}$:

$$\sum_{i=\lceil \delta n \rceil}^{n/2+n^{2/3}} n^{i/n} = n^\delta \cdot \frac{n^{(1/2+n^{-1/3}-\delta)} - 1}{n^{1/n} - 1}.$$

By Lemma 2, $n^{1/n} - 1 = (\ln n)/n \cdot (1 + o(1))$, and $n^{1/2+n^{-1/3}} = \sqrt{n} \cdot e^{(\ln n)/n^{1/3}} = \sqrt{n} \cdot e^{o(1)} = O(\sqrt{n})$, so

$$\frac{n^{1/2+n^{-1/3}}}{\ln n/n} = O\left(\frac{n^{3/2}}{\ln n}\right).$$

Hence the Range 3 contribution is $O(n^{3/2}/\log n)$.

Combining all three ranges, the dominant term is Range 3, giving $E(T) = O(n^{3/2}/\log n)$.

Lower bound. By symmetry of the uniform initialisation, with probability at least $1/2$ the initial number of 0-bits is $i \geq n/2$. We condition on this event.

Since the operator cannot improve the ONEMAX value by more than one per application due to FCM, every level must be visited. At level i (i.e., the current solution has i zero-bits), by the Ballot theorem (Lemma 1) the probability of improvement is at most $2i/n$. The expected waiting time for an improvement is therefore at least $n/(2i)$, and each of the at least $n/(2i) - 1$ failed attempts wastes $n^{i/n}$ fitness evaluations. For $i \leq n/2$ we have $n/(2i) \geq 1$, so the expected cost at level i is at least

$$\left(\frac{n}{2i} - 1\right) \cdot n^{i/n} \geq \frac{1}{2} \cdot \frac{n}{2i} \cdot n^{i/n} \quad \text{for } i \leq n/4,$$

and more simply, for all $i \in [n/4, n/2]$ we use $n/i \geq 2$ to write

$$\frac{n}{2i} \cdot n^{i/n} \geq n^{i/n}.$$

Therefore the total expected runtime is at least

$$\frac{1}{2} \sum_{i=n/4}^{n/2} \frac{n}{2i} \cdot n^{i/n} \geq \frac{n}{4} \sum_{i=n/4}^{n/2} \frac{n^{i/n}}{i}.$$

Using $n/i \geq 2$ for all $i \leq n/2$, this is at least

$$\frac{n}{4} \cdot 2 \cdot \frac{1}{n} \sum_{i=n/4}^{n/2} n^{i/n} = \frac{1}{2} \sum_{i=n/4}^{n/2} n^{i/n}.$$

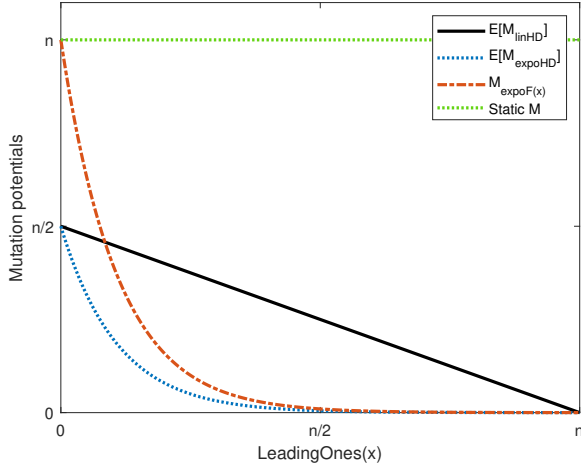


Fig. 2. Static and inversely proportional mutation potentials for LEADING-ONES. Expected values are shown for those using Hamming distance.

The geometric sum evaluates as

$$\begin{aligned} \sum_{i=n/4}^{n/2} n^{i/n} &= n^{1/4} \cdot \frac{n^{1/4} - 1}{n^{1/n} - 1} \geq \frac{n^{1/2} \cdot (1 - n^{-1/4})}{\ln n/n \cdot (1 + o(1))} \\ &= \Omega\left(\frac{n^{3/2}}{\ln n}\right), \end{aligned}$$

using $n^{1/n} - 1 = (\ln n)/n \cdot (1 + o(1))$ from Lemma 2. Hence the expected runtime is $\Omega(n^{3/2}/\log n)$. $\square$

B. Performance for LEADINGONES in Ideal Conditions

In the previous subsection, it was shown that decreasing the mutation rate linearly with the Hamming distance to the optimum allows for a logarithmic factor speed-up for ONEMAX compared to using static hypermutations. The following theorem shows that no asymptotic improvement over static mutation potentials is achieved for LEADINGONES: $\sum_{i=1}^n \prod_{j=1}^i x_i$. The main reason for the lack of improvement is that a linear mutation potential is sustained until at least a linear number of improvements are achieved.

Theorem 3. *The $(1 + 1)$ IA using IPH with M_{linHD} optimises LEADINGONES in $\Theta(n^3)$ expected fitness function evaluations.*

Proof. The proof for the lower bound follows that of Theorem 3.6 in [46] for the expected runtime of the $(1 + 1)$ IA using static hypermutation with potential $M := cn$ with the exception that the amount of wasted fitness function evaluations in case of failure at finding an improvement within a hypermutation is now $H(x, \text{opt})$ instead of cn .

Considering i as the number of leading 1-bits, we denote the expected number of fitness function evaluations until an improvement happens by $E(f_i)$. Any such candidate solution has i leading 1-bits with a 0-bit following, and then $n - i - 1$ trailing bits. The trailing bits stay uniformly distributed throughout the run since the hypermutation operation can only terminate with an accepted solution by immediately flipping

the leftmost 0-bit, which does not alter the trailing bits, or by immediately flipping a random trailing bit, which does not affect the probability of the flipped position being 1 or 0 after it is flipped [43].

We take into account three possible events E_1 , E_2 and E_3 that can happen in the first bit flip:

- E_1 is the event of flipping a leading 1-bit which happens with probability i/n and allows the hypermutation to keep flipping $M - 1$ more bits,
- E_2 is the event of flipping the first 0-bit which happens with probability $1/n$, and,
- E_3 is the event of flipping any other bit which happens with probability $(n - i - 1)/n$.

Since FCM stops the hypermutation operator as soon as an equally fit solution is sampled, E_2 and E_3 each stop the hypermutation operation. So we get $E(f_i|E_1) = H(x, \text{opt}) + E(f_i)$, $E(f_i|E_2) = 1$, and $E(f_i|E_3) = 1 + E(f_i)$.

Hence, by the law of total expectation, the expected number of fitness function evaluations for every improvement is $E(f_i) = \frac{i}{n} (H(x, \text{opt}) + E(f_i)) + \frac{1}{n} \cdot 1 + \frac{n-i-1}{n} (1 + E(f_i))$. Solving the equation for $E(f_i)$ gives us $E(f_i) = i \cdot H(x, \text{opt}) + n - i \geq i \cdot H(x, \text{opt})$.

Since the bits after the leftmost 0-bit are distributed uniformly at random [43], while there are still a linear number of trailing bits the value of $H(x, \text{opt})$ is not less than $1 + \frac{(n-i-1)}{2} - \frac{\epsilon}{2}n$ w.o.p. by Chernoff bounds.

Thus, we conclude $E(f_i) \geq \frac{(1-\epsilon)in-i^2+i}{2} \cdot (1 - o(1))$. The right hand side of the inequality has a second derivative of -2 with respect to i , and therefore it is concave. For the interval $i \in [n/2 - \epsilon^*n, n/2 + \epsilon^*n]$ its value is of order $\Omega(n^2)$ for some arbitrarily small $\epsilon^* > 0$.

As we are computing the lower bound, we have to take into account the probability of skipping any level i . To do this, we need to calculate the expected number of consecutive 1-bits that follow the leftmost 0-bit (called free riders [43]).

The expected number of free riders is $\sum_{i=1}^{n-i-1} i \cdot 1/2^{i+1} < \frac{1}{2} \cdot \sum_{i=0}^{\infty} i \cdot (1/2)^i = \frac{1}{2} \cdot \frac{1/2}{(1-1/2)^2} = 1$. This means that p_i^* , the probability of not skipping level i (i.e., the probability that the number of leading 1-bits increases by exactly one when the leftmost 0-bit is flipped), is $\Omega(1)$. This holds because the expected number of free-riders (consecutive 1-bits following the leftmost 0-bit) is at most $\sum_{j \geq 1} j \cdot 2^{-(j+1)} < 1$, and the level advances by exactly 1 with probability at least $1/2$; see also [43].

On the other hand, with probability at least $1 - 2^{-n/3}$, the initial solution does not have more than $n/3$ leading 1-bits. Thus, we obtain a lower bound of $(1 - 2^{-n/3}) \sum_{i=n/2-\epsilon^*n}^{n/2+\epsilon^*n} p_i^* \cdot E(f_i) = \Omega(1) \sum_{i=n/2-\epsilon^*n}^{n/2+\epsilon^*n} \frac{in-i^2+i-i\epsilon n}{2} = \Omega(n^3)$ on the expected number of fitness function evaluations.

Now we prove the upper bound. The probability of improvement in each step is at least $1/n$ which is the probability of flipping the leftmost 0-bit in the first bit-flip. Since at most n improvements are needed and each failure at improving yields at most n wasted fitness function evaluations, the expected time to find the optimum is at most $\sum_{i=1}^n n \cdot n = O(n^3)$. $\square$

We can now proceed with the analysis of the exponential-based potentials. Theorem 4 shows that the fitness difference based exponential mutation potential provides a $\Theta(\log^2 n)$ factor speed-up compared to static mutation potentials.

Theorem 4. *The $(1+1)$ IA using IPH with $M_{\text{expoF}(x)}$ optimises LEADINGONES in $\Theta(n^3/\log^2 n)$ expected fitness function evaluations.*

Proof. At level i (i.e., with i leading 1-bits), the first bit flipped by the hypermutation is chosen uniformly from $\{1, \dots, n\}$. With probability i/n it hits a leading 1-bit, destroying the prefix and using the full potential of $n^{(n-i)/n}$ evaluations with no improvement; with probability $1/n$ it hits the leftmost 0-bit, yielding an improvement at cost 1; with probability $(n-i-1)/n$ it hits a tail bit, costing 1 evaluation with no improvement. By the law of total expectation,

$$E(f_i) = \frac{i}{n} (n^{(n-i)/n} + E(f_i)) + \frac{1}{n} \cdot 1 + \frac{n-i-1}{n} (1 + E(f_i)),$$

and solving gives

$$E(f_i) = i \cdot n^{(n-i)/n} + n - i. \quad (1)$$

We evaluate the sum $S := \sum_{i=0}^{n-1} E(f_i)$, which governs both bounds. Setting $x := n^{-1/n} < 1$,

$$S = \sum_{i=0}^{n-1} (i \cdot n^{(n-i)/n} + n - i) = n \sum_{i=1}^{n-1} i x^i + \underbrace{\sum_{i=0}^{n-1} (n - i)}_{\Theta(n^2)}.$$

By the standard identity $\sum_{i=1}^{\infty} i x^i = x/(1-x)^2$ and Lemma 2 (with $\epsilon = 1$),

$$1 - x = 1 - n^{-1/n} = \frac{\ln n}{n} (1 + o(1)),$$

so

$$\frac{x}{(1-x)^2} = \frac{1 + o(1)}{\left(\frac{\ln n}{n}\right)^2} = \frac{n^2(1 + o(1))}{\ln^2 n}.$$

The tail of the infinite series from $i = n$ onwards contributes $O(n/\ln n)$ (since $x^n = n^{-1}$), which is $o(n^2/\ln^2 n)$. Therefore $\sum_{i=1}^{n-1} i x^i = \Theta(n^2/\ln^2 n)$, giving

$$S = \Theta\left(\frac{n^3}{\log^2 n}\right). \quad (2)$$

Upper bound. Each improvement advances the LEADINGONES value by at least one, so the total expected number of evaluations satisfies $\mathbb{E}[T] \leq \sum_{i=0}^{n-1} E(f_i) = S = O(n^3/\log^2 n)$.

Lower bound. With probability $\Pr[x_1 = 0] = 1/2$ the initial solution has LEADINGONES value 0, so every level $i \in \{0, \dots, n-1\}$ must be traversed. Following the proof of Theorem 3, the expected number of free-riders per improvement is at most 2, so each level i is visited with probability $p_i^* = \Omega(1)$. Hence

$$\mathbb{E}[T] \geq \frac{1}{2} \sum_{i=0}^{n-1} p_i^* \cdot E(f_i) = \Omega(1) \cdot S = \Omega\left(\frac{n^3}{\log^2 n}\right).$$

An advantage of Hamming distance-based exponential decays of the mutation potential compared to fitness-based ones is provided by the following theorem for LEADINGONES. The reason can be seen in Figure 2. While the initial fitness is very low, the potential is very high, but the actual number of bits that have to be flipped to reach the optimum (i.e., the Hamming distance) is much smaller. More precisely, fitness-based potentials suggest to flip n bits when only $n/2$ bits have to be flipped in expectation to reach the optimum in one step. M_{expoHD} exploits this property, thus wastes fewer fitness evaluations than $M_{\text{expoF}(x)}$.

Theorem 5. *The $(1+1)$ IA using IPH with M_{expoHD} optimises LEADINGONES in $\Theta(n^{5/2}/\log^2 n)$ expected fitness function evaluations.*

Proof. At level i (i.e., with i leading 1-bits), the solution has the form $x = 1^i 0 b_{i+2} \dots b_n$ where the tail bits $b_{i+2}, \dots, b_n$ are uniformly distributed. The Hamming distance to the optimum 1^n is therefore $H(x, \text{opt}) = 1 + B$ where $B \sim \text{Bin}(n-i-1, 1/2)$.

Concentration of the mutation potential. By a Chernoff bound,

$$\begin{aligned} \Pr\left[\left|B - \frac{n-i-1}{2}\right| > n^{2/3}\right] &\leq 2 \exp\left(-\frac{2n^{4/3}}{n-i-1}\right) \\ &\leq 2e^{-2n^{1/3}} = o(n^{-2}). \end{aligned}$$

A union bound over all n levels and polynomially many iterations shows that, with overwhelming probability (w.o.p.), the Hamming distance satisfies

$$H(x, \text{opt}) = 1 + \frac{n-i-1}{2} \pm n^{2/3} = \frac{n-i+1}{2} \pm n^{2/3}$$

throughout the entire run. Setting $h_i := (n-i+1)/(2n)$ and noting that $n^{\pm n^{-1/3}} = e^{\pm n^{-1/3} \ln n} = 1 \pm o(1)$, the mutation potential satisfies

$$M = n^{H/n} = (1 \pm o(1)) n^{h_i} = (1 \pm o(1)) n^{1/2 - (i-1)/(2n)}. \quad (3)$$

If the rare event that $|B - (n-i-1)/2| > n^{2/3}$ occurs, we pessimistically assume $M = n$, which yields polynomial expected runtime and contributes $o(1)$ to the total expectation.

Per-level cost. The first bit flipped by the hypermutation is chosen uniformly from $\{1, \dots, n\}$. With probability i/n it hits a leading 1-bit, destroying the prefix and consuming the full potential of M evaluations with no improvement; with probability $1/n$ it hits the leftmost 0-bit, yielding an improvement at cost 1; with probability $(n-i-1)/n$ it hits a tail bit, costing 1 evaluation with no improvement. By the law of total expectation,

$$E(f_i) = \frac{i}{n} (M + E(f_i)) + \frac{1}{n} \cdot 1 + \frac{n-i-1}{n} (1 + E(f_i)),$$

and solving gives

$$E(f_i) = i \cdot M + n - i. \quad (4)$$

By (3), this equals $(1 \pm o(1)) i \cdot n^{1/2 - (i-1)/(2n)} + n - i$ w.o.p.

□ **Evaluating the sum.** We evaluate $S := \sum_{i=0}^{n-1} E(f_i)$, which

governs both bounds. Factoring out $n^{1/2+1/(2n)}$ from the leading term and setting $y := n^{-1/(2n)} < 1$,

$$\begin{aligned} S &= (1 \pm o(1)) \sum_{i=0}^{n-1} i \cdot n^{1/2-(i-1)/(2n)} + \underbrace{\sum_{i=0}^{n-1} (n-i)}_{\Theta(n^2)} \\ &= (1 \pm o(1)) n^{1/2+1/(2n)} \sum_{i=1}^{n-1} i y^i + \Theta(n^2). \end{aligned}$$

By the standard identity $\sum_{i=1}^{\infty} i y^i = y/(1-y)^2$ and Lemma 2 (with $\epsilon = 1/2$),

$$1 - y = 1 - n^{-1/(2n)} = \frac{\ln n}{2n} (1 + o(1)),$$

so

$$\frac{y}{(1-y)^2} = \frac{1 + o(1)}{\left(\frac{\ln n}{2n}\right)^2} = \frac{4n^2(1 + o(1))}{\ln^2 n}.$$

The tail of the infinite series from $i = n$ onwards: since $y^n = (n^{-1/(2n)})^n = n^{-1/2}$,

$$\begin{aligned} \sum_{i=n}^{\infty} i y^i &= y^n \left(\frac{n}{1-y} + \frac{y}{(1-y)^2} \right) = n^{-1/2} \cdot O\left(\frac{n^2}{\ln n}\right) \\ &= O\left(\frac{n^{3/2}}{\ln n}\right) = o\left(\frac{n^2}{\ln^2 n}\right). \end{aligned}$$

Therefore $\sum_{i=1}^{n-1} i y^i = \Theta(n^2 / \ln^2 n)$. Since $n^{1/2+1/(2n)} = n^{1/2} \cdot n^{1/(2n)} = n^{1/2}(1 + o(1))$,

$$S = \Theta\left(\frac{n^{5/2}}{\log^2 n}\right). \quad (5)$$

Finally, we follow the same arguments in Theorem 4 to establish the upper and the lower bound using S . $\square$

Overall, in this section, we have shown that considerable speed-ups can be achieved in the ideal condition where the optimum is known by using inversely proportional hypermutations compared to static ones. We conclude that M_{expoHD} provides larger hill-climbing speed-ups compared to the other mutation potentials and is stable to the scaling of fitness functions. However, M_{expoHD} only allows to flip at most $\sqrt{n}$ bits when far away from the optimum (e.g., at initialisation for ONEMAX), which means that the algorithm hardly ever performs large mutations except at initialisation defeating the purpose of having a hypermutation operator at all. Thus, we believe that M_{linHD} , while being slower during the hillclimbing phases, provides a better balance between high and low mutation rates for hard multimodal functions when the total runtime is reasonably dominated by the time required to escape from local optima. Notice from Table II that using a linear decay operator with FCM rather than in Clonalg still makes a difference between polynomial and exponential time for the functions when the optimum is known. In the following section we will highlight how being able to perform large mutations more often allows the algorithm to be effective in practice when the optimum is unknown for multimodal functions with very different characteristics.

IV. BEHAVIOUR OF IPH IN PRACTICE

In this section, we consider the usage of IPH in realistic applications where the optimum is unknown. To this end, the mutation rate of the IPH operator will be inversely proportional to the best seen solution rather than to the unknown optimum.

We combine IPH using M_{linHD} with hybrid ageing, as in the Opt-IA AIS [35], and embed it in a $(1+1)$ Opt-IA shown in Algorithms 3 and 4. To use an ageing operator, an age $\alpha(x)$, is assigned to the individual x . After the Hypermutate(x) operator, the ageing operator is applied. Among the different variants of ageing, *hybrid* ageing has been shown to be very efficient at escaping local optima [39], [40], [45], [59]. Using this operator, individuals are assigned with initial age zero. During each iteration of the algorithm, the age increases by 1 and is passed to the offspring if it does not improve over its parent's fitness. If the offspring is fitter than the parent, then its age is set to 0. At the end of each iteration, any individual with age larger than a threshold τ is removed with probability $p_{\text{die}} = 1/2$. In case there is no other individual left, a new individual is initialised uniformly at random³.

Ageing has been shown to enable algorithms to escape from local optima either by identifying a gradient leading away from it or by restarting the whole optimisation process. Our aim behind the algorithm design is that by escaping local optima via ageing, the number of local optima that are identified by the algorithm increases over time. Thus as time goes by the IPH should approximate its ideal behaviour better and better.

Concerning the IPH operator for practical applications we reintroduce the parameter $c \leq 1$ for determining the mutation potential when it grows linearly with the Hamming distance as in the original Opt-IA [42]:

$$M_{\text{linHD}} = \max(\lfloor c \cdot H(x, \text{best}) \rfloor, 1)$$

The parameter will be useful to avoid always ending up in the same basins of attraction as we will see in the analyses of both multimodal function classes we consider in this paper.

We also have a design choice regarding the IPH operator of whether to update the *best* seen solution only when a strictly better solution is identified or whether to update it when the algorithm finds a new solution that is as good as the previous best one. This design choice leads to considerably different algorithmic behaviours. If the current *best* is updated to the new solution when individuals of equal fitness are encountered, then the algorithm will be extremely fast on unimodal functions because the mutation rate will always be the smallest possible (i.e., 1-bit flips). So if the aim is to optimise easy unimodal functions then this version, which we call $(1+1)$ Opt-IA_{LASTBEST} is the preferred choice, as it can be proved to run in optimal asymptotic runtime on benchmark functions such as ONEMAX (i.e., $\Theta(n \log n)$) and LEADINGONES (i.e., $\Theta(n^2)$). However, this choice would mean that the algorithm would keep low mutation rates when trapped in the best found local optima if they are encountered again and again with high probability after the algorithm is restarted.

³When a population is used, the required number of individuals are re-initialised to reach the population size again after ageing is applied.

Algorithm 3 $(1 + 1)$ Opt-IA_{LASTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ ;
3: evaluate  $f(x)$ ;
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ ;
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ ;
9:   else
10:     $\alpha(y) := \alpha(x)$ ;
11:   end if
12:   if  $f(y) \geq f(\text{best})$  then
13:     then set  $\text{best} := y$ ;
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
19:       if  $f(w) \geq f(\text{best})$  then
20:          $\text{best} := w$ 
21:       end if
22:     end if
23:   end for
24:   if  $f(y) \geq f(x)$  then
25:      $x := y$ 
26:   end if
27: end while

```

The other choice, that of only updating the *best* when a strictly better solution is encountered, allows individuals trapped on a large plateau of local optima to increase their *Hamming* distance to the best solution, thus increases the chances of escaping via higher mutation rates. This version, which we call $(1 + 1)$ Opt-IA_{FIRSTBEST}, allows to use high mutation rates more often at the expense of being slower on some unimodal functions like LEADINGONES as we will see. With this operator we remember to update the *best* to the last seen (*best*) individual of highest fitness (i.e. the current solution) when ageing is triggered because this limits the *Hamming* distance between the new solution and the current best when a local optima is escaped by accepting solutions of lower fitness, thus decreases the chances of jumping back to the previous local optimum.

We start with a general reduction. When the mutation potential remains 1 and ageing is unlikely to trigger, both algorithm variants execute single-bit flips with accept-if- $\geq$ and thus behave identically to RLS. Lemma 3 below formalises this observation; we then apply it to show that both algorithms optimise ONEMAX in $\Theta(n \log n)$ and that $(1 + 1)$ Opt-IA_{LASTBEST} optimises LEADINGONES in $\Theta(n^2)$.

Lemma 3 (RLS Reduction). *Let A denote the event that ageing triggers before the optimum is found and set $p := \Pr[A]$. If (i) $M = 1$ throughout every run in which A does not occur, and (ii) $p = o(1)$ and $\tau p = o(1)$, then $\mathbf{E}[T'] = (1 \pm o(1)) T_{\text{RLS}}$, where T_{RLS} is the expected runtime*

Algorithm 4 $(1 + 1)$ Opt-IA_{FIRSTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ 
3: evaluate  $f(x)$ 
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ 
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ 
9:   else
10:     $\alpha(y) := \alpha(x)$ 
11:   end if
12:   if  $f(y) > f(\text{best})$  then
13:      $\text{best} := y$ 
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         if  $f(w) = f(\text{best})$  then
19:            $\text{best} := w$ 
20:         end if
21:       reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
22:       if  $f(w) > f(\text{best})$  then
23:          $\text{best} := w$ 
24:       end if
25:     end if
26:   end for
27:   if  $f(y) \geq f(x)$  then
28:      $x := y$ 
29:   end if
30: end while

```

of RLS on the same function.

Proof. Condition (i) ensures that, conditional on $\neg A$, the algorithm performs only single-bit flips with accept-if- $\geq$, i.e. it runs identically to RLS. This holds for $(1 + 1)$ Opt-IA_{LASTBEST} on any function and for $(1 + 1)$ Opt-IA_{FIRSTBEST} whenever no neutral moves exist (so that every acceptance updates *best*).

Upper bound. When ageing triggers, the algorithm restarts; the expected additional time is at most $\mathbf{E}[T']$. Hence

$$\mathbf{E}[T'] \leq (\tau + \mathbf{E}[T'])p + \mathbf{E}[T_{\text{RLS}} \mid \neg A] \Pr[\neg A].$$

Rearranging and using $\tau p = o(1)$ from (ii):

$$\begin{aligned} \mathbf{E}[T'] &\leq \frac{\mathbf{E}[T_{\text{RLS}} \mid \neg A] \Pr[\neg A] + \tau p}{1 - p} \\ &\leq \frac{T_{\text{RLS}} + o(1)}{1 - o(1)} = (1 + o(1)) T_{\text{RLS}}. \end{aligned}$$

Lower bound. Before event A occurs, at most $T_{\text{RLS}} + \tau$ steps elapse; afterwards RLS needs at most T_{RLS} more steps. So $\mathbf{E}[T_{\text{RLS}} \mid A] \leq 2T_{\text{RLS}} + \tau$, and

$$T_{\text{RLS}} \leq \mathbf{E}[T_{\text{RLS}} \mid \neg A] \Pr[\neg A] + (2T_{\text{RLS}} + \tau)p,$$

giving $\mathbf{E}[T_{\text{RLS}} \mid \neg A] \Pr[\neg A] \geq T_{\text{RLS}}(1 - 2p) - \tau p = (1 - o(1)) T_{\text{RLS}}$. Since $\mathbf{E}[T'] \geq \mathbf{E}[T' \mid \neg A] \Pr[\neg A] = \mathbf{E}[T_{\text{RLS}} \mid \neg A] \Pr[\neg A]$, the lower bound follows. $\square$

Theorem 6. *The expected runtime of $(1+1)$ Opt-IA_{FIRSTBEST} and $(1+1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising ONEMAX is $\Theta(n \log n)$ with $\tau \geq Cn \ln n$ for a sufficiently large constant C .*

Proof. Immediately after initialisation $best = x$, so $H(x, best) = 0$ and $M = c \cdot H = 0$; since the potential is at least 1 by convention, $M = 1$. On ONEMAX, every bit flip changes fitness by ± 1 , so no neutral moves exist; every accepted solution strictly improves fitness and triggers $best \leftarrow x$ in both variants, maintaining $H = 0$ and $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), at any level $k \geq 1$ the improvement probability is $k/n \geq 1/n$, so the probability of τ consecutive non-improvements is at most $(1 - 1/n)^\tau \leq e^{-\tau/n} \leq e^{-C \ln n} = n^{-C}$. A union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$ for $C > 1$, and $\tau \cdot p \leq Cn \ln n \cdot n^{1-C} = Cn^{2-C} \ln n = o(1)$ for $C > 2$. Thus both conditions of Lemma 3 hold for $C > 2$.

By Lemma 3, $\mathbf{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. Since $T_{\text{RLS}} = \sum_{k=1}^n n/k = \Theta(n \log n)$ (lower bound: at least $n/3$ zeros initially w.o.p. by Chernoff, giving $\sum_{k=1}^{n/3} n/k = \Omega(n \log n)$), the result follows. $\square$

In the following two theorems we show how $(1+1)$ Opt-IA_{LASTBEST} achieves a linear factor speed-up over $(1+1)$ Opt-IA_{FIRSTBEST} for LEADINGONES, because the latter increases its mutation rate on plateaus of equal fitness before finding each improvement.

Theorem 7. *The expected runtime of $(1+1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^2)$ with $\tau \geq Cn \ln n$ for a sufficiently large constant C .*

Proof. Since $(1+1)$ Opt-IA_{LASTBEST} accepts equal-or-better solutions and updates $best$ on any acceptance, every accepted move (whether improving or neutral) restores $H(x, best) = 0$, so $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), the improvement probability at every level is exactly $1/n$ (flip the leftmost 0-bit), so $(1 - 1/n)^\tau \leq n^{-C}$ and a union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$. Moreover $\tau \cdot p \leq Cn^{2-C} \ln n = o(1)$ for $C > 2$, verifying condition (ii).

By Lemma 3, $\mathbf{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. For RLS on LEADINGONES, the unique improving move at each of the n levels is the leftmost 0-bit flip, with probability $1/n$; neutral tail flips are accepted but do not advance the level. Hence the expected time per level is n and $T_{\text{RLS}} = n^2$, giving $\mathbf{E}[T'] = \Theta(n^2)$. $\square$

Theorem 8. *The expected runtime of the $(1+1)$ Opt-IA_{FIRSTBEST} with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^3)$ with $\tau = \Omega(n^{1+\epsilon})$ for any arbitrary constant $\epsilon > 0$.*

Proof. Fix a level ℓ with $n/3 \leq \ell < 2n/3$. For the current individual x with $\text{LO}(x) = \ell$, we have $x_1 = \dots = x_\ell = 1$ and $x_{\ell+1} = 0$, and the remaining $m := n - \ell - 1$ positions form the tail. In each iteration, the first flipped bit is uniformly distributed over $\{1, \dots, n\}$; therefore the probability that the

first flip hits the improving bit $\ell + 1$ is $1/n$, the probability that it hits the prefix is ℓ/n , and the probability that it hits the tail is m/n . Let T be the iteration on which the improving bit is first hit. Then T is geometrically distributed with parameter $1/n$, so $\mathbf{E}[T] = n$. Before this improving iteration occurs, the expected number of tail-first iterations is $\frac{m}{n-1}(\mathbf{E}[T] - 1) = \frac{m(n-1)}{n-1} = m = \Theta(n)$ (using that, conditional on not hitting the improving bit $\ell + 1$, the first flip is uniform over the remaining $n - 1$ positions, of which m are tail positions). Each such iteration flips a uniformly random tail bit. By standard coupon-collector arguments, after $\Theta(n)$ such tail flips a constant fraction of the m tail positions has been flipped exactly once with constant probability (since each hypermutation samples bits without replacement, each position is visited at most once per call). Thus, with constant probability, the Hamming distance between x and $best$ is $\Theta(m) = \Theta(n)$, and therefore the IPH mutation potential satisfies $M = \Theta(n)$ at this level.

Consider now a subsequent iteration at the same level while $M \geq \alpha n$ for some constant $\alpha > 0$. If the first flip hits the prefix (an event of probability at least $\ell/n \geq 1/3$), then, because mutation proceeds without replacement, the lost prefix bit cannot be repaired within the same iteration. Consequently, no acceptance can occur in that iteration, and the algorithm executes the entire mutation budget M , rejecting at the end. Hence the expected number of fitness evaluations in such an iteration is at least αn . Since this happens with probability at least $1/3$, the unconditional expected cost of one iteration while $M = \Theta(n)$ is $\Omega(n)$.

The probability that an iteration yields an improvement is exactly $1/n$, because only a first flip that hits position $\ell + 1$ increases the LEADINGONES value. Therefore the expected number of iterations required to advance from level ℓ to $\ell + 1$ is $1/(1/n) = n$. Combining this with the previous observation, the expected number of fitness evaluations spent at level ℓ satisfies $\Omega(n) \cdot n = \Omega(n^2)$. There are $\Theta(n)$ levels ℓ in the interval $[n/3, 2n/3]$, and each such level contributes $\Omega(n^2)$ expected evaluations. Thus the total expected optimisation time is $\Omega(n^3)$. Finally, stochastic ageing can only increase the number of evaluations and therefore cannot invalidate the lower bound. Since the function has n levels, $1/n$ probability of improvement and $M \leq n$, the expected runtime is bounded from above by $O(n^3)$ as well. $\square$

In this section, we have proven the results for M_{linHD} . Naturally the same positive results follow for the M_{expoHD} operator which always has at most the same mutation rate or lower.

V. WHEN IPH OUTPERFORMS TOO HIGH MUTATION RATES

In this section we show that M_{linHD} combined with ageing can efficiently escape from local optima. We will use the well known CLIFF_d function for this purpose where static hypermutations are inefficient [39]:

$$\text{CLIFF}_d(x) := \begin{cases} \sum_{i=1}^n x_i & \text{if } \sum_{i=1}^n x_i \leq n - d, \\ \sum_{i=1}^n x_i - d + 1/2 & \text{otherwise.} \end{cases} \quad (6)$$

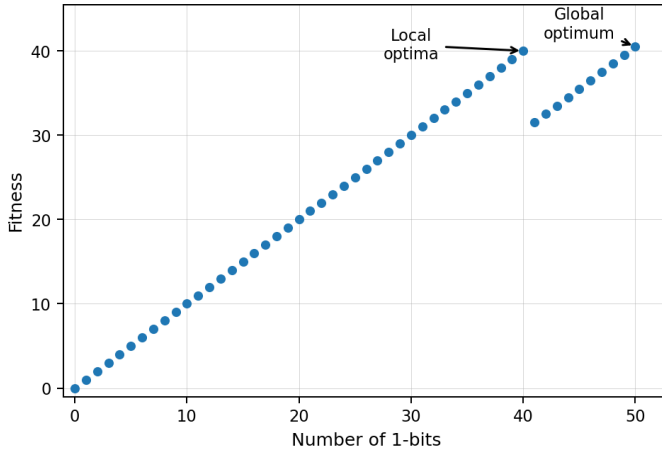


Fig. 3. The CLIFF_d function for $n = 50$, $d = 10$. The local optimum (40 one-bits, fitness 40) and global optimum (50 one-bits, fitness 40.5) are labelled.

The class of CLIFF_d functions, illustrated in Figure 3, is usually used to assess the performance of non-elitist algorithms. It was originally designed as a problem instance where non-elitist EAs outperform elitist EAs [44]. This function has a ONEMAX slope with length $n - d$ which leads the algorithms towards the local optima. The local optima are followed by a second ONEMAX slope (of length d) of lower fitness which leads to the global optimum. While elitist algorithms need to make a jump of size d to find the global optimum, some non-elitist algorithms can easily find the second slope by accepting inferior solutions, and then hillclimb up to the global optimum [3], [24], [39], [60], [61]. Due to the high mutation rates of static hypermutations, even if they were to escape the local optima of CLIFF_d (which is very unlikely due to FCM and linear mutation potentials), they would still jump back with high probability.

Recently *fast AISs* have been introduced [3] that simply flip the required number of bits independently according to the mutation potential (rather than sequentially and using FCM) in a similar fashion to power law mutation operators [4]. These have been shown to optimise CLIFF_d as long as the gap size is linear in expected time $O(n^{5/2} \log n)$.

Now we show that differently from static hypermutations, M_{linHD} combined with ageing can escape from the local optima of CLIFF_d also for small cliff gaps, and faster than the *fast AISs* for sublinear gaps, hence optimises the function efficiently. The condition that d is not prohibitively large is necessary to avoid that the reinitialised solutions due to ageing have larger fitness than a solution with $n - d + 1$ 1-bits, which would cause the latter to be removed from the population. We believe that this assumption is realistic for practical applications.

The following theorem is the main result of this section.

Theorem 9. *The $(1 + 1)$ $\text{Opt-IA}_{\text{FIRSTBEST}}$ with M_{linHD} (factor $c < \frac{1+4\epsilon}{3-4\epsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon})$ optimises*

CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected

$$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$$

fitness function evaluations.

We now present the proof sketch of our main result on the CLIFF_d function, together with all supporting lemmata. The argument proceeds in two major stages: (i) obtaining a canonical entry point onto the second slope immediately after an ageing event at the cliff-top; and (ii) showing that from this configuration the algorithm reaches the optimum without dangerous increases in mutation potential.

For stage (ii) we will use the generalization of Markov's Inequality by Ville [62](Ch. VII) which will allow us to show that the mutation potential stays bounded with constant probability.

Theorem 10 (Ville's inequality for nonnegative supermartingales). *Let $(M_t)_{t \geq 0}$ be a nonnegative supermartingale adapted to $(\mathcal{F}_t)_{t \geq 0}$. Then for every $a > 0$,*

$$\Pr\left(\sup_{t \geq 0} M_t \geq a\right) \leq \frac{E[M_0]}{a}.$$

To establish the result, we first analyse how the algorithm reaches a highly structured “canonical” state shortly after ageing triggers at the cliff-top. At the cliff-top the algorithm cannot improve except by sampling the global optimum, and so ageing will eventually be triggered. The key insight is that the iteration of ageing followed by two additional fitness improvements can place the algorithm into a *canonical starting configuration on the second slope*. Throughout the CLIFF_d analysis we index the successful fitness improvements on the second slope by $r = r_0, r_0 + 1, \dots$, and write $\text{HD}(r)$ for the Hamming distance between the current solution and *best* after r such improvements. With this notation, the canonical start corresponds to $r_0 = 2$ (two improvements completed from the cliff bottom) and $\text{HD}(r_0) = 2$ (the current solution differs from the locked *best* in exactly two positions). The following lemma formalises this.

Lemma 4. *Suppose ageing is triggered at the cliff-top (d zero-bits). The $(1 + 1)$ $\text{Opt-IA}_{\text{FIRSTBEST}}$ with M_{linHD} with $c \in (0, 1)$ reaches within at most three steps the configuration*

$$r_0 = 2, \quad \text{HD}(r_0) = 2,$$

with probability at least

$$\frac{3}{256} \left(\frac{d}{n}\right)^3.$$

Proof. We exhibit a consecutive three-step event which happens with probability $\Omega((d/n)^3)$ and yields the stated starting position on the second slope.

In the ageing iteration we require that the first two flips are $1 \rightarrow 0$ then $0 \rightarrow 1$ without replacement. This has probability $\frac{n-d}{n} \cdot \frac{d}{n-1}$ and produces an *equal-fitness* offspring accepted by FCM (first return to $\geq$ parent). The offspring inherits the parent's age (no improvement), so both parent and offspring face the ageing coin and die independently with probability

1/2. Requiring “parent dies, offspring survives” contributes a factor 1/4, and by the update rule the reference locks to the last cliff-top (the parent that died). The surviving equal-fitness offspring differs from this locked reference in exactly two bits, hence $HD = 2$ and, since $c < 1$, the next iteration has $M = 1$.

With $M = 1$, exactly one flip is performed and FCM does not accept worse-than-parent, so the algorithm returns the single-flip string after the potential exhaustion. A $0 \rightarrow 1$ flip occurs with probability at least d/n and returns the *cliff-bottom* ($n - d + 1$ ones), at Hamming distance 1 from the locked reference. Ageing does not reset (no improvement), so both the current parent (equal-fitness cliff-top) and the returned offspring face the coin; requiring “parent dies, offspring survives” contributes another factor 1/4 and preserves the locked reference at the last cliff-top.

In the following iteration with probability at least $(d-1)/n$ the first bit flip is a $0 \rightarrow 1$ flip, and the offspring is *accepted* by FCM (strict improvement from the cliff-bottom), producing a point with $n - d + 2$ ones. Relative to the locked cliff-top reference, the Hamming distance is now 2, and exactly two successful phases have completed: $r_0 = 2$, $HD(r_0) = 2$.

Multiplying the lower bounds yields

$$\left(\frac{n-d}{n} \cdot \frac{d}{n-1} \cdot \frac{1}{4}\right) \cdot \left(\frac{d}{n} \cdot \frac{1}{4}\right) \cdot \left(\frac{d-1}{n}\right).$$

For $d \leq (\frac{1}{4} - \varepsilon)n$ we have $(n-d)/n \geq 3/4$, and for $d \geq 2$ we have $\frac{d}{n-1} \geq \frac{d}{n}$ and $\frac{d-1}{n} \geq \frac{1}{2} \cdot \frac{d}{n}$. Hence the product is at least

$$\frac{3}{256} \left(\frac{d}{n}\right)^3,$$

which proves the claim. $\square$

The canonical condition $(r_0, HD(r_0)) = (2, 2)$ means that we begin the drift analysis two successful improvements into the second slope, at a point differing from the reference in exactly two bit positions.

Repeated ascents to the cliff-top therefore form a geometric waiting-time process. *It takes at most $\sum_{i=d}^n n/i = O(n \log(n/d))$ for RLS to climb the first slope and the mutation potential is at most linear.* Since each ascent takes $O(n^2 \log(n/d))$, the total time to obtain the canonical start is provided by the following corollary.

Corollary 1. *The expected evaluation count until the first occurrence of $(r_0, HD(r_0)) = (2, 2)$ is*

$$O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

From the canonical starting point, the key difficulty is showing that the mutation potential does not increase too much. With only few zero-bits remaining, successful steps on the second slope require flipping one of these rare zeros, whereas each $1 \rightarrow 0$ flip increases the Hamming distance and thus the mutation potential. If this potential grows too large, hypermutations become long enough that, under FCM($\geq$), they are increasingly likely to wander back to the cliff-top; once there, the stopping rule causes the process to lock onto the plateau and erase all progress. Hence the analysis must show that with constant probability the mutation potential

stays small throughout the entire second-slope climb. This requires controlling tightly how many detrimental flips a hypermutation can accumulate before recovering to parent-level fitness. The next lemma provides the crucial starting point: an exponential tail bound for this damaging prefix.

Lemma 5. *Let the current solution on the second slope have $z \leq d$ zero-bits and define $p = z/n$. Consider a single application of the hypermutation operator; let ξ_z be the number of $1 \rightarrow 0$ flips made within that one hypermutation before the fitness first returns to $\geq$ the parent’s fitness (i.e., before the first catch-up). Then for all $k \geq 1$,*

$$\Pr(\xi_z \geq k) \leq (2p)^k.$$

Consequently, the probability that the Hamming distance increases by at least $2k$ in a single hypermutation is also at most $(2p)^k$.

Proof. If the first flip is a $0 \rightarrow 1$ move, the offspring is strictly better and the hypermutation stops immediately, yielding $\xi_z = 0$. Hence the event $\{\xi_z \geq 1\}$ implies that the first flip was a $1 \rightarrow 0$ move. From that point on the process carries a *deficit* of one-bits $D_t := \#(1 \rightarrow 0) - \#(0 \rightarrow 1)$ that starts at $D_1 = 1$ and decreases by 1 on each $0 \rightarrow 1$ flip and increases by 1 on each $1 \rightarrow 0$ flip. The hypermutation stops at the *first* time t with $D_t = 0$ (a catch-up). Thus the event $\{\xi_z \geq k\}$ is exactly the event that the first return to 0 happens no earlier than time $2k$.

We bound this probability from above by a simple “coarse blocking” argument that uses only that z is small and that flips occur without replacement.

Condition on any history of length $t \leq 2k - 1$ that keeps $D_t \geq 1$ (so no catch-up yet) and in which at most t distinct positions have been flipped. Among the $n - t$ remaining positions there are at most z zeros. Hence the conditional probability that the *next* flip is a $0 \rightarrow 1$ move is at most

$$\frac{z}{n-t} \leq \frac{z}{n-(2k-1)} \leq \frac{z}{(1/2+2\varepsilon)n} \leq 2p,$$

because $2k \leq 2d < (\frac{1}{2} - 2\varepsilon)n$. In other words, throughout any hypermutation that lasts up to $2k$ flips, a zero-flip is never more likely than $2p$.

If the first catch-up occurs at some time $\geq 2k$, then along the way the trajectory must realize *at least* k distinct $0 \rightarrow 1$ flips while $D > 0$ (each such flip reduces the deficit by 1 but, by assumption, does not yet hit 0 before the k -th such zero-flip). Expose these k zero-flips in order and ignore everything else (the interspersed $1 \rightarrow 0$ flips only make catch-up *harder* and so can be dropped for an upper bound). By Step 1, each of these k zero-flips occurs with conditional probability at most $2p$, uniformly over the entire run while $D > 0$. Therefore, by the chain rule,

$$\Pr(\text{first catch-up occurs at time } \geq 2k) \leq (2p)^k.$$

Since the left-hand side is exactly $\Pr(\xi_z \geq k)$, the claim follows. $\square$

A whole *phase* consists of all attempts at the same zero-bit count until the first success. Using the tail bound above, we

obtain uniform exponential moment bounds for entire phases. negative.

Lemma 6. Consider a phase at zero-bit count z and let Y_z denote the total Hamming-distance increase from all failed attempts in the phase. Let $\rho = 2d/n$. For all $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$,

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1-\rho}{1-\rho e^{2\lambda}} =: K(\lambda).$$

Proof. For a single failed attempt starting the hypermutation with z zeroes, let ξ count the number of $1 \rightarrow 0$ flips while below the parent's fitness before the first catch-up. By Lemma 5 and the change in Hamming distance $\Delta_{\text{fail}} \leq 2\xi$ with $\xi \geq 1$,

$$\begin{aligned} \mathbb{E}[e^{\lambda \Delta_{\text{fail}}}] &\leq \sum_{k \geq 1}^{\infty} e^{2\lambda k} \Pr(\xi = k) \\ &\leq \sum_{k \geq 1}^{\infty} (2p e^{2\lambda})^k \\ &= \frac{2p e^{2\lambda}}{1 - 2p e^{2\lambda}} =: t. \end{aligned}$$

Let F be the number of failed attempts before the first success in the phase. Since Y_z is the sum over failed attempts only (the success contributes 0), conditioning stepwise on the phase history gives

$$\mathbb{E}[e^{\lambda Y_z}] = \mathbb{E}\left[\prod_{i=1}^F e^{\lambda \Delta_{\text{fail}}^{(i)}}\right] \leq \mathbb{E}[t^F].$$

Using the geometric MGF for $t \in [0, 1)$,

$$\mathbb{E}[t^F] = \sum_{f \geq 0} (1-p)^f p t^f = \frac{p}{1 - (1-p)t}.$$

Substituting $t = \frac{2p e^{2\lambda}}{1 - 2p e^{2\lambda}}$ and simplifying,

$$\begin{aligned} \mathbb{E}[e^{\lambda Y_z}] &\leq \frac{p}{1 - (1-p) \frac{2p e^{2\lambda}}{1 - 2p e^{2\lambda}}} \\ &= \frac{p(1 - 2p e^{2\lambda})}{1 - 2p e^{2\lambda}(2 - p)} =: K_z(\lambda). \end{aligned}$$

Since $p \leq \rho/2$ and the map $p \mapsto K_z(\lambda)$ is increasing on $(0, \frac{1}{2})$ for fixed $\lambda \in (0, \frac{1}{2} \ln(1/\rho))$, we obtain the uniform bound

$$\mathbb{E}[e^{\lambda Y_z}] \leq K_z(\lambda) \Big|_{p=\rho/2} = \frac{\frac{\rho}{2}(1 - \rho e^{2\lambda})}{1 - \rho e^{2\lambda}(2 - \frac{\rho}{2})}$$

Now, our claimed upper bound reduces to showing that a concave quadratic function of $u := \rho e^{2\lambda} \in [0, \frac{1}{2}]$ is non-

$$\begin{aligned} \frac{\frac{\rho}{2}(1 - \rho e^{2\lambda})}{1 - \rho e^{2\lambda}(2 - \frac{\rho}{2})} &\leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda), \\ \iff \frac{\rho}{2}(1 - u)^2 &\leq (1 - \rho)\left(1 - (2 - \frac{\rho}{2})u\right), \quad u := \rho e^{2\lambda}, \\ D(u) &:= (1 - \rho)\left(1 - (2 - \frac{\rho}{2})u\right) - \frac{\rho}{2}(1 - u)^2 \\ &= \left(1 - \frac{3}{2}\rho\right) + \left(-2 + \frac{7}{2}\rho - \frac{1}{2}\rho^2\right)u - \frac{\rho}{2}u^2, \\ D(0) &= 1 - \frac{3}{2}\rho \geq \frac{1}{4}, \quad D\left(\frac{1}{2}\right) = \frac{\rho}{8}(1 - 2\rho) \geq 0 \implies \\ D(u) &\geq 0 \text{ for } u \in [0, \frac{1}{2}], \end{aligned}$$

□

To track the safety of the mutation potential across phases, we introduce a linear *buffer* $B_r = \theta r - s_0 - E_r$ capturing the invariant $c \text{HD}(r) \leq r$.

Lemma 7. Let successful phases be indexed by $r = r_0, r_0 + 1, \dots$ and define

$$s_0 = \text{HD}(r_0) - r_0, \quad \theta = \frac{1}{c} - 1, \quad E_r = \sum_{j=r_0+1}^r Y_{z_j},$$

and $B_r = \theta r - s_0 - E_r$. Then for all $r \geq r_0$:

- 1) $B_r \geq 0$ if and only if $c \cdot \text{HD}(r) \leq r$;
- 2) $B_{r+1} - B_r = \theta - Y_{z_{r+1}}$;
- 3) $B_{r_0} = \theta r_0 - s_0$.

Proof. For any $r \geq r_0$, the Hamming distance admits the exact decomposition

$$\begin{aligned} \text{HD}(r) &= \underbrace{\text{HD}(r_0)}_{= r_0 + s_0} + \underbrace{(r - r_0)}_{\text{one per success}} + \underbrace{E_r}_{\text{extra HD in failed attempts}} \\ &= r + s_0 + E_r, \end{aligned}$$

since each strict improvement increases the HD by exactly 1 and $E_r = \sum_{j=r_0+1}^r Y_{z_j}$ collects the extra HD from failed attempts (the successful attempt contributes 0 because $\Delta_z^{\text{succ}} = 1$).

(i) *Safety invariant.* Using $\theta = \frac{1}{c} - 1$ and $B_r := \theta r - s_0 - E_r$,

$$\begin{aligned} B_r \geq 0 &\iff \theta r \geq s_0 + E_r \iff c\theta r \geq c(s_0 + E_r) \\ &\iff (1 - c)r \geq c(s_0 + E_r). \end{aligned}$$

But $\text{HD}(r) = r + s_0 + E_r$, so the last inequality is equivalent to

$$r \geq c(r + s_0 + E_r) = c \text{HD}(r),$$

i.e., $c \text{HD}(r) \leq r$. Thus $c \text{HD}(r) \leq r \iff B_r \geq 0$.

(ii) *One-phase update.* By definition,

$$\begin{aligned} B_{r+1} - B_r &= (\theta(r+1) - s_0 - E_{r+1}) - (\theta r - s_0 - E_r) \\ &= \theta - (E_{r+1} - E_r) = \theta - Y_{z_{r+1}}. \end{aligned}$$

(iii) Trivial from the definition at $r = r_0$. □

Combining the MGF of the buffer growth with the buffer update yields an exponential drift envelope suitable for applying Ville's inequality for non-negative supermartingales (Theorem 10).

Lemma 8. Let $\rho = 2d/n$. For any $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ define

$$\frac{1-\rho}{1-\rho e^{2\lambda}} =: K(\lambda), \quad \alpha = e^{-\lambda\theta} K(\lambda).$$

Then for all $t \geq 0$,

$$\mathbb{E}[e^{-\lambda(B_{t+1}-B_t)} \mid \mathcal{F}_t] \leq \alpha.$$

Proof. By Lemma 7, $B_{t+1} - B_t = \theta - Y_{z_{t+1}}$, hence

$$e^{-\lambda(B_{t+1}-B_t)} = e^{-\lambda\theta} e^{\lambda Y_{z_{t+1}}}.$$

By Lemma 6, for $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ we have $\mathbb{E}[e^{\lambda Y_{z_{t+1}}} \mid \mathcal{F}_t] \leq K(\lambda)$ for all phases (all $z_{t+1} \leq d$). Therefore

$$\begin{aligned} \mathbb{E}[e^{-\lambda(B_{t+1}-B_t)} \mid \mathcal{F}_t] &= e^{-\lambda\theta} \mathbb{E}[e^{\lambda Y_{z_{t+1}}} \mid \mathcal{F}_t] \\ &\leq e^{-\lambda\theta} K(\lambda) = \alpha \end{aligned}$$

□

We now apply Ville's inequality to turn such a drift envelope into a bound on barrier crossing.

Corollary 2. If $B_0 = b > 0$ and there exist $\lambda > 0$ and $\alpha < 1$ with

$$\mathbb{E}[e^{-\lambda(B_{t+1}-B_t)} \mid \mathcal{F}_t] \leq \alpha,$$

then

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda b}.$$

Proof. Ville's inequality (Theorem 10) applies to nonnegative supermartingales, but the buffer B_t is not one (it can drift and take arbitrary values). We therefore work instead with the transformed process $M_t := \alpha^{-t} e^{-\lambda B_t}$, which is a nonnegative supermartingale: the assumption $\mathbb{E}[e^{-\lambda(B_{t+1}-B_t)} \mid \mathcal{F}_t] \leq \alpha$ gives $\mathbb{E}[M_{t+1} \mid \mathcal{F}_t] \leq M_t$. Applying Theorem 10 to M_t with $a = 1$ yields $\Pr(\sup_{t \geq 0} M_t \geq 1) \leq \mathbb{E}[M_0] = e^{-\lambda b}$. Finally, $B_t \leq 0$ implies $e^{-\lambda B_t} \geq 1$, hence $M_t \geq 1$, giving the claim. □

To guarantee the drift negativity required above, we identify the admissible range of the mutation-potential factor c .

Lemma 9. Let $\rho = 2d/n$ and $\theta = \frac{1}{c} - 1$. Define

$$c_*(n, d) = \frac{1-\rho}{1+\rho}.$$

If $c < c_*(n, d)$, then there exists $\lambda > 0$ such that

$$e^{-\lambda\theta} K(\lambda) < 1,$$

and hence the buffer satisfies the Ville barrier bound.

Proof. Let $g(\lambda) := \ln \alpha(\lambda) = -\lambda\theta + \ln\left(\frac{1-\rho}{1-\rho e^{2\lambda}}\right)$. Then $g(0) = 0$ and $g'(0) = -\theta + \frac{2\rho}{1-\rho}$. If $\theta > \frac{2\rho}{1-\rho}$ (equivalently $c < \frac{1-\rho}{1+\rho}$), continuity gives $g(\lambda) < 0$ for all sufficiently small $\lambda > 0$, i.e., $\alpha(\lambda) < 1$ in the admissible domain. □

A uniform instantiation over all permitted d shows that the feasible c range has a nontrivial lower bound.

Corollary 3. If $d \leq (\frac{1}{4} - \varepsilon)n$, then

$$c_*(n, d) \geq \frac{1+4\varepsilon}{3-4\varepsilon} \geq \frac{1}{3}.$$

Finally, combining the threshold on c with the initial buffer value at $(r_0, \text{HD}(r_0)) = (2, 2)$ yields a uniform constant bound on the probability of avoiding jumping back.

Lemma 10. If $c < c_*(n, d)$ and $b = B_{r_0} = \frac{r_0}{c} - \text{HD}(r_0) > 0$, then

$$\Pr(\inf B_t \leq 0) \leq e^{-\lambda b}$$

for some $\lambda > 0$ depending only on c and d/n . For fixed ε , this yields a uniform constant < 1 independent of n, d .

Proof. By Lemma 9 (with $\rho = 2d/n$), there exists $\lambda \in (0, \frac{1}{2} \ln \frac{1}{2\rho}) = (0, \frac{1}{2} \ln \frac{n}{4d})$ such that $e^{-\lambda\theta} K(\lambda) < 1$. Applying Corollary 2 to B_t started at $B_{r_0} = b > 0$ yields $\Pr(\inf_{t \geq 0} B_t \leq 0) \leq e^{-\lambda b}$. □

Once the mutation potential remains safe throughout, progress on the second slope is monotone and its runtime is easily bounded.

Lemma 11. If $B_t > 0$ for all t , then the second-slope ascent completes in expected

$$\Theta(n \log d)$$

iterations. If $\tau = \Omega(n^{1+\varepsilon})$, then $\Theta(n \log d) = o(\tau)$.

Proof. When the zero-bit count is z , the success probability per attempt is at least z/n , so the expected number of attempts for $z \rightarrow z-1$ is at most n/z . Summing over $z = 1, \dots, d$ gives $\sum_{z=1}^d n/z = \Theta(n \log d)$. Since $d \leq (\frac{1}{4} - \varepsilon)n$, we have $\log d = O(\log n)$, hence $n \log d = o(n^{1+\varepsilon})$ and $\mathbb{E}[T_{\text{climb}}] = o(\tau)$. □

The expected time to reach the canonical start is $O(n^5 \log(n/d)/d^3)$. Once there, with constant probability the buffer remains positive for all phases; conditioned on this event, the second-slope ascent costs $O(n^2 \log d)$ evaluations. Since the success probability is constant, this remains the unconditional expectation. Summing the two contributions establishes the main runtime bound.

Proof [of Theorem 9.]

From a uniformly random start, the expected iterations to reach the cliff-top ($n - d$ ones) are $\sum_{i=d}^n \frac{n}{i} = \Theta(n \log \frac{n}{d})$, which at $\Theta(n)$ evaluations/iteration costs $\Theta(n^2 \log \frac{n}{d})$ evaluations per ascent to the cliff-top. At the cliff-top, the only fitter point is 1^n , so ageing triggers unless the optimum is found within τ iterations.

By Lemma 4, within at most three iterations after an ageing kick the process reaches the canonical second-slope start

$$r_0 = 2, \quad \text{HD}(r_0) = 2$$

with probability

$$p_{\text{can}}(n, d) \geq \frac{3}{256} \left(\frac{d}{n}\right)^3.$$

Hence the number of ageing epochs needed to obtain this start is geometric with mean $1/p_{\text{can}}$, and the expected evaluation cost up to the canonical start (including restarts) is

$$\Theta\left(\frac{1}{p_{\text{can}}(n, d)} \cdot n^2 \log \frac{n}{d}\right) = O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

Under the reference-update rule the start on the second slope is $r_0 = 2$, $\text{HD}(r_0) = 2$, so $s_0 = 0$ and with $\theta = \frac{1}{c} - 1$ define

$$B_r := \theta r - \sum_{j=r_0+1}^r Y_{z_j} \quad (r \geq r_0),$$

so $B_{r_0} = 2\theta$ and $B_{t+1} - B_t = \theta - Y_{z_{t+1}}$ by Lemma 7. By the refined phase MGF (Lemma 6), for every $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ with $\rho := 2d/n$,

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda).$$

Therefore, by Lemma 8,

$$\mathbb{E}[e^{-\lambda(B_{t+1}-B_t)} \mid \mathcal{F}_t] \leq e^{-\lambda\theta} K(\lambda) \quad \text{for all } t \geq 0.$$

Since $d \leq (\frac{1}{4} - \varepsilon)n$ and the theorem assumes

$$c < \frac{1 + 4\varepsilon}{3 - 4\varepsilon} \leq \frac{1 - 2d/n}{1 + 2d/n} =: c_*(n, d),$$

Lemma 9 guarantees an admissible $\lambda \in (0, \frac{1}{2} \ln \frac{n}{4d})$ with $e^{-\lambda\theta} K(\lambda) < 1$. With $B_{r_0} = 2\theta$, Corollary 2 yields

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda 2\theta}.$$

Moreover, since $\frac{1}{2} \ln \frac{n}{4d} \geq \frac{1}{2} \ln \frac{1}{1-4\varepsilon}$ for all admissible d , we may fix a uniform $\lambda_\varepsilon \in (0, \frac{1}{2} \ln \frac{1}{1-4\varepsilon})$ and conclude

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda_\varepsilon 2\theta}.$$

Because λ_ε depends only on ε and $\theta = \frac{1}{c} - 1$ depends only on c , this right-hand side is a constant in $(0, 1)$, independent of n and d .

On the event $\{\inf_{t \geq 0} B_t > 0\}$ (no jump-back), the second-slope ascent is monotone and completes in expected

$$\sum_{z=1}^d \frac{n}{z} = \Theta(n \log d)$$

iterations, which is $o(\tau)$ since $\tau = \Omega(n^{1+\epsilon'})$. Since the mutation potential satisfies $M \leq cn$ (where c is the constant from the M_{linHD} potential), each iteration evaluates at most $cn = \Theta(n)$ individuals; one second-slope attempt therefore costs $\Theta(n^2 \log d)$ evaluations, and the constant no-jump-back probability implies a constant expected number of such attempts.

Summing the (restarted) approach to the canonical start and the second-slope cost gives

$$\mathbb{E}[\text{evaluations}] = O\left(\frac{n^5 \log(n/d)}{d^3}\right) + O(n^2 \log d),$$

as claimed. $\square$

The analysis for $(1+1)$ Opt-IA_{LASTBEST} differs only in Lemma 4 and actually yields a d/n factor better probability bound since the mutation potential is 1 at the cliff-top and thus, creating a solution at the bottom of the cliff is immediately possible when any cliff top individual dies, whereas for $(1+1)$ Opt-IA_{FIRSTBEST} to work we have to first accept a neutral mutation to get the mutation potential

to 1, and then mutate into a cliff bottom solution in the next iteration. Consequently the runtime for $(1+1)$ Opt-IA_{LASTBEST} is $O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$.

Corollary 4. *The $(1+1)$ Opt-IA_{LASTBEST} with M_{linHD} (factor $c < \frac{1+4\varepsilon}{3-4\varepsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon'})$ optimises CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected*

$$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$$

fitness function evaluations.

The following section will highlight the advantages of using FIRST-BEST over using LAST-BEST.

VI. WHEN IPH OUTPERFORMS TOO LOW MUTATION RATES

In this section we introduce a function class where inversely proportional hypermutations outperform the lower mutation rates of SBM, and the higher mutation rate on local optima of the $(1+1)$ Opt-IA_{FIRSTBEST} will be essential. The function class is inspired by the one previously used to show the advantages and disadvantages of using high mutation rates in the context of rank-based mutation in population-based algorithms [48]. In rank-based mutation algorithms [63] each individual has a different mutation rate based on their rank in the population i.e., the best individual has the lowest mutation rate while the worst individual has a very high one. The LEADINGTRAPJUMP function was introduced as an example where the use of high mutation rates is detrimental while low mutation rates are efficient. However, if the locations of the local and global optima are swapped, then the opposite result can be shown (i.e. high mutation rates are effective while low ones are ineffective). We have slightly modified the location of the trap compared to [48] for our objectives.

Formally, for constants $k \geq 1$ and $\rho \in (0.8, 0.9)$, let $r := 2k + 1$, $L := \lfloor \log_2 n \rfloor$, $\mathcal{P} := \{x : \text{LO}(x) \geq 0.9n\}$, $\mathcal{B}_k := \{0^r 1^{(0.9n-r)*}\}$, and $\mathcal{D}_\rho := \{0^L * : |x|_1 \geq \rho n\}$. The function $\text{LTJ}_{k,\rho} : \{0, 1\}^n \rightarrow \mathbb{R}$ is

$$\text{LTJ}_{k,\rho}(x) = \begin{cases} 0, & x \in \mathcal{P}, \\ \text{LO}(x_{i>r}) + r, & x \in \mathcal{B}_k, \\ n + 1, & x \in \mathcal{D}_\rho, \\ \text{LO}(x), & \text{otherwise.} \end{cases}$$

Here $x_{i>r}$ denotes the suffix of x at positions $i > r$, i.e., $x_{i>r} := (x_{r+1}, x_{r+2}, \dots, x_n)$. Note that $r := 2k + 1$ is an odd number, as required for the block-jump prefix $0^r 1^{(0.9n-r)*}$ to force an even number of bit-flips to reach $\mathcal{B}_k$, which is the key parameter controlling the per-iteration success probability $\Theta(n^{-(r+1)}) = \Theta(n^{-(2k+2)})$. We introduce r as shorthand to avoid writing $2k + 1$ in every exponent throughout the proof.

The landscape consists of a long LeadingOnes slope rising from fitness $\text{LO}(x)$ up to level $0.9n - 1$, followed by a large penalised region. Two off-slope structures contain the two sets of optima but are inaccessible to standard hill-climbers. The set $\mathcal{B}_k$ begins with the prefix $0^r 1^{(0.9n-r)*}$ and forms a short

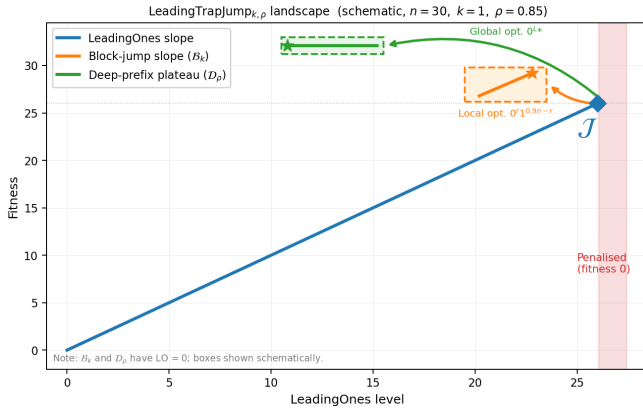


Fig. 4. The $\text{LEADINGTRAPJUMP}_{k,\rho}$ landscape (schematic, $n = 30, k = 1, \rho = 0.85$). The x-axis is the LEADINGONES level of the current solution. From the junction J (end of the main slope), large mutations can reach either the block-jump slope $\mathcal{B}_k$ (leading to the local optimum) or the deep-prefix plateau $\mathcal{D}_\rho$ (global optimum). Both off-slope structures have $\text{LO}(x) = 0$; their positions on the y-axis reflect their fitness values, not their LEADINGONES level.

slope, called block-jump slope, that deterministically increases toward its unique local optimum $0^r 1^{(0.9n-r)}$; reaching this prefix is therefore equivalent to entering the leading ones path to the local optimum $0^r 1^{(0.9n-r)}$. In contrast, the set $\mathcal{D}_\rho$ consists of solutions with prefix 0^{L*} whose suffix has ones-density at least ρ i.e., ρn ones, and forms a wide plateau of globally optimal fitness, which we call deep-prefix plateau. Both structures are hidden behind the penalised barrier (illustrated in Figure 4), and the only reliable way to reach them is via large hypermutation events. The junction $J := \{x : \text{LO}(x) = 0.9n - 1\}$ acts as a central gateway: each ageing epoch that restarts the search will, with high probability, ascend the LeadingOnes slope back to J before any off-slope structure is encountered. From J , the $(1+1)$ Opt-IA using IPH with M_{linHD} can perform jumps of the appropriate magnitude to reach either the entrance of the block-jump slope $\mathcal{B}_k$ or the plateau $\mathcal{D}_\rho$ i.e., via ageing restarts it will eventually find both optima efficiently. On the other hand, algorithms with lower mutation rates, such as those using SBM will with high probability not be able to flip the logarithmic number of bits required to reach the global optimum, thus have a runtime of $n^{\Omega(\log n)}$ with high probability.

Theorem 11. *The $(1+1)$ Opt-IA_{FIRSTBEST} with ageing threshold $\tau = \Omega(n^{1+\epsilon})$ using M_{linHD} with $c < 5/8$ identifies both local and global optima of $\text{LTJ}_{k,\rho}$ in $O(n\tau + n^{2k+3} + n^{\beta+1})$ fitness function evaluations where $\beta := \log_2 \frac{20}{c}$.*

To establish Theorem 11 we rely on three ingredients. First, every ageing epoch returns to the junction J with high probability. Second, an epoch beginning at J performs sufficiently long jumps to reach the beginning of the block-jump slope with probability $\Theta(n^{-(2k+2)})$ per iteration. Third, an epoch beginning at J performs a logarithmic jump onto the deep-prefix plateau with probability $\Theta(n^{-\beta})$ per iteration.

Since each event corresponds to reaching either the start of a short slope leading deterministically to its optimum (for $\mathcal{B}_k$) or directly a plateau of global optima (for $\mathcal{D}_\rho$), these per-epoch

probabilities govern the entire runtime.

We first need to bound from below the probability that the algorithm visits the junction point after restarts in order to later argue that from the junction point both sets of optima will be accessed.

Lemma 12. *Starting from any random individual, the $(1+1)$ Opt-IA_{FIRSTBEST} using IPH with M_{linHD} and $c < 5/8$ reaches the junction $J = \{x : \text{LO}(x) = 0.9n - 1\}$ before accepting any individual in $\mathcal{B}_k$ or $\mathcal{D}_\rho$ with high probability.*

Within n^2 iterations, the algorithm performs at least $0.9n$ LeadingOnes improvements with high probability. Before reaching J , deep-prefix entrances require flipping the L -bit prefix to 0^{L*} , an event whose probability per iteration is at most $(M(x)/n)^L$. Since under M_{linHD} the mutation potential satisfies $M(x) \leq q^* n$ for some constant $q^* < 1$ (determined solely by c and ρ), this becomes $(q^*)^L = n^{-\alpha^*}$ for $\alpha^* := \log_2(1/q^*) > 2$, yielding a total probability of $o(1)$ over n^2 iterations. Thus the slope is climbed to J with high probability.

Proof. Let $T := n^2$. We will show that in T iterations with high probability we sample a solution from J before we sample one from $\mathcal{B}_k$ or $\mathcal{D}_\rho$.

Before J , $\mathcal{B}_k$ or $\mathcal{D}_\rho$ is sampled from, we flip the leftmost zero-bit with probability $1/n$ and accept immediately by FCM($\geq$). If $X \sim \text{Bin}(T, 1/n)$ counts such improvements then $\mathbb{E}[X] = n$ and a Chernoff bound gives $\Pr(X < (9/10)n) \leq e^{-an}$ for some constant $a > 0$. Hence J is reached within T iterations with probability $1 - e^{-an}$ if we condition on that neither $\mathcal{B}_k$ nor $\mathcal{D}_\rho$ is sampled first.

Next, we will show that the probability of sampling a $\mathcal{B}_k$ or a $\mathcal{D}_\rho$ solution in T iterations is $o(1)$, starting our argument with $\mathcal{D}_\rho$.

Let $\alpha := |x|_1/n$ and $\alpha^\top := |\text{best}|_1/n$. Since the probability of improvement is at least $1/n$ in the initial leading-ones slope the ageing does not trigger with overwhelming probability unless the current solution is in J , $\mathcal{D}_\rho$ or $\mathcal{B}_k$, all of which lead to $\alpha^\top \geq \rho$ after the restart. We make a case distinction according to $\alpha \geq \rho - \xi$ for some constant $\xi \in (0, \rho - \frac{1}{2}]$ such that $q^* := 2c(1 - \rho + \xi) < 1/4$. This is achievable: choosing ξ sufficiently close to 0 gives $q^* \approx 2c(1 - \rho)$; since $\rho > 4/5$ we have $1 - \rho < 1/5$, so $2c(1 - \rho) < 2 \cdot \frac{5}{8} \cdot \frac{1}{5} = \frac{1}{4}$ (using $c < 5/8$). Any fixed $\xi \in (0, \rho - 1/2]$ small enough to keep $2c(1 - \rho + \xi) < 1/4$ then satisfies all requirements.

Case 1: $\alpha \geq \rho - \xi$. Using $|x \wedge \text{best}|_1 \geq |x|_1 + |\text{best}|_1 - n$,

$$\text{HD}(x, \text{best}) \leq (2 - \alpha - \alpha^\top)n \leq 2(1 - \rho + \xi)n,$$

$$M(x) \leq q^* n < n/4.$$

$\mathcal{D}_\rho$ definition requires all L leading bits among the M flips; hence

$$\Pr(\mathcal{D}_\rho \text{ in this iteration}) \leq \left(\frac{M}{n}\right)^L \leq (q^*)^L = n^{-\log_2(1/q^*)}.$$

Because $q^* < 1/4$, we have $\log_2(1/q^*) > 2$, so $T \cdot n^{-\log_2(1/q^*)} = o(1)$.

Case 2: $\alpha \leq \rho - \xi$. Since all $\mathcal{D}_\rho$ solutions have $|x|_1 \geq \rho n$, at least ξn net $0 \rightarrow 1$ flips ($Z - O$) are required within at most

cn bit-flips. With initial zero/one ratio giving negative drift, a Serfling (hypergeometric) tail bound yields

$$\Pr(Z - O \geq \xi n) \leq e^{-\gamma n}$$

for some constant $\gamma > 0$. Including the necessity to flip all L prefix bits only decreases the probability. Summing over at most T iterations remains $e^{-\gamma n}$. Therefore $\Pr(\mathcal{D}_\rho \text{ before } J) = o(1)$.

Similarly we will next establish that sampling from $\mathcal{B}_k$ in T iterations has probability $o(1)$. Let $r := 2k+1$, $S := \{r+1, \dots, \frac{9}{10}n\}$, and fix any constant $\delta \in (0, \frac{9}{10})$. Consider any iteration with parent $x \notin \mathcal{B}_k$ and write $\ell := \text{LO}(x)$ and $v := |\{i \in S : x_i = 0\}|$. We distinguish two cases depending on ℓ .

Case 1: $\ell < (\frac{9}{10} - \delta)n$. Then the unconstrained segment $S \cap \{\ell+1, \dots, \frac{9}{10}n\}$ has size at least δn . Conditioned on the history up to the current iteration, the bits in this segment are unbiased and independent, hence by Chernoff $v = \Omega(n)$ with probability $1 - e^{-\Theta(n)}$. On this event, to enter $\mathcal{B}_k = \{0^r 1^{(0.9n-r)*}\}$ and be accepted under FCM($\geq$), the offspring at acceptance must flip all r positions $\{1, \dots, r\}$ and must flip all $v = \Omega(n)$ zeros in S to 1, i.e., it must flip $\Omega(n)$ specified positions. Therefore, regardless of the mutation potential, the per-iteration probability of entering $\mathcal{B}_k$ is at most $\binom{n}{\Omega(n)}^{-1} = e^{-\Theta(n)}$. Union-bounding over at most $T \leq n^2$ iterations yields a total contribution $e^{-\Theta(n)}$. The exceptional event $v = o(n)$ itself has probability $e^{-\Theta(n)}$ per iteration and is negligible under the same union bound.

Case 2: $\ell \geq (\frac{9}{10} - \delta)n$. Then the set $F := \{r+1, \dots, \ell\}$ of leading-one positions that must not be flipped before acceptance has size $|F| = \ell - r = \Omega(n)$. Since $x \notin \mathcal{B}_k$, at least one bit in S is 0, so entering $\mathcal{B}_k$ and being accepted requires flipping the r positions $\{1, \dots, r\}$ and at least one further specified position in S before any position in F is flipped (otherwise LO decreases and FCM($\geq$) rejects). Let R denote any fixed choice of such $r+1$ required positions. A necessary condition for acceptance into $\mathcal{B}_k$ is that all positions in R appear before all positions in F in the random order in which the hypermutation flips bit positions. Since the relative order of the $|F| + |R|$ positions in $F \cup R$ is uniform, we obtain

$$\Pr(\mathcal{B}_k \text{ in this iteration}) \leq \binom{|F| + r + 1}{r + 1}^{-1} = O(n^{-(r+1)}).$$

Union-bounding over at most $T \leq n^2$ iterations yields a total contribution $O(n^{-(r-1)}) = O(n^{-2k}) = o(1)$. $\square$

The following two lemmata will condition throughout on the event that during an ageing epoch the junction J is reached. This event happens with high probability by Lemma 12.

Lemma 13. *Let an ageing epoch reach the junction J . Then, with probability $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$, the epoch performs a jump placing the individual at the beginning of the block-jump slope.*

The suffix of the individual at J is essentially random relative to the best individual stored from the previous epoch. With high probability their Hamming distance is at least $n/20$,

guaranteeing $M(x) = cn/20$ throughout the epoch. Under this event, the probability that the first $r+1$ flips of the hypermutation hit exactly the positions defining the prefix $0^r 1^{(0.9n-r)*}$ is $\Theta(n^{-(r+1)})$. Evaluating this over τ iterations yields the stated bound.

Proof. At the beginning of the epoch the length- $0.1n+1$ suffix of the new parent x is still uniformly random, so with probability at least $\frac{1}{2}$

$$\text{HD}(x, \text{best}) \geq \frac{n}{20}.$$

Conditioning on this event we lose only a factor 2. Then the mutation potential in every subsequent iteration of the epoch satisfies $M \geq \frac{cn}{20} \geq r+1$ for large enough n .

A solution in $\mathcal{B}_k$ is accepted as soon as the first $r+1$ distinct flips include exactly the set $\{1, \dots, r, 0.9n\}$ in any order; if this occurs, FCM stops immediately and no other leading-one bit can be destroyed. Since bits are sampled uniformly without replacement, the probability of this event is

$$p_{\text{iter}} = \prod_{i=0}^r \frac{r+1-i}{n-i} = \frac{(r+1)!}{(n)_{r+1}} = \Theta(n^{-(r+1)}) = \Theta(n^{-(2k+2)}),$$

independent of the exact value of M (we only need $M \geq r+1$ to allow that many flips).

Since an epoch lasts for τ independent iterations, a union bound yields

$$p_B \geq 1 - (1 - p_{\text{iter}})^\tau \geq \Omega(\min\{1, \tau n^{-(2k+2)}\}).$$

$\square$

Lemma 14. *Let an ageing epoch reach the junction J . Then, with probability $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, the epoch performs a jump placing the individual onto the deep-prefix plateau.*

At J , the ones-density of the suffix already exceeds ρ with high probability. Thus it suffices to flip the prefix to 0^L* . Under the same high-Hamming-distance event ensuring $M(x) \geq cn/20$, the probability that the first L flips hit positions $\{1, \dots, L\}$ is at least $(c/20)^L = n^{-\beta}$. Evaluating over τ iterations gives the lemma.

Proof of Lemma 14. At the beginning of the epoch the reference is the best string from the previous epoch (from J or $\mathcal{B}_k$), and the new parent x itself lies in J . Hence,

$$|x|_1 = 0.9n - 1 + |x_{\text{suffix}}|_1.$$

The suffix has length $0.1n+1$ and is uniform; its number of ones is $\text{Bin}(0.1n+1, \frac{1}{2})$ with expectation $0.05n + O(1)$. By a Chernoff bound $|x_{\text{suffix}}|_1 \geq 0.04n$ with probability $1 - e^{-\Omega(n)}$. Thus, already before any mutation,

$$|x|_1 \geq 0.9n - 1 + 0.04n > 0.94n > \rho n \quad (\rho < 0.9). \quad (1)$$

As in the block-jump argument we have $\Pr(\text{HD}(x, \text{best}) \geq n/20) \geq \frac{1}{2}$ (owing to the uniform suffix). We condition on this event, so we have $M \geq cn/20$ throughout the epoch.

To generate a $\mathcal{D}_\rho$ optimum it is necessary to flip the L leading ones from 1 to 0 before FCM stops. Sampling without replacement,

$$p_{\text{prefix}} = \frac{(M \cdot (M-1) \cdot \dots \cdot (M-L))}{(n \cdot (n-1) \cdot \dots \cdot (n-L))} \geq (M/n)^L \\ \geq (cn/20)^L = n^{-\beta}.$$

where $\beta := \log_2 \frac{20}{c}$.

When those L flips occur, since the L -bit prefix of the new parent is a uniformly random string (from the reinitialisation), it contains approximately $L/2$ one-bits in expectation; the total number of ones in the string therefore drops by at most $L/2 \leq \frac{1}{2} \log_2 n$ when these are flipped. Thus using (1) and $L = o(n)$ we get, $|x|_1 - L/2 \geq 0.94n - \frac{1}{2} \log_2 n \geq \rho n$ for all sufficiently large n . Hence the density requirement $|x|_1 \geq \rho n$ is already satisfied once the prefix flips have happened. Thus no additional tail flips are needed, and FCM immediately accepts a $\mathcal{D}_\rho$ search point.

Since each iteration satisfies $p_{\text{iter}} \geq n^{-\beta}$. Across the $\tau \geq n$ iterations of the epoch,

$$p_D \geq 1 - (1 - p_{\text{iter}})^\tau \geq 1 - (1 - n^{-\beta})^\tau = \Omega(\min\{1, \tau n^{-\beta}\}),$$

completing the proof. $\square$

Each *ageing epoch* begins with a uniformly random reinitialisation and runs for τ iterations; by Lemma 12 it ascends to the junction J with high probability before reaching any off-slope structure. We henceforth condition on epochs reaching J . Each ageing epoch evaluates $\Theta(n\tau)$ individuals. By Lemma 12, epochs reach J with high probability, and by lemmata 13 and 14, an epoch discovers each off-slope structure with probabilities p_B and p_D , respectively. Since B (entering $\mathcal{B}_k$) and D (entering $\mathcal{D}_\rho$) are mutually exclusive within a single epoch, the algorithm must discover them in *different* epochs. By a coupon-collector argument (we need to “collect” both coupons B and D , which arrive independently across epochs with probabilities p_B and p_D respectively), the expected number of epochs until both optima have been seen is $O(1/p_B + 1/p_D)$. Multiplying by the cost per epoch yields the bound stated in Theorem 11.

Proof [of Theorem 11.]

By Lemma 12 we reach the junction J with probability $1 - o(1)$. Starting an epoch in the junction, the two lemmata give $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$ and $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, and B and D are mutually exclusive within one epoch. Thus the expected number of epochs to sample from both sets of optima is:

$$O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right).$$

Multiplying by the cost of each epoch we get,

$$\mathbb{E}[\text{evals}] \leq \Theta(n\tau) O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right) \\ = O(n\tau + n^{2k+3} + n^{\beta+1}).$$

$\square$

The static hypermutation would yield a same upper bound with a similar analysis except for the value of β which would be smaller and for the range of c which would be more restricted. Both modifications are necessary due to the larger mutation size of the static hypermutation compared to IPH for a given c value. The $(1+1)$ Opt-IA_{LASTBEST} IPH cannot make the jumps from the junction since any current junction solution would be the last best seen, yielding a mutation potential of one 1. $\mathcal{B}_k$ and $\mathcal{D}_\rho$ sets cannot be accessed from the junction via mutation but can still be sampled at any reinitialization with $2^{-\Omega(n)}$ probability. We conjecture that power-law mutation operators such as those employed by the Fast-IA [3] and the Fast-EA [4] would also succeed on $\text{LTJ}_{k,\rho}$ if equipped with an ageing operator such as the algorithms introduced in [3], and the same should hold for the Rank-GA [63]. However, we believe that the Rank-GA with ageing would fail on CLIFF_d because of its high mutation rate at the bottom of the second slope. On the other hand, the Fast-IA and the Fast-EA, which have been shown to be parameterisable such that they behave equivalently, can quite efficiently (a $\sqrt{n}$ factor slower than the $(1+1)$ Opt-IA_{FIRSTBEST} and the $(1+1)$ Opt-IA_{LASTBEST} IPH i.e., in expected time $O(n^{5/2} \log n)$) optimise CLIFF_d functions with gaps of linear size [3]. However, they have not been shown to be efficient for CLIFF_d functions with sublinear gap sizes. Stagnation detection algorithms [64]–[66] may also be efficient for $\text{LTJ}_{k,\rho}$ if equipped with the ageing operator. It is not obvious, though, how the operator would be incorporated in such an algorithm. Thus, to date the $(1+1)$ Opt-IA_{FIRSTBEST} IPH is the best known algorithm to optimise both Cliff-like functions while at the same time optimise functions where large mutations are necessary to escape from local optima. An open question is whether it may be possible to avoid the use of ageing altogether for $\text{LTJ}_{k,\rho}$ by using the recently introduced population-based EAs with low selective pressure [67], [68]. Low selective pressure may avoid the entire population converging on one of the two local optima shortly after it is identified.

VII. CONCLUSION

We have presented a thorough analysis of Inversely Proportional Hypermutations (IPH). Previous theoretical studies have shown disappointing results concerning the IPH operators from the literature. In this paper we have shown their effectiveness in isolation for unimodal functions compared to static hypermutations in the ideal conditions when the optimum is known. Furthermore, we have provided an analysis of the operator embedded in the complete Opt-IA AIS to be used in practical applications where the optimum is usually unknown. We have proved its effectiveness for two well-studied multimodal functions with considerably different characteristics showing how it harmonises together with the AIS operator better than static hypermutations. In particular the $(1+1)$ Opt-IA_{FIRSTBEST} can escape local optima in three different ways: by mutating into far away solutions (see `LEADINGTRAPJUMP`), by accepting solutions of inferior quality (see `CLIFF`) and naturally by automatically reinitialising the trapped solutions via ageing as understood regarding ageing for a long time [45], [47], [69],

[70]. In particular, compared to the conference version of this paper, we have now proposed an Opt-IA algorithm with IPH that has the capability to escape from local optima via large mutations, something that was not possible with the previous version since the mutation rate remained lowest on the local optima.

Although our analysis used standard benchmark functions capturing specific structures that may appear in combinatorial optimisation problems, we point out that the Opt-IA AIS with our proposed IPH can be shown to efficiently find arbitrarily good approximations to the NP-Hard Number Partitioning problem by following the same proof strategy of [41] that uses that local optima are escaped from thanks the ageing operator that simply restarts the algorithm once trapped on a local optimum. Concerning escaping local optima via hypermutation, IPH would have an insufficient mutation potential to find an approximation in the first run until reinitialisation. However establishing whether after restarts due to ageing triggering on local optima occur, subsequent runs with higher mutation potential can approximate arbitrarily well without relying on a good starting position require further analysis of the problem landscape.

Future work should evaluate the performance of the proposed algorithms and inversely proportional mutation operators both theoretically and experimentally for other combinatorial optimisation problems with practical applications.

REFERENCES

- [1] T. Friedrich, F. Quinzan, and M. Wagner, “Escaping large deceptive basins of attraction with heavy-tailed mutation operators,” in *Proc. of GECCO 2018*, 2018, pp. 293–300.
- [2] T. Friedrich, A. Göbel, F. Quinzan, and M. Wagner, “Heavy-tailed mutation operators in single-objective combinatorial optimization,” in *Proc. of PPSN XV*, 2018, pp. 134–145.
- [3] D. Corus, P. S. Oliveto, and D. Yazdani, “Fast immune system inspired hypermutation operators for combinatorial optimisation,” *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 956–970, 2021.
- [4] B. Doerr, H. P. Le, R. Makhmara, and T. D. Nguyen, “Fast genetic algorithms,” in *Proc. of GECCO 2017*, 2017, pp. 777–784.
- [5] D. Corus and P. S. Oliveto, “Standard steady state genetic algorithms can hillclimb faster than mutation-only evolutionary algorithms,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 720–732, 2018.
- [6] D.-C. Dang, T. Friedrich, T. Kötzing, M. S. Krejca, P. K. Lehre, P. S. Oliveto, D. Sudholt, and A. M. Sutton, “Escaping local optima using crossover with emergent diversity,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 3, pp. 484–497, 2018.
- [7] D. Corus and P. S. Oliveto, “On the benefits of populations on the exploitation speed of standard steady-state genetic algorithms,” in *Proc. of GECCO 2019*, 2019, pp. 1452–1460.
- [8] P. S. Oliveto, D. Sudholt, and C. Witt, “A tight lower bound on the expected runtime of standard steady state genetic algorithms,” in *Proc. of GECCO 2020*, 2020, pp. 1323–1331.
- [9] B. Doerr, C. Doerr, and F. Ebel, “From black-box complexity to designing new genetic algorithms,” *Theoretical Computer Science*, vol. 567, pp. 87–104, 2015.
- [10] J. Lengler, “A general dichotomy of evolutionary algorithms on monotone functions,” *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 6, pp. 995–1009, 2019.
- [11] B. Doerr and C. Doerr, “Theory of parameter control for discrete black-box optimization: Provable performance gains through dynamic parameter choices,” in *Theory of Evolutionary Computation - Recent Developments in Discrete Optimization*, B. Doerr and F. Neumann, Eds. Springer, 2020, pp. 271–321.
- [12] B. Doerr, A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, “On the runtime analysis of selection hyper-heuristics with adaptive learning periods,” in *Proc. of GECCO 2018*, 2018, pp. 1015–1022.
- [13] B. Doerr, C. Doerr, and J. Yang, “k-bit mutation with self-adjusting k outperforms standard bit mutation,” in *Proc. of PPSN’16*. Springer, 2016, pp. 824–834.
- [14] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, “Simple Hyper-Heuristics Control the Neighbourhood Size of Randomised Local Search Optimally for LeadingOnes,” *Evolutionary Computation*, vol. 28, no. 3, pp. 437–461, 09 2020.
- [15] —, “How the duration of the learning period affects the performance of random gradient selection hyper-heuristics,” in *Proc. of AAAI-20*, 2020, pp. 2376–2383.
- [16] P. S. Oliveto, T. Paixão, J. Pérez Heredia, D. Sudholt, and B. Trubenová, “How to escape local optima in black box optimisation: when non-elitism outperforms elitism,” *Algorithmica*, vol. 80, pp. 1604–1633, 2018.
- [17] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, “When move acceptance selection hyper-heuristics outperform metropolis and elitist evolutionary algorithms and when not,” *Artificial Intelligence*, vol. 314, p. 103804, 2023.
- [18] D.-C. Dang, A. V. Eremeev, and P. K. Lehre, “Escaping local optima with non-elitist evolutionary algorithms,” in *Proc. of AAAI 2021*, 2021.
- [19] B. Doerr, “Does comma selection help to cope with local optima?” in *Proc. of GECCO ’20*. ACM, 2020, p. 1304–1313.
- [20] B. Case and P. K. Lehre, “Self-adaptation in non-elitist evolutionary algorithms on discrete problems with unknown structure,” *IEEE Transactions on Evolutionary Computation (To appear - arXiv:2004.00327)*, vol. 24, no. 4, pp. 650–663, 2020.
- [21] D.-C. Dang, T. Jansen, and P. K. Lehre, “Populations can be essential in dynamic optimisation,” in *Proc. of GECCO 2015*, 2015, pp. 1407–1414.
- [22] D.-C. Dang and P. K. Lehre, “Efficient optimisation of noisy fitness functions with population-based evolutionary algorithms,” in *Proc. of FOGA 2015*, 2015, pp. 62–68.
- [23] J. Lengler, L. Schiller, and O. Sieberling, “Plus strategies are exponentially slower for planted optima of random height,” in *Proc. of GECCO 2024*, 2024, pp. 1587–1595.
- [24] M. A. Hevia-Fajardo and D. Sudholt, “Self-adjusting population sizes for non-elitist evolutionary algorithms: Why success rates matter,” *Algorithmica*, vol. 86, no. 2, pp. 526–565, 2024.
- [25] L. R. de Castro and J. Timmis, *Artificial Immune Systems: A New Computational Intelligence Paradigm*. Springer-Verlag, 2002.
- [26] Z. Huang and Y. Zhou, “Runtime analysis of somatic contiguous hypermutation operators in MOEA/D framework,” in *Proc. of AAAI*, 2020, pp. 2359–2366.
- [27] F. M. Burnet, *The Clonal Selection Theory of Acquired Immunity*. Cambridge University Press, 1959.
- [28] E. A. Aguilar Arroyo and C. A. Coello Coello, “Moaisdx: A new multi-objective artificial immune system based on decomposition,” in *Proc. of EMO 2025*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 46–59.
- [29] L. Li, Q. Lin, Z. Ming, M. Gong, and C. A. Coello Coello, “An immune-inspired resource allocation strategy for many-objective optimization,” *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 6, pp. 3284–3297, 2022.
- [30] D. Corus, P. S. Oliveto, and D. Yazdani, “Fast contiguous somatic hypermutations for single-objective optimisation and multi-objective optimisation via decomposition,” in *Proc. of AAAI’25*, 2025, pp. 26 922–26 930.
- [31] Z. Huang and Y. Zhou, “Runtime analysis of somatic contiguous hypermutation operators in moea/d framework,” in *Proc. of AAAI 2020*, 2020, pp. 2359–2366.
- [32] L. Li, Q. Lin, and Z. Ming, “A survey of artificial immune algorithms for multi-objective optimization,” *Neurocomputing*, vol. 489, pp. 211–229, 2022.
- [33] J. Brownlee, “Clonal selection algorithms,” Complex Intelligent Systems Laboratory (CIS), Swinburne University of Technology, Tech. Rep., 2007.
- [34] L. N. de Castro and F. J. V. Zuben, “Learning and optimization using the clonal selection principle,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 3, pp. 239–251, 2002.
- [35] V. Cutello, G. Nicosia, M. Pavone, and J. Timmis, “An immune algorithm for protein structure prediction on lattice models,” *IEEE Transactions on Evolutionary Computation*, vol. 11, pp. 101–117, 2007.
- [36] B. H. Ulutas and S. Kulturel-Konak, “A review of clonal selection algorithm and its applications,” *Artificial Intelligence Review*, vol. 36, pp. 117–138, 2011.
- [37] A. Auger and N. Hansen, “Theory of evolution strategies: a new perspective,” in *Theory of Randomized Search Heuristics: Foundations and Recent Developments*, A. Auger and B. Doerr, Eds. World Scientific Publishing Co., Inc., 2011, pp. 289–325.

- [38] C. Zarges, “Rigorous runtime analysis of inversely fitness proportional mutation rates,” in *Proc. of PPSN*, 2008, pp. 112–122.
- [39] D. Corus, P. S. Oliveto, and D. Yazdani, “When hypermutations and ageing enable artificial immune systems to outperform evolutionary algorithms,” *Theoretical Computer Science*, vol. 832, pp. 166–185, 2019.
- [40] —, “Artificial immune systems can find arbitrarily good approximations for the NP-hard partition problem,” in *Proc. of PPSN 2018*, 2018, pp. 16–28.
- [41] —, “Artificial immune systems can find arbitrarily good approximations for the NP-hard number partitioning problem,” *Artificial Intelligence*, vol. 247, pp. 180–196, 2019.
- [42] T. Jansen and C. Zarges, “Variation in artificial immune systems: hypermutations with mutation potential,” in *Proc. of ICARIS 2011*, 2011, pp. 132–145.
- [43] S. Droste, T. Jansen, and I. Wegener, “On the analysis of the $(1 + 1)$ evolutionary algorithm,” *Theoretical Computer Science*, vol. 276, no. 1-2, pp. 51–81, 2002.
- [44] J. Jägersküpper and T. Storch, “When the plus strategy outperforms the comma strategy and when not,” in *Proc. of FOCI 2007*, 2007, pp. 25–32.
- [45] P. S. Oliveto and D. Sudholt, “On the runtime analysis of stochastic ageing mechanisms,” in *Proc. of GECCO 2014*, 2014, pp. 113–120.
- [46] D. Corus, P. S. Oliveto, and D. Yazdani, “On the runtime analysis of the Opt-IA artificial immune system,” in *Proc. of GECCO 2017*, 2017, pp. 83–90.
- [47] C. Horoba, T. Jansen, and C. Zarges, “Maximal age in randomized search heuristics with aging,” in *Proc. of GECCO 2009*, 2009, pp. 803–810.
- [48] P. S. Oliveto, P. K. Lehre, and F. Neumann, “Theoretical analysis of rank-based mutation - combining exploration and exploitation,” in *Proc. of CEC 2009*, 2009, pp. 1455–1462.
- [49] D. Corus, P. S. Oliveto, and D. Yazdani, “On inversely proportional hypermutations with mutation potential,” in *Proc. of GECCO 2019*, 2019, p. 215–223.
- [50] P. S. Oliveto and C. Witt, “On the runtime analysis of the simple genetic algorithm,” *Theoretical Computer Science*, vol. 545, pp. 2–19, 2014.
- [51] —, “Improved time complexity analysis of the simple genetic algorithm,” *Theoretical Computer Science*, vol. 605, pp. 21–41, 2015.
- [52] D. Whitley, “The genitor algorithm and selection pressure: Why rank-based allocation of reproductive trials is best,” in *Proc. of the Third International Conference on Genetic Algorithms*, 1989, pp. 116–121.
- [53] F. Neumann, P. S. Oliveto, and C. Witt, “Theoretical analysis of fitness-proportional selection: landscapes and efficiency,” in *Proc. of GECCO 2009*, 2009, pp. 835–842.
- [54] D. Corus, D.-C. Dang, A. V. Eremeev, and P. K. Lehre, “Level-based analysis of genetic algorithms and other search processes,” *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 707–719, 2017.
- [55] D. Corus and P. S. Oliveto, “On the benefits of populations for the exploitation speed of standard steady-state genetic algorithms,” *Algorithmica*, vol. 82, pp. 3676–3706, 2020.
- [56] S. Böttcher, B. Doerr, and F. Neumann, “Optimal fixed and adaptive mutation rates for the leadingones problem,” in *Proc. of PPSN’10*. Springer, 2010, pp. 1–10.
- [57] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, “How the duration of the learning period affects the performance of random gradient selection hyper-heuristics,” in *Proc. of AAAI 2020*, 2020, pp. 2376–2383.
- [58] W. Feller, *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, 1968.
- [59] D. Corus, P. S. Oliveto, and D. Yazdani, “Fast artificial immune systems,” in *Proc. of PPSN 2018*, 2018, pp. 67–78.
- [60] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, “On the time complexity of algorithm selection hyper-heuristics for multimodal optimisation,” in *Proc. of AAAI 2019*, 2019, pp. 2322–2329.
- [61] A. Bendahi, B. Doerr, A. Fradin, and J. F. Lutzeyer, “Speeding up hyper-heuristics with markov-chain operator selection and the only-worsening acceptance operator,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, ser. IJCAI ’25, 2025.
- [62] J. L. Doob, *Stochastic Processes*. John Wiley & Sons, 1953.
- [63] J. Cervantes and C. R. Stephens, “Rank based variation operators for genetic algorithms,” in *Proc. of GECCO 2008*, 2008, pp. 905–912.
- [64] M. S. Krejca and C. Witt, “A flexible evolutionary algorithm with dynamic mutation rate archive,” in *Proc. of GECCO ’24*, 2024, p. 1578–1586.
- [65] A. Rajabi and C. Witt, “Self-adjusting evolutionary algorithms for multimodal optimization,” *Algorithmica*, vol. 84, no. 6, pp. 1694–1723, 2022.
- [66] —, “Stagnation detection with randomized local search,” *Evolutionary Computation*, vol. 31, no. 1, pp. 1–29, 2023.
- [67] D. Corus, A. Lissovoi, P. S. Oliveto, and C. Witt, “On steady-state evolutionary algorithms and selective pressure: Why inverse rank-based allocation of reproductive trials is best,” *ACM Transactions on Evolutionary Learning and Optimization*, vol. 1, no. 1, 2021.
- [68] D. Corus, P. S. Oliveto, and F. Zheng, “Hybrid selection allows steady-state evolutionary algorithms to control the selective pressure in multimodal optimisation,” in *Proc. of GECCO 2025*, 2025, pp. 881–889.
- [69] T. Jansen and C. Zarges, “On benefits and drawbacks of aging strategies for randomized search heuristics,” *Theoretical Computer Science*, vol. 412, no. 6, pp. 543–559, 2011.
- [70] —, “Aging beyond restarts,” in *Proc. of GECCO 2010*, 2010, pp. 705–712.

VIII. BIOGRAPHY SECTION

If you have an EPS/PDF photo (graphicx package needed), extra braces are needed around the contents of the optional argument to biography to prevent the LaTeX parser from getting confused when it sees the complicated `\includegraphics` command within an optional argument. (You can create your own custom macro containing the `\includegraphics` command to make things simpler here.)

If you include a photo:

Michael Shell Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

If you will not include a photo:

John Doe Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.

Effective Inversely Proportional Hypermutations for Unimodal and Multimodal Optimisation

Anonymous authors

Abstract—Artificial Immune Systems (AIS) employing hypermutations with linear static mutation potential have recently been shown to be very effective at escaping local optima of combinatorial optimisation problems at the expense of being slower during the exploitation phase compared to standard evolutionary algorithms. In this paper, we examine the more commonly used dynamic inversely proportional hypermutations (IPH) with mutation potential for which previous theoretical analyses have shown disappointing performance. We first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations in the ideal situation where the optimum is known if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. Afterwards, we define a simple (1+1) Opt-IA that uses IPH and ageing for realistic applications where optimal solutions are unknown. The aim of this AIS is to approximate the ideal behaviour of the IPH better and better as the search space is explored and local optima of higher quality are identified. We present two variants, one that decreases the mutation rate proportionally until the local (global) optimum is identified, and another where the mutation rate may increase again when detecting that the algorithm is stuck on a large plateau of local optima. We show that the latter variant outperforms standard bit mutations by escaping the local optima on a multimodal function from the literature and that both variants outperform static hypermutations by escaping the local optima of the standard CLIFF benchmark function by accepting solutions of lower fitness.

Index Terms—Artificial immune systems, heavy-tailed mutations, inversely proportional hypermutations, theory, runtime analysis

I. INTRODUCTION

Three recent hot topics in the theory of randomised search heuristics for optimisation in general and in artificial intelligence in particular have been the benefits of mutation operators with higher mutation rates than traditionally employed [1]–[4] (or standard operators using higher parameter values than those previously recommended [5]–[10]), the automatic adaptation of the mutation rate [11]–[15] and the analysis of when non-elitist search heuristics are beneficial compared to elitist ones [16]–[24].

Artificial Immune Systems (AIS) are a class of bio-inspired algorithms which naturally use both high mutation rates via so-called hypermutation operators, often automatically adapt the mutation rate, and use non-elitism implicitly through the use of ageing operators, characteristics that are typical in the immune system of vertebrates from which AIS draw inspiration. In this paper we analyse the performance of an adaptive mutation operator called Inversely Proportional Hypermutation (IPH) that aims to increase the mutation rate with (an estimate of) the distance from the optimum, and propose a considerably improved one for efficient unimodal and multimodal optimisation.

AIS for optimisation [25], [26] are generally inspired by the clonal selection principle [27]. For this reason they are also often referred to as *clonal selection* algorithms [25]. Clonal selection algorithms are also *popular* in evolutionary multi-objective optimisation [28]–[32]. In the literature two key features of clonal selection algorithms have been identified [33]:

- 1) The proliferation rate of each immune cell is proportional to its affinity (similarity) with the selective antigen (pathogen): the higher the affinity, the higher the number of offspring generated (clonal selection and expansion).
- 2) The mutation suffered by each immune cell during reproduction is inversely proportional to the affinity of the cell receptor with the antigen: the higher the affinity, the smaller the mutation (affinity maturation).

Indeed, well-known clonal selection algorithms employ mutation operators applied to immune cells (i.e., candidate solutions) with a rate that decreases with their similarity to the antigen (i.e., global optima) during affinity maturation. Often such operators are referred to as *inversely proportional hypermutations* (IPH). Two of the most popular clonal selection algorithms, Clonalg [34] and the Opt-IA immune algorithm [35] (as well as their many variants [36]) both employ inversely proportional hypermutations. The ideal behaviour of the IPH operator is that the mutation rate is minimal in proximity of the global optimum and increases as the difference between the fitness of the global optimum and that of the candidate solution increases. However, achieving such behaviour in practice may be problematic because the fitness of the global optimum is usually unknown. As a result, in practical applications information about the problem is used to find bounds on (or estimates of) the value of the global optimum¹. Thus, the closer the estimate is to the actual value of the global optimum, the closer should the behaviour of the IPH operator be to the desired one. On the other hand, if the bound is much higher (e.g., for a maximisation problem) than the true value, then there is a risk that the mutation rate is too high in proximity of the global optimum i.e., the algorithm will struggle to identify the optimum.

While decreasing the step size as the optimum is approached is a common practice in continuous optimisation [37], previous theoretical analyses for combinatorial optimisation, though, have highlighted various problems with IPH operators from the literature even when the fitness value of the global optimum is known. Zarges [38] analysed the effects of mutating

¹Alternatively, the fitness of the best candidate solution is sometimes used and the mutation rate of the rest of the population is inversely proportional to the best.

candidate solutions with a rate that is inversely proportional to their fitness for the ONEMAX problem. She considered two different rates for the decrease of the mutation rate as the fitness increases: a linear decay (i.e., each bit is flipped with probability $\frac{\text{OPT}-\text{ONEMAX}(x)}{\text{OPT}}$, where OPT is the value of the optimum) and an exponential decay (i.e., each bit flips with probability $e^{-\rho \frac{\text{ONEMAX}(x)}{\text{OPT}}}$ where ρ is called the *decay* parameter). The motivation behind these choices are that the former operator flips in expectation exactly the number of bits that maximises the probability of reaching the optimum in the next step while the latter is the operator used in the Clonal algorithm. She showed that if the optimum of ONEMAX is known, then an algorithm employing such mutation operators will require exponential time to optimise ONEMAX with overwhelming probability (w.o.p.) in both cases of linear or exponential decays of the mutation rate. The reason is that the initial random solutions that have roughly half the fitness (i.e., $n/2$) of the global optimum (i.e., n) have very high mutation rates. Such rates do not allow the algorithm to make any progress with any reasonable probability even if the decay parameter ρ is chosen very carefully (i.e., $\rho = \ln n$ leads to reasonable mutation rates between $1/n$ and $1/2$ during the optimisation of ONEMAX). Hence, the behaviour of the mutation operator leads to very inefficient performance even for the simple OneMax function when the optimum is assumed to be known.

On the other hand, the Opt-IA clonal selection algorithm, that uses a mutation operator called 'hypermutations with mutation potential' which also has very high mutation rates has been proven to be efficient by employing a selection strategy called *stop at first constructive mutation* (FCM). The strategy evaluates the fitness after each bit is flipped and interrupts the hypermutation immediately if an improvement is detected. Otherwise it returns the bit-string obtained after it has flipped the maximum number of bits it is allowed to flip i.e., the mutation potential. Indeed, the operator using a static mutation potential (i.e., a linear number of bits are flipped unless an improvement is found along the way) has been proven to be very effective at escaping local optima of standard multimodal benchmark functions [39] and at finding arbitrarily good approximations for the Number Partitioning NP-Hard problem [40], [41] at the expense of being slower at hillclimbing during the exploitation phases - it is up to a linear factor slower than the standard bit mutation (SBM) used by evolutionary algorithms for easy unimodal functions such as OneMax and LeadingOnes. Hence, the first insight we provide in this paper is that, differently from other clonal selection algorithms (such as Clonalg and Opt-IA without FCM), hypermutations with mutation potential coupled with FCM can cope with the desired behaviour of inversely proportional mutation rates by being capable of making progress efficiently with high mutation rates. In this paper we first consider whether Opt-IA may become faster during the exploitation phases if inversely proportional mutation potentials are applied rather than static ones. The reason to believe this is that the hypermutations waste many fitness function evaluations during exploitation, since the probability of flipping bits that lead to an improvement decreases when the optimum is approached.

Hence, with high probability the operator flips wrong bits at the beginning of a hypermutation and ends up wasting a linear number of fitness function evaluations in each hypermutation. On the other hand, if the mutation potential decreases as the optimum is approached, then the amount of wasted evaluations should decrease accordingly.

A previous runtime analysis for OneMax of the inversely proportional hypermutations with mutation potential used by Opt-IA [42] has shown that the mutation rate is always in the range $[(c/2)n, cn]$ where $M = cn$ is the highest mutation potential. Hence it does not decrease proportionally with the decrease in distance to the optimum as desired and, as a result, no asymptotic speed-ups are achieved compared to static mutation potentials.

In this paper we first show that considerable speed-ups in hillclimbing phases may be achieved compared to static hypermutations, if the mutation rates decrease appropriately with either the fitness or the distance to the optimum. To show this, we analyse the IPH operators for standard unimodal benchmark functions where the performance of static mutation potentials is well-understood [39]. We first identify what speed-ups may be hoped for by analysing the operators in the ideal situation where the location of the optimum is known. A result of this first analysis is that a mutation potential that increases exponentially with the Hamming distance to the optimum provides the larger speed-ups for hillclimbing unimodal functions. Furthermore, using the Hamming distance rather than the fitness as the measure to quantify proximity to the optimum makes the operator robust to fitness function scaling. However, an exponential decay with the distance to the optimum also implies that high mutation rates would occur only when the current solution is prohibitively far away from the optimum, *thus, the algorithm would flip just one bit most of the time, i.e., very rare hyper-mutations*. For instance, a randomly generated solution with expected Hamming distance $n/2$ from the optimum flips $\sqrt{n}$ bits and this mutation will decrease exponentially fast with further improvements. For this reason we find that a linear decrease of the mutation rate is a better compromise in *practice* as it allows for longer phases where large mutations occur and it flips the number of bits that increase the probability of hitting the optimum in the next step.

Afterwards we propose a clonal selection algorithm that we call (1+1) Opt-IA that uses IPH and ageing² [35] to be applied in practical unimodal and multimodal applications where the optimum is not known. The algorithm uses the best solution it has encountered to estimate the mutation rates. In the literature it has been shown that ageing allows algorithms to escape from local optima either by identifying a new slope of increasing fitness or by completely restarting the optimisation process if it cannot escape [41], [42], [45]–[47]. The idea behind our proposed AIS is that the more the search space is explored, the better is the ideal behaviour of the IPH approximated through the discovery of better and better local optima. We propose two variants. The first one, called (1 + 1) Opt-IA_{LASTBEST}, updates the location of the best seen solution whenever a new solution with fitness value at least as good as the previous one

²The well-known Opt-IA AIS uses both operators in combination [35].

TABLE I: Expected runtimes of the different mutation operators when embedded in a (1+1) algorithmic framework for the benchmark functions considered in this paper. For all results the optimum is unknown to the algorithms. The * indicates that we do not find it worth the effort to perform a formal analysis, yet for reasons provided at the end of Section VI, the upper bound is correct. $\beta := \log_2 \frac{20}{c}$, where $c \leq 1$ is the constant factor in the M_{linHD} potential $M = \lfloor c \cdot H(x, \text{best}) \rfloor$.

(1+1) Algorithms	ONEMAX	LEADINGONES	CLIFF _d	LEADINGTRAPJUMP _{k, ρ}
EA (SBM)	$\Theta(n \log n)$ [43]	$\Theta(n^2)$ [43]	$\Theta(n^d)$ [44]	$n^{\Omega(\log n)}$
Opt-IA _{STATIC}	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]	$O(\frac{n^{d+1} \cdot e^d}{d^d})$ [39]	$O(n\tau + n^{2k+3} + n^{\beta+1})^*$
Opt-IA _{LASTBEST}	$\Theta(n \log n)$	$\Theta(n^2)$	$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$	$2^{\Omega(n)}$
Opt-IA _{FIRSTBEST}	$\Theta(n \log n)$	$\Theta(n^3)$	$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$	$O(n\tau + n^{2k+3} + n^{\beta+1})$

is identified. This guarantees that the mutation rate is minimal on the optimum once it is identified, exactly in the spirit of inversely proportional hypermutations, and allows for the best possible expected runtimes when hillclimbing unimodal functions such as ONEMAX and LEADINGONES. However, the Opt-IA using such an operator loses power compared to static hypermutations because it cannot escape from the best identified local optima via large mutations as its mutation rate will be minimal. To this end, we define a second variant called (1 + 1) Opt-IA_{FIRSTBEST}, that only updates the location of the best seen solution when a strictly better solution is identified. This algorithm is capable of increasing its mutation rate again when stuck on large plateaus of local optima by increasing its Hamming distance from the best seen solution.

We first use the CLIFF benchmark function to prove the effectiveness of the IPH over static hypermutations. In particular, we show that the IPH can escape local optima when coupled with ageing, because this operator allows the algorithm to escape the local optimum by accepting solutions of inferior fitness. We remark that static hypermutations cannot optimise the function efficiently because the operator only stops if a constructive mutation is identified, thus will not return inferior solutions that may survive the ageing operator and allow the algorithm to escape the local optima of the function class. This makes a difference between polynomial and superpolynomial runtimes for super-constant cliff distances, and an expected runtime of $O(n^2 \log n)$ versus exponential for linear cliffs. A different problem would occur if hypermutations inversely proportional to fitness were to be used. Since the difference in fitness between the local optima and the bottom of the Cliff is very large, the operator's mutation rate at the bottom of the cliff would be too high leading the algorithm back to the local optima with high probability.

We conclude the paper by providing an example benchmark function where it is necessary to escape from local optima using large mutations. Thus the (1 + 1) Opt-IA_{FIRSTBEST} is efficient while traditional standard bit mutations (SBM) that use lower mutation rates are not. The benchmark function, called LEADINGTRAPJUMP, is inspired from one previously used in the literature to highlight when high mutation rates are beneficial and when not [48]. Essentially, by switching the fitness values of the local optima and the global optimum one can provide examples where SBM outperforms high mutation

rates and viceversa. Herein we show that the (1 + 1) Opt-IA_{FIRSTBEST} can identify the global optimum in both cases.

A summary of the results obtained in the paper is provided in Table I highlighting the speed-ups achieved by the two IPH operators compared to the standard bit mutations traditionally used by evolutionary algorithms and to the static hypermutation operator with FCM. It can be noticed how the (1 + 1) Opt-IA_{FIRSTBEST} is a better compromise to hillclimb faster and allow escaping local optima both by accepting solutions of inferior fitness *and* by performing large jumps in the search space making a difference between polynomial and super-polynomial runtimes.

The original extended abstract [49] of this paper analysed the advantages of using an exponential decay of the mutation rate since this provides the largest speed-ups for unimodal functions in ideal conditions where the optimum is known (see Section III). However, a great disadvantage of the exponential decay is that large mutations hardly ever occur in practice. Indeed no superiority of IPH over a SBM operator was provided. To this end, in this paper we analyse the use of the linear decay operator and provide examples where the IPH indeed considerably outperforms SBM providing a difference between polynomial and superpolynomial expected runtimes. Due to space restrictions a self-contained paper presenting all proofs is provided as supplementary material.

II. INVERSELY PROPORTIONAL MUTATION POTENTIALS

Hypermutations with mutation potential using FCM flip at most a linear amount of bits (i.e., the mutation potential $M = cn$) chosen uniformly at random without replacement from the interval $[1..n]$. After each of the M bit flips, the fitness of the produced solution is evaluated and if a *constructive mutation* is identified, then the operator is stopped and the solution returned. A constructive mutation is a solution that is at least as good as that of the original parent. Otherwise the algorithm continues flipping the bits without setting them back until the potential M of bit flips is reached. If no constructive mutation has been identified earlier, then the M -th solution is returned. The original inversely proportional hypermutation operator (IPH) proposed for Opt-IA uses mutation potential $M = \lceil (1 - f_{\text{OPT}}/v)cn \rceil$ for minimisation problems, where $f_{\text{OPT}} \geq 0$ is the optimal fitness value, $v > 0$ is the fitness of the current individual, and $0 < c \leq 1$ is some constant [35],

[42]. In an analysis for $\text{ZEROMIN} := n + 1 - \text{ONEMAX}$ [42], such a mutation potential was shown not to decrease below $cn/2$, with cn being the highest possible mutation potential. As a result no speed-ups at hillclimbing are achieved compared to static hypermutations as the mutation potential is always at least linear. In this section, we introduce three different IPH operators that will be analysed in this paper and that provably speed-up the hillclimbing performance for ONEMAX. Two have been already considered in the literature while the third one is newly proposed by us based on analysed drawbacks of the first two at speeding up hillclimbing.

Hamming Distance Based Linear Decrease

Zarges analysed an inversely proportional hypermutation operator where the probability of flipping each bit increases linearly with the Hamming distance to the optimum or its best available estimate [38]. Precisely, this operator flips each bit with probability $H(x, \text{best})/n$ where n is the size of the problem and $H(x, \text{best})$ is the Hamming distance of the current point to the best individual (or an estimate if not available). As the expected number of bit-flips is $H(x, \text{best})$ during each execution of a mutation operator, a mutation potential inspired by this inversely proportional hypermutation is:

$$M_{\text{linHD}}(x) := H(x, \text{best}).$$

Fitness Difference Based Exponential Decrease

In Clonalg's IPH, the mutation probability decreases as an exponential function of the fitness of the current solution [34]. Precisely, each bit flips with probability $e^{-\rho \cdot v}$ where v is the normalised fitness value and ρ is a decay parameter that regulates the speed at which the mutation rate decreases. Since we consider maximisation problems, we use $v = \frac{f(x)}{f(\text{best})}$ as suggested by [38] where $f(\text{best})$ is the best known fitness value. Using the expected number of bits flipped by this mutation operator as a mutation potential gives $M = n \cdot e^{-\rho v}$. According to both experimental and theoretical analyses [38], a reasonable value for ρ is $\ln n$. We call this mutation potential $M_{\text{expoF}(x)}$ and define it as:

$$M_{\text{expoF}(x)}(x) := \left\lfloor n^{1 - \frac{f(x)}{f(\text{best})}} \right\rfloor.$$

Hamming Distance Based Exponential Decrease

It is well understood that fitness-proportional selection is sensitive to the difference between the fitness values of candidate solutions, i.e., it struggles to distinguish between solutions that have similar, yet different, fitness values [50]–[53]. For this reason, nowadays selection operators that rank individuals by fitness (e.g., tournament, linear ranking, comma selection) are generally preferred. We also consider a measure that avoids using the fitness values directly because, apart from the above consideration, the ideal amount of bits to be flipped clearly depends on the genotypic distance of the candidate solution to the optimum rather than on their difference in fitness values. To this end, we suggest a mutation potential which is similar to $M_{\text{expoF}(x)}$ with the exception that it uses the

normalised Hamming distance to the best estimate rather than the normalised fitness. We call this mutation potential M_{expoHD} and define it as $M_{\text{expoHD}} = \left\lfloor n \cdot e^{-\rho \frac{n - H(x, \text{best})}{n}} \right\rfloor$ where n is the problem size (the maximum Hamming distance between any two points), $H(x, \text{best})$ is the Hamming distance to the best known solution and ρ is the decay of the mutation potential. For the choice of $\rho = \ln n$, we get:

$$M_{\text{expoHD}}(x) := \left\lfloor n^{\frac{H(x, \text{best})}{n}} \right\rfloor.$$

III. PERFORMANCE GAINS IN IDEAL CONDITIONS

In this section, we evaluate the performance of the three different IPH operators, assuming the optimum is known (i.e., $\text{best} = \text{opt}$). Under this assumption, the operators exhibit their ideal behaviour (i.e., the mutation potential decreases with the desired rate as the optimum is approached). Our aim is to evaluate what speed-ups can be achieved in ideal conditions compared to the well-studied static mutation potentials. To achieve such comparisons we perform runtime analyses of the (1+1) IA using the IPH on the standard ONEMAX and LEADINGONES unimodal benchmark functions for which the performance of the same algorithm using static mutation potentials is known [39]. The *simple-to-define* ONEMAX function counts the number of 1-bits in a bit string and is normally used to evaluate the hill climbing ability of algorithms [5], [9], [54], [55]. On the other hand, LEADINGONES is a slightly harder unimodal problem which counts the consecutive number of 1-bits at the beginning of the bit string before the first 0-bit *or the end of the string*. Evolutionary algorithms using standard bit mutation (or bit flip local search mutation) optimise the functions respectively in $\Theta(n \log n)$ and $\Theta(n^2)$ expected fitness function evaluations [12], [13], [43], [56], [57], while static hypermutations are a linear factor slower [39].

While in practice both evolutionary algorithms and artificial immune systems use a population of solutions, in this paper we consider single trajectory algorithms to better highlight the performance of all the considered hypermutation operators. The pseudo-code of a simple AIS, the (1+1) Immune Algorithm ((1+1) IA), is given in Algorithm 1. It simply uses one candidate solution in each iteration to which inversely proportional hypermutations are applied. For the function $\text{Hypermutate}(x)$, IPH performs two steps: 1) it calculates the mutation potential M , and 2) it creates y by flipping at most M distinct bits of x selected uniformly at random without replacement one after another until a constructive mutation happens. Otherwise it returns the last constructed solution. A constructive mutation is a mutation step where the evaluated bit string is at least as fit as the parent x . The pseudo-code of the operator once the mutation potential M has been calculated is depicted in Algorithm 2. The results proven in this section and comparisons with static hypermutations and the IPH operators from the literature are summarised in Table II.

Before stating the main results, we first introduce two lemmata. The first is the Ballot theorem which is used throughout this paper to derive lower bounds on the expected runtime of the algorithms [39], [42], [58]. This theorem is essentially

TABLE II: Comparison of the expected runtimes obtained by different hypermutation schemes for ONEMAX and LEADINGONES. The original IPH operator of Opt-IA [35] has the same expected runtimes as the $(1+1)$ IA using static hypermutations [42]. ‘*’ indicates that the result can be easily drawn from the analysis for ONEMAX [38].

$(1+1)$ IA	ONEMAX	LEADINGONES
Using static hypermutation	$\Theta(n^2 \log n)$ [39]	$\Theta(n^3)$ [39]
Clonalg with linear decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)*}$
Clonalg with expo decay	$e^{\Omega(n)}$ [38]	$e^{\Omega(n)*}$
Traditional Opt-IA	$\Theta(n^2 \log n)$ [42]	$\Theta(n^3)$ [42]
FCM with M_{linHD}	$\Theta(n^2)$	$\Theta(n^3)$
FCM with $M_{\text{expoF}(x)}$	$\Theta(n^{3/2}/\log n)$	$\Theta(n^3/\log^2 n)$
FCM with M_{expoHD}	$\Theta(n^{3/2}/\log n)$	$\Theta(n^{5/2}/\log^2 n)$

Algorithm 1 $(1+1)$ IA

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: Evaluate  $f(x)$ ;
3: while termination condition is not satisfied do
4:   Update mutation potential  $M$ ;
5:    $y := \text{Hypermutate}(x, M)$ ;
6:   if  $f(y) \geq f(x)$  then
7:      $x := y$ .
8:   end if
9: end while

```

used to show the probability of picking more 1-bits than 0-bits during one hypermutation operation given that there are more 1-bits in the initial bit-string (or vice-versa).

Lemma 1 (Ballot Theorem [58]). *In a ballot, suppose that candidate P receives p votes and candidate Q receives q votes such that $p > q$. The probability that throughout the counting P is always ahead of Q is $(p - q)/(p + q)$.*

In the context of hypermutation operators, the vote counts are the initial number of 0 and 1-bits in the solutions, while counting the votes correspond to flipping them until the first constructive mutation. The probability that a constructive mutation is found is equivalent to the probability of the Q candidate having as many votes counted as the other during any time in the counting process.

Our second lemma will be used to transform inverse polynomial exponents that arise from the exponential potential.

Lemma 2. *For large enough n and any arbitrary constant ϵ , $n^{1/n^\epsilon} = (1 + \frac{\ln n}{n^\epsilon})(1 \pm o(1))$.*

Proof. $n^{\frac{1}{n^\epsilon}} = e^{\frac{\ln(n)}{n^\epsilon}} = 1 + \frac{\ln(n)}{n^\epsilon} + o(\frac{\ln(n)}{n^\epsilon})$ by the Taylor’s series of the exponent. $\square$

A. Performance for ONEMAX in Ideal Conditions

The following theorems derive the expected runtimes using each of the three IPH operators for the function

Algorithm 2 HYPERMUTATE(x, M)

```

1:  $y \leftarrow x$ ;
2: for  $i = 1$  to  $M$  do
3:   Flip one previously unflipped bit of  $y$  chosen uniformly at random;
4:   Evaluate  $f(y)$ ;
5:   if  $f(y) \geq f(x)$  then
6:     return  $y$ ;
7:   end if
8: end for
9: return  $y$ ;

```

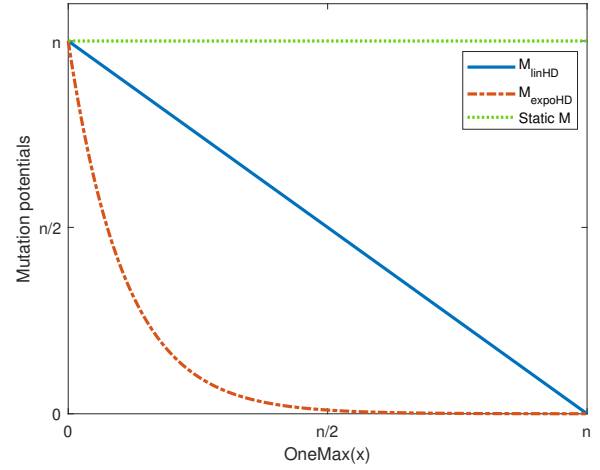


Fig. 1: Static and inversely proportional mutation potentials for ONEMAX.

$\text{ONEMAX}(x) := \sum_{i=1}^n x_i$. Figure 1 shows how the studied mutation potentials ideally decrease during the run of the algorithm when optimising the function. Notice that for ONEMAX the behaviours of $M_{\text{expoF}(x)}$ and M_{expoHD} are the same. By decreasing the potential linearly with the decrease of the Hamming distance, a logarithmic factor may be shaved off from the expected runtime of the $(1+1)$ IA compared to the expected runtime achieved by the same algorithm using static hypermutation potentials.

Theorem 1. *The $(1+1)$ IA using IPH with M_{linHD} optimises ONEMAX in $\Theta(n^2)$ expected fitness function evaluations.*

Proof. Considering i as the number of 0-bits in the candidate solution, the probability of improvement in the first step is i/n . Knowing that at most n improvements are needed to find the optimum and that in case of failure $H(x, \text{opt}) = i$ fitness function evaluations would be wasted, the total expected time to optimise ONEMAX is at most $\sum_{i=1}^n \frac{n}{i} \cdot i = O(n^2)$.

For the lower bound, we use the Ballot theorem (Lemma 1). We flip at most $M \leq n$ bits, and the probability that an improvement happens is the probability that at some moment there will be at least as many flipped zeros as there are flipped ones. So pessimistically we can assume that $M = n$ bits are always flipped, since it maximizes the chances of this unfortunate event.

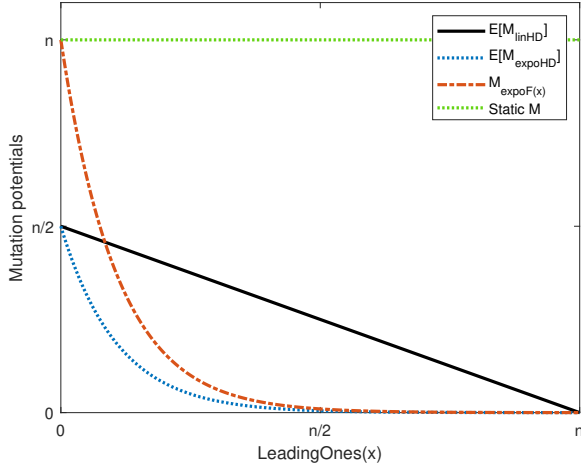


Fig. 2: Static and inversely proportional mutation potentials for LEADINGONES. Expected values are shown for those using Hamming distance.

By Chernoff bounds, the number of 0-bits in the initialised solution is at least $n/3$ w.o.p. Considering the number of 0-bits as $i = q$ and the number of 1-bits as $n - i = p$, the probability of an improvement is at most $1 - (p - q)/(p + q) = 1 - (n - 2i)/n = \frac{2i}{n}$ by the Ballot theorem, where $i = H(x, \text{opt})$. This means that we need to wait at least $\frac{n}{2i}$ iterations to see an improvement and each time the mutation operator fails to improve the fitness ($\frac{n}{2i} - 1$ times in expectation), i fitness function evaluations will be wasted. Considering that at least $n/3$ improvements are needed, the expected time to optimise ONEMAX is larger than $(1 - 2^{-\Omega(n)}) \sum_{i=1}^{n/3} (\frac{n}{2i} - 1) \cdot i = \Omega(n^2)$ since the operator cannot improve the ONEMAX function value by more than one in each application due to FCM. The factor $(1 - 2^{-\Omega(n)})$ accounts for the w.o.p. Chernoff event; since $2^{-\Omega(n)} = o(1)$, the bound holds unconditionally. $\square$

The following theorem shows that a greater speed up may be achieved if the potential decreases exponentially rather than linearly. Note that for ONEMAX the Hamming distance of a solution to the optimum and its difference in fitness are the same. Hence, there is no difference between the behaviours of the algorithm when using $M_{\text{expoF}(x)}$ and M_{expoHD} . Figure 1 highlights the reason why these operators waste fewer fitness evaluations.

Theorem 2. *The $(1 + 1)$ IA using IPH with either $M_{\text{expoF}(x)}$ or M_{expoHD} optimises ONEMAX in $\Theta(n^{3/2}/\log n)$ expected fitness function evaluations.*

B. Performance for LEADINGONES in Ideal Conditions

In the previous subsection, it was shown that decreasing the mutation rate linearly with the Hamming distance to the optimum allows for a logarithmic factor speed-up for ONEMAX compared to using static hypermutations. The following theorem shows that no asymptotic improvement over static mutation potentials is achieved for LEADINGONES: $\sum_{i=1}^n \prod_{j=1}^i x_i$. The main reason for the lack of improvement

is that a linear mutation potential is sustained until at least a linear number of improvements are achieved.

Theorem 3. *The $(1 + 1)$ IA using IPH with M_{linHD} optimises LEADINGONES in $\Theta(n^3)$ expected fitness function evaluations.*

We can now proceed with the analysis of the exponential-based potentials. Theorem 4 shows that the fitness difference based exponential mutation potential provides a $\Theta(\log^2 n)$ factor speed-up compared to static mutation potentials.

Theorem 4. *The $(1 + 1)$ IA using IPH with $M_{\text{expoF}(x)}$ optimises LEADINGONES in $\Theta(n^3/\log^2 n)$ expected fitness function evaluations.*

An advantage of Hamming distance-based exponential decays of the mutation potential compared to fitness-based ones is provided by the following theorem for LEADINGONES. The reason can be seen in Figure 2. While the initial fitness is very low, the potential is very high, but the actual number of bits that have to be flipped to reach the optimum (i.e., the Hamming distance) is much smaller. More precisely, fitness-based potentials suggest to flip n bits when only $n/2$ bits have to be flipped in expectation to reach the optimum in one step. M_{expoHD} exploits this property, thus wastes fewer fitness evaluations than $M_{\text{expoF}(x)}$.

Theorem 5. *The $(1 + 1)$ IA using IPH with M_{expoHD} optimises LEADINGONES in $\Theta(n^{5/2}/\log^2 n)$ expected fitness function evaluations.*

Overall, in this section, we have shown that considerable speed-ups can be achieved in the ideal condition where the optimum is known by using inversely proportional hypermutations compared to static ones. We conclude that M_{expoHD} provides larger hill-climbing speed-ups compared to the other mutation potentials and is stable to the scaling of fitness functions. However, M_{expoHD} only allows to flip at most $\sqrt{n}$ bits when far away from the optimum (e.g., at initialisation for ONEMAX), which means that the algorithm hardly ever performs large mutations except at initialisation defeating the purpose of having a hypermutation operator at all. Thus, we believe that M_{linHD} , while being slower during the hillclimbing phases, provides a better balance between high and low mutation rates for hard multimodal functions when the total runtime is reasonably dominated by the time required to escape from local optima. Notice from Table II that using a linear decay operator with FCM rather than in Clonalg still makes a difference between polynomial and exponential time for the functions when the optimum is known. In the following section we will highlight how being able to perform large mutations more often allows the algorithm to be effective in practice when the optimum is unknown for multimodal functions with very different characteristics.

IV. BEHAVIOUR OF IPH IN PRACTICE

In this section, we consider the usage of IPH in realistic applications where the optimum is unknown. To this end, the mutation rate of the IPH operator will be inversely proportional to the best seen solution rather than to the unknown optimum.

We combine IPH using M_{linHD} with hybrid ageing, as in the Opt-IA AIS [35], and embed it in a $(1+1)$ Opt-IA shown in Algorithms 3 and 4. To use an ageing operator, an age $\alpha(x)$, is assigned to the individual x . After the Hypermutate(x) operator, the ageing operator is applied. Among the different variants of ageing, *hybrid* ageing has been shown to be very efficient at escaping local optima [39], [40], [45], [59]. Using this operator, individuals are assigned with initial age zero. During each iteration of the algorithm, the age increases by 1 and is passed to the offspring if it does not improve over its parent's fitness. If the offspring is fitter than the parent, then its age is set to 0. At the end of each iteration, any individual with age larger than a threshold τ is removed with probability $p_{\text{die}} = 1/2$. In case there is no other individual left, a new individual is initialised uniformly at random³.

Ageing has been shown to enable algorithms to escape from local optima either by identifying a gradient leading away from it or by restarting the whole optimisation process. Our aim behind the algorithm design is that by escaping local optima via ageing, the number of local optima that are identified by the algorithm increases over time. Thus as time goes by the IPH should approximate its ideal behaviour better and better.

Concerning the IPH operator for practical applications we reintroduce the parameter $c \leq 1$ for determining the mutation potential when it grows linearly with the Hamming distance as in the original Opt-IA [42]:

$$M_{\text{linHD}} = \max(\lfloor c \cdot H(x, \text{best}) \rfloor, 1)$$

The parameter will be useful to avoid always ending up in the same basins of attraction as we will see in the analyses of both multimodal function classes we consider in this paper.

We also have a design choice regarding the IPH operator of whether to update the *best* seen solution only when a strictly better solution is identified or whether to update it when the algorithm finds a new solution that is as good as the previous best one. This design choice leads to considerably different algorithmic behaviours. If the current *best* is updated to the new solution when individuals of equal fitness are encountered, then the algorithm will be extremely fast on unimodal functions because the mutation rate will always be the smallest possible (i.e., 1-bit flips). So if the aim is to optimise easy unimodal functions then this version, which we call $(1+1)$ Opt-IA_{LASTBEST} is the preferred choice, as it can be proved to run in optimal asymptotic runtime on benchmark functions such as ONEMAX (i.e., $\Theta(n \log n)$) and LEADINGONES (i.e., $\Theta(n^2)$). However, this choice would mean that the algorithm would keep low mutation rates when trapped in the best found local optima if they are encountered again and again with high probability after the algorithm is restarted.

The other choice, that of only updating the *best* when a strictly better solution is encountered, allows individuals trapped on a large plateau of local optima to increase their Hamming distance to the best solution, thus increases the chances of escaping via higher mutation rates. This version,

³When a population is used, the required number of individuals are re-initialised to reach the population size again after ageing is applied.

Algorithm 3 $(1+1)$ Opt-IA_{LASTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ ;
3: evaluate  $f(x)$ ;
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ ;
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ ;
9:   else
10:     $\alpha(y) := \alpha(x)$ ;
11:   end if
12:   if  $f(y) \geq f(\text{best})$  then
13:     then set  $\text{best} := y$ ;
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
19:       if  $f(w) \geq f(\text{best})$  then
20:          $\text{best} := w$ 
21:       end if
22:     end if
23:   end for
24:   if  $f(y) \geq f(x)$  then
25:      $x := y$ 
26:   end if
27: end while

```

which we call $(1+1)$ Opt-IA_{FIRSTBEST}, allows to use high mutation rates more often at the expense of being slower on some unimodal functions like LEADINGONES as we will see. With this operator we remember to update the *best* to the last seen (best) individual of highest fitness (i.e. the current solution) when ageing is triggered because this limits the Hamming distance between the new solution and the current best when a local optima is escaped by accepting solutions of lower fitness, thus decreases the chances of jumping back to the previous local optimum.

We start with a general reduction. When the mutation potential remains 1 and ageing is unlikely to trigger, both algorithm variants execute single-bit flips with accept-if- $\geq$ and thus behave identically to RLS. Lemma 3 below formalises this observation; we then apply it to show that both algorithms optimise ONEMAX in $\Theta(n \log n)$ and that $(1+1)$ Opt-IA_{LASTBEST} optimises LEADINGONES in $\Theta(n^2)$.

Lemma 3 (RLS Reduction). *Let A denote the event that ageing triggers before the optimum is found and set $p := \Pr[A]$. If (i) $M = 1$ throughout every run in which A does not occur, and (ii) $p = o(1)$ and $\tau p = o(1)$, then $\mathbb{E}[T'] = (1 \pm o(1)) T_{\text{RLS}}$, where T_{RLS} is the expected runtime of RLS on the same function.*

Theorem 6. *The expected runtime of $(1+1)$ Opt-IA_{FIRSTBEST} and $(1+1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising ONEMAX is $\Theta(n \log n)$ with $\tau \geq C n \ln n$ for*

Algorithm 4 $(1 + 1)$ Opt-IA_{FIRSTBEST}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random
2: set  $\alpha(x) := 0$  and  $\text{best} := x$ 
3: evaluate  $f(x)$ 
4: while termination condition is not satisfied do
5:    $\alpha(x) := \alpha(x) + 1$ 
6:    $y := \text{Hypermutate}(x)$ 
7:   if  $f(y) > f(x)$  then
8:      $\alpha(y) := 0$ 
9:   else
10:     $\alpha(y) := \alpha(x)$ 
11:   end if
12:   if  $f(y) > f(\text{best})$  then
13:      $\text{best} := y$ 
14:   end if
15:   for  $w \in \{x, y\}$  do
16:     if  $\alpha(w) \geq \tau$  then
17:       with probability  $p_{\text{die}} = 1/2$ :
18:         if  $f(w) = f(\text{best})$  then
19:            $\text{best} := w$ 
20:         end if
21:       reinitialise  $w$  uniformly at random,  $\alpha(w) := 0$ 
22:       if  $f(w) > f(\text{best})$  then
23:          $\text{best} := w$ 
24:       end if
25:     end if
26:   end for
27:   if  $f(y) \geq f(x)$  then
28:      $x := y$ 
29:   end if
30: end while

```

a sufficiently large constant C .

Proof. Immediately after initialisation $\text{best} = x$, so $H(x, \text{best}) = 0$ and $M = c \cdot H = 0$; since the potential is at least 1 by convention, $M = 1$. On ONEMAX, every bit flip changes fitness by ± 1 , so no neutral moves exist; every accepted solution strictly improves fitness and triggers $\text{best} \leftarrow x$ in both variants, maintaining $H = 0$ and $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), at any level $k \geq 1$ the improvement probability is $k/n \geq 1/n$, so the probability of τ consecutive non-improvements is at most $(1 - 1/n)^\tau \leq e^{-\tau/n} \leq e^{-C \ln n} = n^{-C}$. A union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$ for $C > 1$, and $\tau \cdot p \leq Cn \ln n \cdot n^{1-C} = Cn^{2-C} \ln n = o(1)$ for $C > 2$. Thus both conditions of Lemma 3 hold for $C > 2$.

By Lemma 3, $\mathbb{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. Since $T_{\text{RLS}} = \sum_{k=1}^n n/k = \Theta(n \log n)$ (lower bound: at least $n/3$ zeros initially w.o.p. by Chernoff, giving $\sum_{k=1}^{n/3} n/k = \Omega(n \log n)$), the result follows. $\square$

In the following two theorems we show how $(1 + 1)$ Opt-IA_{LASTBEST} achieves a linear factor speed-up over $(1 + 1)$ Opt-IA_{FIRSTBEST} for LEADINGONES, because the latter increases

its mutation rate on plateaus of equal fitness before finding each improvement.

Theorem 7. *The expected runtime of $(1 + 1)$ Opt-IA_{LASTBEST} using IPH with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^2)$ with $\tau \geq Cn \ln n$ for a sufficiently large constant C .*

Proof. Since $(1 + 1)$ Opt-IA_{LASTBEST} accepts equal-or-better solutions and updates best on any acceptance, every accepted move (whether improving or neutral) restores $H(x, \text{best}) = 0$, so $M = 1$ throughout. Condition (i) of Lemma 3 holds.

For (ii), the improvement probability at every level is exactly $1/n$ (flip the leftmost 0-bit), so $(1 - 1/n)^\tau \leq n^{-C}$ and a union bound over n levels gives $p := \Pr[A] \leq n^{1-C} = o(1)$. Moreover $\tau \cdot p \leq Cn^{2-C} \ln n = o(1)$ for $C > 2$, verifying condition (ii).

By Lemma 3, $\mathbb{E}[T'] = (1 \pm o(1))T_{\text{RLS}}$. For RLS on LEADINGONES, the unique improving move at each of the n levels is the leftmost 0-bit flip, with probability $1/n$; neutral tail flips are accepted but do not advance the level. Hence the expected time per level is n and $T_{\text{RLS}} = n^2$, giving $\mathbb{E}[T'] = \Theta(n^2)$. $\square$

Theorem 8. *The expected runtime of the $(1 + 1)$ Opt-IA_{FIRSTBEST} with M_{linHD} and any $c \leq 1$ for optimising LEADINGONES is $\Theta(n^3)$ with $\tau = \Omega(n^{1+\epsilon})$ for any arbitrary constant $\epsilon > 0$.*

Proof. Fix a level ℓ with $n/3 \leq \ell < 2n/3$. For the current individual x with $\text{LO}(x) = \ell$, we have $x_1 = \dots = x_\ell = 1$ and $x_{\ell+1} = 0$, and the remaining $m := n - \ell - 1$ positions form the tail. In each iteration, the first flipped bit is uniformly distributed over $\{1, \dots, n\}$; therefore the probability that the first flip hits the improving bit $\ell + 1$ is $1/n$, the probability that it hits the prefix is ℓ/n , and the probability that it hits the tail is m/n . Let T be the iteration on which the improving bit is first hit. Then T is geometrically distributed with parameter $1/n$, so $\mathbb{E}[T] = n$. Before this improving iteration occurs, the expected number of tail-first iterations is $\frac{m}{n-1}(\mathbb{E}[T] - 1) = \frac{m(n-1)}{n-1} = m = \Theta(n)$ (using that, conditional on not hitting the improving bit $\ell + 1$, the first flip is uniform over the remaining $n - 1$ positions, of which m are tail positions). Each such iteration flips a uniformly random tail bit. By standard coupon-collector arguments, after $\Theta(n)$ such tail flips a constant fraction of the m tail positions has been flipped exactly once with constant probability (since each hypermutation samples bits without replacement, each position is visited at most once per call). Thus, with constant probability, the Hamming distance between x and best is $\Theta(m) = \Theta(n)$, and therefore the IPH mutation potential satisfies $M = \Theta(n)$ at this level.

Consider now a subsequent iteration at the same level while $M \geq \alpha n$ for some constant $\alpha > 0$. If the first flip hits the prefix (an event of probability at least $\ell/n \geq 1/3$), then, because mutation proceeds without replacement, the lost prefix bit cannot be repaired within the same iteration. Consequently, no acceptance can occur in that iteration, and the algorithm executes the entire mutation budget M , rejecting at the end. Hence the expected number of fitness evaluations in such an

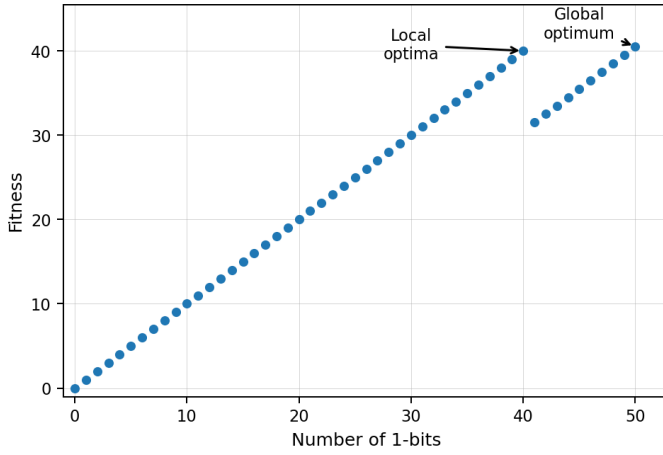


Fig. 3: The CLIFF_d function for $n = 50$, $d = 10$. The local optimum (40 one-bits, fitness 40) and global optimum (50 one-bits, fitness 40.5) are labelled.

iteration is at least αn . Since this happens with probability at least $1/3$, the unconditional expected cost of one iteration while $M = \Theta(n)$ is $\Omega(n)$.

The probability that an iteration yields an improvement is exactly $1/n$, because only a first flip that hits position $\ell + 1$ increases the LEADINGONES value. Therefore the expected number of iterations required to advance from level ℓ to $\ell + 1$ is $1/(1/n) = n$. Combining this with the previous observation, the expected number of fitness evaluations spent at level ℓ satisfies $\Omega(n) \cdot n = \Omega(n^2)$. There are $\Theta(n)$ levels ℓ in the interval $[n/3, 2n/3]$, and each such level contributes $\Omega(n^2)$ expected evaluations. Thus the total expected optimisation time is $\Omega(n^3)$. Finally, stochastic ageing can only increase the number of evaluations and therefore cannot invalidate the lower bound. Since the function has n levels, $1/n$ probability of improvement and $M \leq n$, the expected runtime is bounded from above by $O(n^3)$ as well. $\square$

In this section, we have proven the results for M_{linHD} . Naturally the same [positive](#) results follow for the M_{expoHD} operator which always has at most the same mutation rate or lower.

V. WHEN IPH OUTPERFORMS TOO HIGH MUTATION RATES

In this section we show that M_{linHD} combined with ageing can efficiently escape from local optima. We will use the well known CLIFF_d function for this purpose where static hypermutations are inefficient [39]:

$$\text{CLIFF}_d(x) := \begin{cases} \sum_{i=1}^n x_i & \text{if } \sum_{i=1}^n x_i \leq n - d, \\ \sum_{i=1}^n x_i - d + 1/2 & \text{otherwise.} \end{cases} \quad (1)$$

The class of CLIFF_d functions, illustrated in Figure 3, is usually used to assess the performance of non-elitist algorithms. It was originally designed as a problem instance where non-elitist EAs outperform elitist EAs [44]. This function has a ONEMAX slope with length $n - d$ which leads the algorithms

towards the local optima. The local optima are followed by a second ONEMAX slope (of length d) of lower fitness which leads to the global optimum. While elitist algorithms need to make a jump of size d to find the global optimum, [some](#) non-elitist algorithms can easily find the second slope by accepting inferior solutions, and then hillclimb up to the global optimum [3], [24], [39], [60], [61]. Due to the high mutation rates of static hypermutations, even if they were to escape the local optima of CLIFF_d (which is very unlikely due to FCM and linear mutation potentials), they would still jump back with high probability.

Recently [fast AISs](#) have been introduced [3] that simply flip the required number of bits independently according to the mutation potential (rather than sequentially and using FCM) in a similar fashion to power law mutation operators [4]. These have been shown to optimise CLIFF_d as long as the gap size is linear in expected time $O(n^{5/2} \log n)$.

Now we show that differently from static hypermutations, M_{linHD} combined with ageing can escape from the local optima of CLIFF_d [also for small cliff gaps](#), and [faster than the fast AISs for sublinear gaps](#), hence optimises the function efficiently. The condition that d is not prohibitively large is necessary to avoid that the reinitialised solutions due to ageing have larger fitness than a solution with $n - d + 1$ 1-bits, which would cause the latter to be removed from the population. We believe that this assumption is realistic for practical applications.

The following theorem is the main result of this section.

Theorem 9. *The $(1 + 1)$ $\text{Opt-IA}_{\text{FIRSTBEST}}$ with M_{linHD} (factor $c < \frac{1+4\epsilon}{3-4\epsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon'})$ optimises CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected*

$$O\left(\frac{n^5 \log(n/d)}{d^3} + n^2 \log d\right)$$

fitness function evaluations.

We now present the proof sketch of our main result on the CLIFF_d function, together with all supporting lemmata. All proofs are deferred to the supplementary materials. The argument proceeds in two major stages: (i) obtaining a canonical entry point onto the second slope immediately after an ageing event at the cliff-top; and (ii) showing that from this configuration the algorithm reaches the optimum without dangerous increases in mutation potential.

For stage (ii) we will use the generalization of Markov's Inequality by Ville [62](Ch. VII) which will allow us to show that the mutation potential stays bounded with constant probability.

Theorem 10 (Ville's inequality for nonnegative supermartingales). *Let $(M_t)_{t \geq 0}$ be a nonnegative supermartingale adapted to $(\mathcal{F}_t)_{t \geq 0}$. Then for every $a > 0$,*

$$\Pr\left(\sup_{t \geq 0} M_t \geq a\right) \leq \frac{E[M_0]}{a}.$$

To establish the result, we first analyse how the algorithm reaches a highly structured “canonical” state shortly after ageing triggers at the cliff-top. At the cliff-top the algorithm

cannot improve except by sampling the global optimum, and so ageing will eventually be triggered. The key insight is that the iteration of ageing followed by two additional fitness improvements can place the algorithm into a **canonical starting configuration on the second slope**. Throughout the **CLIFF_d** analysis we index the successful fitness improvements on the second slope by $r = r_0, r_0 + 1, \dots$, and write $\text{HD}(r)$ for the Hamming distance between the current solution and *best* after r such improvements. With this notation, the canonical start corresponds to $r_0 = 2$ (two improvements completed from the cliff bottom) and $\text{HD}(r_0) = 2$ (the current solution differs from the locked *best* in exactly two positions). The following lemma formalises this.

Lemma 4. *Suppose ageing is triggered at the cliff-top (d zero-bits). The **(1 + 1) Opt-IA_{FIRSTBEST}** with M_{linHD} with $c \in (0, 1)$ reaches within at most three steps the configuration*

$$r_0 = 2, \quad \text{HD}(r_0) = 2,$$

with probability at least

$$\frac{3}{256} \left(\frac{d}{n} \right)^3.$$

The canonical condition $(r_0, \text{HD}(r_0)) = (2, 2)$ means that we begin the drift analysis two successful improvements into the second slope, at a point differing from the reference in exactly two bit positions.

Repeated ascents to the cliff-top therefore form a geometric waiting-time process. It takes at most $\sum_{i=d}^n n/i = O(n \log(n/d))$ for **RLS** to climb the first slope and the mutation potential is at most linear. Since each ascent takes $O(n^2 \log(n/d))$, the total time to obtain the canonical start is provided by the following corollary.

Corollary 1. *The expected evaluation count until the first occurrence of $(r_0, \text{HD}(r_0)) = (2, 2)$ is*

$$O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

From the canonical starting point, the key difficulty is showing that the mutation potential does not increase too much. With only few zero-bits remaining, successful steps on the second slope require flipping one of these rare zeros, whereas each $1 \rightarrow 0$ flip increases the Hamming distance and thus the mutation potential. If this potential grows too large, hypermutations become long enough that, under **FCM**($\geq$), they are increasingly likely to wander back to the cliff-top; once there, the stopping rule causes the process to lock onto the plateau and erase all progress. Hence the analysis must show that with constant probability the mutation potential stays small throughout the entire second-slope climb. This requires controlling tightly how many detrimental flips a hypermutation can accumulate before recovering to parent-level fitness. The next lemma provides the crucial starting point: an exponential tail bound for this damaging prefix.

Lemma 5. *Let the current solution on the second slope have $z \leq d$ zero-bits and define $p = z/n$. Consider a single application of the hypermutation operator; let ξ_z be the*

number of $1 \rightarrow 0$ flips made within that one hypermutation before the fitness first returns to $\geq$ the parent's fitness (i.e., before the first catch-up). Then for all $k \geq 1$,

$$\Pr(\xi_z \geq k) \leq (2p)^k.$$

Consequently, the probability that the Hamming distance increases by at least $2k$ in a single hypermutation is also at most $(2p)^k$.

A whole phase consists of all attempts at the same zero-bit count until the first success. Using the tail bound above, we obtain uniform exponential moment bounds for entire phases.

Lemma 6. *Consider a phase at zero-bit count z and let Y_z denote the total Hamming-distance increase from all failed attempts in the phase. Let $\rho = 2d/n$. For all $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$,*

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda).$$

To track the safety of the mutation potential across phases, we introduce a linear *buffer* $B_r = \theta r - s_0 - E_r$ capturing the invariant $c \text{HD}(r) \leq r$.

Lemma 7. *Let successful phases be indexed by $r = r_0, r_0 + 1, \dots$ and define*

$$s_0 = \text{HD}(r_0) - r_0, \quad \theta = \frac{1}{c} - 1, \quad E_r = \sum_{j=r_0+1}^r Y_{z_j},$$

and $B_r = \theta r - s_0 - E_r$. Then for all $r \geq r_0$:

- 1) $B_r \geq 0$ if and only if $c \cdot \text{HD}(r) \leq r$;
- 2) $B_{r+1} - B_r = \theta - Y_{z_{r+1}}$;
- 3) $B_{r_0} = \theta r_0 - s_0$.

Combining the **MGF of the buffer growth** with the buffer update yields an exponential drift envelope suitable for applying Ville's inequality for non-negative supermartingales (Theorem 10).

Lemma 8. *Let $\rho = 2d/n$. For any $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ define*

$$\frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda), \quad \alpha = e^{-\lambda \theta} K(\lambda).$$

Then for all $t \geq 0$,

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq \alpha.$$

We now apply Ville's inequality to turn such a drift envelope into a bound on barrier crossing.

Corollary 2. *If $B_0 = b > 0$ and there exist $\lambda > 0$ and $\alpha < 1$ with*

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq \alpha,$$

then

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda b}.$$

To guarantee the drift negativity required above, we identify the admissible range of the mutation-potential factor c .

Lemma 9. *Let $\rho = 2d/n$ and $\theta = \frac{1}{c} - 1$. Define*

$$c_*(n, d) = \frac{1 - \rho}{1 + \rho}.$$

If $c < c_*(n, d)$, then there exists $\lambda > 0$ such that

$$e^{-\lambda\theta} K(\lambda) < 1,$$

and hence the buffer satisfies the Ville barrier bound.

A uniform instantiation over all permitted d shows that the feasible c range has a nontrivial lower bound.

Corollary 3. If $d \leq (\frac{1}{4} - \varepsilon)n$, then

$$c_*(n, d) \geq \frac{1 + 4\varepsilon}{3 - 4\varepsilon} \geq \frac{1}{3}.$$

Finally, combining the threshold on c with the initial buffer value at $(r_0, \text{HD}(r_0)) = (2, 2)$ yields a uniform constant bound on the probability of avoiding jumping back.

Lemma 10. If $c < c_*(n, d)$ and $b = B_{r_0} = \frac{r_0}{c} - \text{HD}(r_0) > 0$, then

$$\Pr(\inf B_t \leq 0) \leq e^{-\lambda b}$$

for some $\lambda > 0$ depending only on c and d/n . For fixed ε , this yields a uniform constant < 1 independent of n, d .

Once the mutation potential remains safe throughout, progress on the second slope is monotone and its runtime is easily bounded.

Lemma 11. If $B_t > 0$ for all t , then the second-slope ascent completes in expected

$$\Theta(n \log d)$$

iterations. If $\tau = \Omega(n^{1+\varepsilon})$, then $\Theta(n \log d) = o(\tau)$.

The expected time to reach the canonical start is $O(n^5 \log(n/d)/d^3)$. Once there, with constant probability the buffer remains positive for all phases; conditioned on this event, the second-slope ascent costs $O(n^2 \log d)$ evaluations. Since the success probability is constant, this remains the unconditional expectation. Summing the two contributions establishes the main runtime bound.

Proof [of Theorem 9.]

From a uniformly random start, the expected iterations to reach the cliff-top ($n - d$ ones) are $\sum_{i=d}^n \frac{n}{i} = \Theta(n \log \frac{n}{d})$, which at $\Theta(n)$ evaluations/iteration costs $\Theta(n^2 \log \frac{n}{d})$ evaluations per ascent to the cliff-top. At the cliff-top, the only fitter point is 1^n , so ageing triggers unless the optimum is found within τ iterations.

By Lemma 4, within at most three iterations after an ageing kick the process reaches the canonical second-slope start

$$r_0 = 2, \quad \text{HD}(r_0) = 2$$

with probability

$$p_{\text{can}}(n, d) \geq \frac{3}{256} \left(\frac{d}{n}\right)^3.$$

Hence the number of ageing epochs needed to obtain this start is geometric with mean $1/p_{\text{can}}$, and the expected evaluation cost up to the canonical start (including restarts) is

$$\Theta\left(\frac{1}{p_{\text{can}}(n, d)} \cdot n^2 \log \frac{n}{d}\right) = O\left(\frac{n^5 \log(n/d)}{d^3}\right).$$

Under the reference-update rule the start on the second slope is $r_0 = 2$, $\text{HD}(r_0) = 2$, so $s_0 = 0$ and with $\theta = \frac{1}{c} - 1$ define

$$B_r := \theta r - \sum_{j=r_0+1}^r Y_{z_j} \quad (r \geq r_0),$$

so $B_{r_0} = 2\theta$ and $B_{t+1} - B_t = \theta - Y_{z_{t+1}}$ by Lemma 7. By the refined phase MGF (Lemma 6), for every $0 < \lambda < \frac{1}{2} \ln \frac{1}{2\rho}$ with $\rho := 2d/n$,

$$\mathbb{E}[e^{\lambda Y_z}] \leq \frac{1 - \rho}{1 - \rho e^{2\lambda}} =: K(\lambda).$$

Therefore, by Lemma 8,

$$\mathbb{E}[e^{-\lambda(B_{t+1} - B_t)} \mid \mathcal{F}_t] \leq e^{-\lambda\theta} K(\lambda) \quad \text{for all } t \geq 0.$$

Since $d \leq (\frac{1}{4} - \varepsilon)n$ and the theorem assumes

$$c < \frac{1 + 4\varepsilon}{3 - 4\varepsilon} \leq \frac{1 - 2d/n}{1 + 2d/n} =: c_*(n, d),$$

Lemma 9 guarantees an admissible $\lambda \in (0, \frac{1}{2} \ln \frac{n}{4d})$ with $e^{-\lambda\theta} K(\lambda) < 1$. With $B_{r_0} = 2\theta$, Corollary 2 yields

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda 2\theta}.$$

Moreover, since $\frac{1}{2} \ln \frac{n}{4d} \geq \frac{1}{2} \ln \frac{1}{1-4\varepsilon}$ for all admissible d , we may fix a uniform $\lambda_\varepsilon \in (0, \frac{1}{2} \ln \frac{1}{1-4\varepsilon})$ and conclude

$$\Pr\left(\inf_{t \geq 0} B_t \leq 0\right) \leq e^{-\lambda_\varepsilon 2\theta}.$$

Because λ_ε depends only on ε and $\theta = \frac{1}{c} - 1$ depends only on c , this right-hand side is a constant in $(0, 1)$, independent of n and d .

On the event $\{\inf_{t \geq 0} B_t > 0\}$ (no jump-back), the second-slope ascent is monotone and completes in expected

$$\sum_{z=1}^d \frac{n}{z} = \Theta(n \log d)$$

iterations, which is $o(\tau)$ since $\tau = \Omega(n^{1+\varepsilon})$. Since the mutation potential satisfies $M \leq cn$ (where c is the constant from the M_{linHD} potential), each iteration evaluates at most $cn = \Theta(n)$ individuals; one second-slope attempt therefore costs $\Theta(n^2 \log d)$ evaluations, and the constant no-jump-back probability implies a constant expected number of such attempts.

Summing the (restarted) approach to the canonical start and the second-slope cost gives

$$\mathbb{E}[\text{evaluations}] = O\left(\frac{n^5 \log(n/d)}{d^3}\right) + O(n^2 \log d),$$

as claimed. $\square$

The analysis for $(1 + 1)$ Opt-IA_{LASTBEST} differs only in Lemma 4 and actually yields a d/n factor better probability bound since the mutation potential is 1 at the cliff-top and thus, creating a solution at the bottom of the cliff is immediately possible when any cliff top individual dies, whereas for $(1 + 1)$ Opt-IA_{FIRSTBEST} to work we have to first accept a neutral mutation to get the mutation potential

to 1, and then mutate into a cliff bottom solution in the next iteration. Consequently the runtime for $(1 + 1)$ Opt-IA_{LASTBEST} is $O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$.

Corollary 4. *The $(1 + 1)$ Opt-IA_{LASTBEST} with M_{linHD} (factor $c < \frac{1+4\epsilon}{3-4\epsilon}$) and ageing parameter $\tau = \Omega(n^{1+\epsilon'})$ optimises CLIFF_d with $d < n(\frac{1}{4} - \epsilon)$ in expected*

$$O\left(\frac{n^4 \log(n/d)}{d^2} + n^2 \log d\right)$$

fitness function evaluations.

The following section will highlight the advantages of using FIRST-BEST over using LAST-BEST.

VI. WHEN IPH OUTPERFORMS TOO LOW MUTATION RATES

In this section we introduce a function class where inversely proportional hypermutations outperform the lower mutation rates of SBM, and the higher mutation rate on local optima of the $(1 + 1)$ Opt-IA_{FIRSTBEST} will be essential. The function class is inspired by the one previously used to show the advantages and disadvantages of using high mutation rates in the context of rank-based mutation in population-based algorithms [48]. In rank-based mutation algorithms [63] each individual has a different mutation rate based on their rank in the population i.e., the best individual has the lowest mutation rate while the worst individual has a very high one. The LEADINGTRAPJUMP function was introduced as an example where the use of high mutation rates is detrimental while low mutation rates are efficient. However, if the locations of the local and global optima are swapped, then the opposite result can be shown (i.e. high mutation rates are effective while low ones are ineffective). We have slightly modified the location of the trap compared to [48] for our objectives.

Formally, for constants $k \geq 1$ and $\rho \in (0.8, 0.9)$, let $r := 2k + 1$, $L := \lfloor \log_2 n \rfloor$, $\mathcal{P} := \{x : \text{LO}(x) \geq 0.9n\}$, $\mathcal{B}_k := \{0^r 1^{(0.9n-r)*}\}$, and $\mathcal{D}_\rho := \{0^L * : |x|_1 \geq \rho n\}$. The function $\text{LTJ}_{k,\rho} : \{0, 1\}^n \rightarrow \mathbb{R}$ is

$$\text{LTJ}_{k,\rho}(x) = \begin{cases} 0, & x \in \mathcal{P}, \\ \text{LO}(x_{i>r}) + r, & x \in \mathcal{B}_k, \\ n + 1, & x \in \mathcal{D}_\rho, \\ \text{LO}(x), & \text{otherwise.} \end{cases}$$

Here $x_{i>r}$ denotes the suffix of x at positions $i > r$, i.e., $x_{i>r} := (x_{r+1}, x_{r+2}, \dots, x_n)$. Note that $r := 2k + 1$ is an odd number, as required for the block-jump prefix $0^r 1^{(0.9n-r)*}$ to force an even number of bit-flips to reach $\mathcal{B}_k$, which is the key parameter controlling the per-iteration success probability $\Theta(n^{-(r+1)}) = \Theta(n^{-(2k+2)})$. We introduce r as shorthand to avoid writing $2k + 1$ in every exponent throughout the proof.

The landscape consists of a long LeadingOnes slope rising from fitness $\text{LO}(x)$ up to level $0.9n - 1$, followed by a large penalised region. Two off-slope structures contain the two sets of optima but are inaccessible to standard hill-climbers. The set $\mathcal{B}_k$ begins with the prefix $0^r 1^{(0.9n-r)*}$ and forms a short

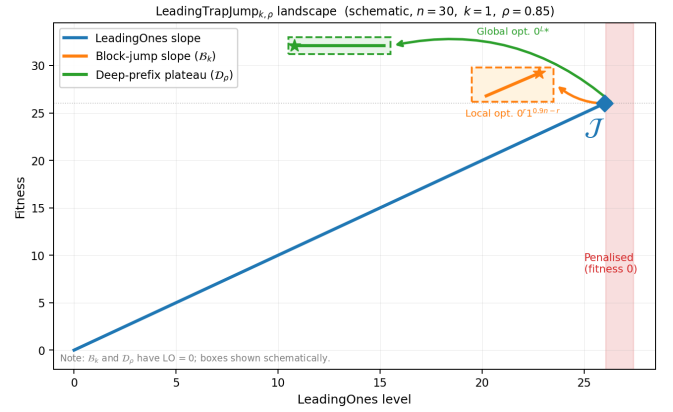


Fig. 4: The $\text{LEADINGTRAPJUMP}_{k,\rho}$ landscape (schematic, $n = 30$, $k = 1$, $\rho = 0.85$). The x-axis is the LEADINGONES level of the current solution. From the junction J (end of the main slope), large mutations can reach either the block-jump slope $\mathcal{B}_k$ (leading to the local optimum) or the deep-prefix plateau $\mathcal{D}_\rho$ (global optimum). Both off-slope structures have $\text{LO}(x) = 0$; their positions on the y-axis reflect their fitness values, not their LEADINGONES level.

slope, called block-jump slope, that deterministically increases toward its unique local optimum $0^r 1^{(0.9n-r)*}$; reaching this prefix is therefore equivalent to entering the leading ones path to the local optimum $0^r 1^{(0.9n-r)*}$. In contrast, the set $\mathcal{D}_\rho$ consists of solutions with prefix $0^L *$ whose suffix has ones-density at least ρ i.e., ρn ones, and forms a wide plateau of globally optimal fitness, which we call deep-prefix plateau. Both structures are hidden behind the penalised barrier (illustrated in Figure 4), and the only reliable way to reach them is via large hypermutation events. The junction $J := \{x : \text{LO}(x) = 0.9n - 1\}$ acts as a central gateway: each ageing epoch that restarts the search will, with high probability, ascend the LeadingOnes slope back to J before any off-slope structure is encountered. From J , the $(1+1)$ Opt-IA using IPH with M_{linHD} can perform jumps of the appropriate magnitude to reach either the entrance of the block-jump slope $\mathcal{B}_k$ or the plateau $\mathcal{D}_\rho$ i.e., via ageing restarts it will eventually find both optima efficiently. On the other hand, algorithms with lower mutation rates, such as those using SBM will with high probability not be able to flip the logarithmic number of bits required to reach the global optimum, thus have a runtime of $n^{\Omega(\log n)}$ with high probability.

Theorem 11. *The $(1 + 1)$ Opt-IA_{FIRSTBEST} with ageing threshold $\tau = \Omega(n^{1+\epsilon})$ using M_{linHD} with $c < 5/8$ identifies both local and global optima of $\text{LTJ}_{k,\rho}$ in $O(n\tau + n^{2k+3} + n^{\beta+1})$ fitness function evaluations where $\beta := \log_2 \frac{20}{c}$.*

To establish Theorem 11 we rely on three ingredients. First, every ageing epoch returns to the junction J with high probability. Second, an epoch beginning at J performs sufficiently long jumps to reach the beginning of the block-jump slope with probability $\Theta(n^{-(2k+2)})$ per iteration. Third, an epoch beginning at J performs a logarithmic jump onto the deep-prefix plateau with probability $\Theta(n^{-\beta})$ per iteration.

Since each event corresponds to reaching either the start of a short slope leading deterministically to its optimum (for $\mathcal{B}_k$) or directly a plateau of global optima (for $\mathcal{D}_\rho$), these per-epoch probabilities govern the entire runtime.

We first need to bound from below the probability that the algorithm visits the junction point after restarts in order to later argue that from the junction point both sets of optima will be accessed.

Lemma 12. *Starting from any random individual, the $(1 + 1)$ Opt-IA_{FIRSTBEST} using IPH with M_{linHD} and $c < 5/8$ reaches the junction $J = \{x : \text{LO}(x) = 0.9n - 1\}$ before accepting any individual in $\mathcal{B}_k$ or $\mathcal{D}_\rho$ with high probability.*

Within n^2 iterations, the algorithm performs at least $0.9n$ LeadingOnes improvements with high probability. Before reaching J , deep-prefix entrances require flipping the L -bit prefix to 0^L , an event whose probability per iteration is at most $(M(x)/n)^L$. Since under M_{linHD} the mutation potential satisfies $M(x) \leq q^*n$ for some constant $q^* < 1$ (determined solely by c and ρ), this becomes $(q^*)^L = n^{-\alpha^*}$ for $\alpha^* := \log_2(1/q^*) > 2$, yielding a total probability of $o(1)$ over n^2 iterations. Thus the slope is climbed to J with high probability.

The following two lemmata will condition throughout on the event that during an ageing epoch the junction J is reached. This event happens with high probability by Lemma 12.

Lemma 13. *Let an ageing epoch reach the junction J . Then, with probability $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$, the epoch performs a jump placing the individual at the beginning of the block-jump slope.*

The suffix of the individual at J is essentially random relative to the best individual stored from the previous epoch. With high probability their Hamming distance is at least $n/20$, guaranteeing $M(x) = cn/20$ throughout the epoch. Under this event, the probability that the first $r + 1$ flips of the hypermutation hit exactly the positions defining the prefix $0^r 1^{(0.9n-r)*}$ is $\Theta(n^{-(r+1)})$. Evaluating this over τ iterations yields the stated bound.

Lemma 14. *Let an ageing epoch reach the junction J . Then, with probability $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, the epoch performs a jump placing the individual onto the deep-prefix plateau.*

At J , the ones-density of the suffix already exceeds ρ with high probability. Thus it suffices to flip the prefix to 0^L . Under the same high-Hamming-distance event ensuring $M(x) \geq cn/20$, the probability that the first L flips hit positions $\{1, \dots, L\}$ is at least $(c/20)^L = n^{-\beta}$. Evaluating over τ iterations gives the lemma.

Each ageing epoch begins with a uniformly random reinitialisation and by Lemma 12 it ascends to the junction J with high probability before reaching any off-slope structure. We henceforth condition on epochs reaching J . Each ageing epoch evaluates $\Theta(n\tau)$ individuals. By Lemma 12, epochs reach J with high probability, and by lemmata 13 and 14, an epoch discovers each off-slope structure with probabilities p_B and p_D , respectively. Since B (entering $\mathcal{B}_k$) and D (entering $\mathcal{D}_\rho$) are mutually exclusive within a single epoch, the algorithm

must discover them in *different* epochs. By a coupon-collector argument (we need to “collect” both coupons B and D , which arrive independently across epochs with probabilities p_B and p_D respectively), the expected number of epochs until both optima have been seen is $O(1/p_B + 1/p_D)$. Multiplying by the cost per epoch yields the bound stated in Theorem 11.

Proof [of Theorem 11.]

By Lemma 12 we reach the junction J with probability $1 - o(1)$. Starting an epoch in the junction, the two lemmata give $p_B = \Omega(\min\{1, \tau n^{-(2k+2)}\})$ and $p_D = \Omega(\min\{1, \tau n^{-\beta}\})$, and B and D are mutually exclusive within one epoch. Thus the expected number of epochs to sample from both sets of optima is:

$$O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right).$$

Multiplying by the cost of each epoch we get,

$$\begin{aligned} \mathbb{E}[\text{evals}] &\leq \Theta(n\tau) O\left(\max\{1, n^{2k+2}/\tau\} + \max\{1, n^\beta/\tau\}\right) \\ &= O(n\tau + n^{2k+3} + n^{\beta+1}). \end{aligned}$$

□

The static hypermutation would yield a same upper bound with a similar analysis except for the value of β which would be smaller and for the range of c which would be more restricted. Both modifications are necessary due to the larger mutation size of the static hypermutation compared to IPH for a given c value. The $(1 + 1)$ Opt-IA_{LASTBEST} IPH cannot make the jumps from the junction since any current junction solution would be the last best seen, yielding a mutation potential of one 1. $\mathcal{B}_k$ and $\mathcal{D}_\rho$ sets cannot be accessed from the junction via mutation but can still be sampled at any reinitialization with $2^{-\Omega(n)}$ probability. We conjecture that power-law mutation operators such as those employed by the Fast-IA [3] and the Fast-EA [4] would also succeed on $\text{LTJ}_{k,\rho}$ if equipped with an ageing operator such as the algorithms introduced in [3], and the same should hold for the Rank-GA [63]. However, we believe that the Rank-GA with ageing would fail on CLIFF_d because of its high mutation rate at the bottom of the second slope. On the other hand, the Fast-IA and the Fast-EA, which have been shown to be parameterisable such that they behave equivalently, can quite efficiently (a $\sqrt{n}$ factor slower than the $(1 + 1)$ Opt-IA_{FIRSTBEST} and the $(1 + 1)$ Opt-IA_{LASTBEST} IPH i.e., in expected time $O(n^{5/2} \log n)$) optimise CLIFF_d functions with gaps of linear size [3]. However, they have not been shown to be efficient for CLIFF_d functions with sublinear gap sizes. Stagnation detection algorithms [64]–[66] may also be efficient for $\text{LTJ}_{k,\rho}$ if equipped with the ageing operator. It is not obvious, though, how the operator would be incorporated in such an algorithm. Thus, to date the $(1 + 1)$ Opt-IA_{FIRSTBEST} IPH is the best known algorithm to optimise both Cliff-like functions while at the same time optimise functions where large mutations are necessary to escape from local optima. An open question is whether it may be possible to avoid the use of ageing altogether for $\text{LTJ}_{k,\rho}$ by using the recently introduced population-based EAs with low selective pressure [67], [68]. Low selective pressure may

avoid the entire population converging on one of the two local optima shortly after it is identified.

VII. CONCLUSION

We have presented a thorough analysis of Inversely Proportional Hypermutations (IPH). Previous theoretical studies have shown disappointing results concerning the IPH operators from the literature. In this paper we have shown their effectiveness in isolation for unimodal functions compared to static hypermutations in the ideal conditions when the optimum is known. Furthermore, we have provided an analysis of the operator embedded in the complete Opt-IA AIS to be used in practical applications where the optimum is usually unknown. We have proved its effectiveness for two well-studied multimodal functions with considerably different characteristics showing how it harmonises together with the AIS operator better than static hypermutations. In particular the $(1+1)$ Opt-IA_{FIRSTBEST} can escape local optima in three different ways: by mutating into far away solutions (see LEADINGTRAPJUMP), by accepting solutions of inferior quality (see CLIFF) and naturally by automatically reinitialising the trapped solutions via ageing as understood regarding ageing for a long time [45], [47], [69], [70]. In particular, compared to the conference version of this paper, we have now proposed an Opt-IA algorithm with IPH that has the capability to escape from local optima via large mutations, something that was not possible with the previous version since the mutation rate remained lowest on the local optima.

Although our analysis used standard benchmark functions capturing specific structures that may appear in combinatorial optimisation problems, we point out that the Opt-IA AIS with our proposed IPH can be shown to efficiently find arbitrarily good approximations to the NP-Hard Number Partitioning problem by following the same proof strategy of [41] that uses that local optima are escaped from thanks the ageing operator that simply restarts the algorithm once trapped on a local optimum. Concerning escaping local optima via hypermutation, IPH would have an insufficient mutation potential to find an approximation in the first run until reinitialisation. However establishing whether after restarts due to ageing triggering on local optima occur, subsequent runs with higher mutation potential can approximate arbitrarily well without relying on a good starting position require further analysis of the problem landscape.

Future work should evaluate the performance of the proposed algorithms and inversely proportional mutation operators both theoretically and experimentally for other combinatorial optimisation problems with practical applications.

REFERENCES

- [1] T. Friedrich, F. Quinzan, and M. Wagner, "Escaping large deceptive basins of attraction with heavy-tailed mutation operators," in *Proc. of GECCO 2018*, 2018, pp. 293–300.
- [2] T. Friedrich, A. Göbel, F. Quinzan, and M. Wagner, "Heavy-tailed mutation operators in single-objective combinatorial optimization," in *Proc. of PPSN XV*, 2018, pp. 134–145.
- [3] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast immune system inspired hypermutation operators for combinatorial optimisation," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 5, pp. 956–970, 2021.
- [4] B. Doerr, H. P. Le, R. Makhmara, and T. D. Nguyen, "Fast genetic algorithms," in *Proc. of GECCO 2017*, 2017, pp. 777–784.
- [5] D. Corus and P. S. Oliveto, "Standard steady state genetic algorithms can hillclimb faster than mutation-only evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 720–732, 2018.
- [6] D.-C. Dang, T. Friedrich, T. Kötzing, M. S. Krejca, P. K. Lehre, P. S. Oliveto, D. Sudholt, and A. M. Sutton, "Escaping local optima using crossover with emergent diversity," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 3, pp. 484–497, 2018.
- [7] D. Corus and P. S. Oliveto, "On the benefits of populations on the exploitation speed of standard steady-state genetic algorithms," in *Proc. of GECCO 2019*, 2019, pp. 1452–1460.
- [8] P. S. Oliveto, D. Sudholt, and C. Witt, "A tight lower bound on the expected runtime of standard steady state genetic algorithms," in *Proc. of GECCO 2020*, 2020, pp. 1323–1331.
- [9] B. Doerr, C. Doerr, and F. Ebel, "From black-box complexity to designing new genetic algorithms," *Theoretical Computer Science*, vol. 567, pp. 87–104, 2015.
- [10] J. Lengler, "A general dichotomy of evolutionary algorithms on monotone functions," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 6, pp. 995–1009, 2019.
- [11] B. Doerr and C. Doerr, "Theory of parameter control for discrete black-box optimization: Provable performance gains through dynamic parameter choices," in *Theory of Evolutionary Computation - Recent Developments in Discrete Optimization*, B. Doerr and F. Neumann, Eds. Springer, 2020, pp. 271–321.
- [12] B. Doerr, A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "On the runtime analysis of selection hyper-heuristics with adaptive learning periods," in *Proc. of GECCO 2018*, 2018, pp. 1015–1022.
- [13] B. Doerr, C. Doerr, and J. Yang, "k-bit mutation with self-adjusting k outperforms standard bit mutation," in *Proc. of PPSN'16*. Springer, 2016, pp. 824–834.
- [14] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "Simple Hyper-Heuristics Control the Neighbourhood Size of Randomised Local Search Optimally for LeadingOnes," *Evolutionary Computation*, vol. 28, no. 3, pp. 437–461, 09 2020.
- [15] —, "How the duration of the learning period affects the performance of random gradient selection hyper-heuristics," in *Proc. of AAAI-20*, 2020, pp. 2376–2383.
- [16] P. S. Oliveto, T. Paixão, J. Pérez Heredia, D. Sudholt, and B. Trubenová, "How to escape local optima in black box optimisation: when non-elitism outperforms elitism," *Algorithmica*, vol. 80, pp. 1604–1633, 2018.
- [17] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "When move acceptance selection hyper-heuristics outperform metropolis and elitist evolutionary algorithms and when not," *Artificial Intelligence*, vol. 314, p. 103804, 2023.
- [18] D.-C. Dang, A. V. Eremeev, and P. K. Lehre, "Escaping local optima with non-elitist evolutionary algorithms," in *Proc. of AAAI 2021*, 2021.
- [19] B. Doerr, "Does comma selection help to cope with local optima?" in *Proc. of GECCO '20*. ACM, 2020, p. 1304–1313.
- [20] B. Case and P. K. Lehre, "Self-adaptation in non-elitist evolutionary algorithms on discrete problems with unknown structure," *IEEE Transactions on Evolutionary Computation (To appear - arXiv:2004.00327)*, vol. 24, no. 4, pp. 650–663, 2020.
- [21] D.-C. Dang, T. Jansen, and P. K. Lehre, "Populations can be essential in dynamic optimisation," in *Proc. of GECCO 2015*, 2015, pp. 1407–1414.
- [22] D.-C. Dang and P. K. Lehre, "Efficient optimisation of noisy fitness functions with population-based evolutionary algorithms," in *Proc. of FOGA 2015*, 2015, pp. 62–68.
- [23] J. Lengler, L. Schiller, and O. Sieberling, "Plus strategies are exponentially slower for planted optima of random height," in *Proc. of GECCO 2024*, 2024, pp. 1587–1595.
- [24] M. A. Hevia-Fajardo and D. Sudholt, "Self-adjusting population sizes for non-elitist evolutionary algorithms: Why success rates matter," *Algorithmica*, vol. 86, no. 2, pp. 526–565, 2024.
- [25] L. R. de Castro and J. Timmis, *Artificial Immune Systems: A New Computational Intelligence Paradigm*. Springer-Verlag, 2002.
- [26] Z. Huang and Y. Zhou, "Runtime analysis of somatic contiguous hypermutation operators in MOEA/D framework," in *Proc. of AAAI*, 2020, pp. 2359–2366.
- [27] F. M. Burnet, *The Clonal Selection Theory of Acquired Immunity*. Cambridge University Press, 1959.
- [28] E. A. Aguilar Arroyo and C. A. Coello Coello, "Moaisdx: A new multi-objective artificial immune system based on decomposition," in *Proc. of EMO 2025*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 46–59.

- [29] L. Li, Q. Lin, Z. Ming, M. Gong, and C. A. Coello Coello, "An immune-inspired resource allocation strategy for many-objective optimization," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 53, no. 6, pp. 3284–3297, 2022.
- [30] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast contiguous somatic hypermutations for single-objective optimisation and multi-objective optimisation via decomposition," in *Proc. of AAAI'25*, 2025, pp. 26922–26930.
- [31] Z. Huang and Y. Zhou, "Runtime analysis of somatic contiguous hypermutation operators in moea/d framework," in *Proc. of AAAI 2020*, 2020, pp. 2359–2366.
- [32] L. Li, Q. Lin, and Z. Ming, "A survey of artificial immune algorithms for multi-objective optimization," *Neurocomputing*, vol. 489, pp. 211–229, 2022.
- [33] J. Brownlee, "Clonal selection algorithms," Complex Intelligent Systems Laboratory (CIS), Swinburne University of Technology, Tech. Rep., 2007.
- [34] L. N. de Castro and F. J. V. Zuben, "Learning and optimization using the clonal selection principle," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 3, pp. 239–251, 2002.
- [35] V. Cutello, G. Nicosia, M. Pavone, and J. Timmis, "An immune algorithm for protein structure prediction on lattice models," *IEEE Transactions on Evolutionary Computation*, vol. 11, pp. 101–117, 2007.
- [36] B. H. Ulutas and S. Kulturel-Konak, "A review of clonal selection algorithm and its applications," *Artificial Intelligence Review*, vol. 36, pp. 117–138, 2011.
- [37] A. Auger and N. Hansen, "Theory of evolution strategies: a new perspective," in *Theory of Randomized Search Heuristics: Foundations and Recent Developments*, A. Auger and B. Doerr, Eds. World Scientific Publishing Co., Inc., 2011, pp. 289–325.
- [38] C. Zarges, "Rigorous runtime analysis of inversely fitness proportional mutation rates," in *Proc. of PPSN*, 2008, pp. 112–122.
- [39] D. Corus, P. S. Oliveto, and D. Yazdani, "When hypermutations and ageing enable artificial immune systems to outperform evolutionary algorithms," *Theoretical Computer Science*, vol. 832, pp. 166–185, 2019.
- [40] —, "Artificial immune systems can find arbitrarily good approximations for the NP-hard partition problem," in *Proc. of PPSN 2018*, 2018, pp. 16–28.
- [41] —, "Artificial immune systems can find arbitrarily good approximations for the NP-hard number partitioning problem," *Artificial Intelligence*, vol. 247, pp. 180–196, 2019.
- [42] T. Jansen and C. Zarges, "Variation in artificial immune systems: hypermutations with mutation potential," in *Proc. of ICARIS 2011*, 2011, pp. 132–145.
- [43] S. Droste, T. Jansen, and I. Wegener, "On the analysis of the $(1 + 1)$ evolutionary algorithm," *Theoretical Computer Science*, vol. 276, no. 1–2, pp. 51–81, 2002.
- [44] J. Jägersküpper and T. Storch, "When the plus strategy outperforms the comma strategy and when not," in *Proc. of FOCI 2007*, 2007, pp. 25–32.
- [45] P. S. Oliveto and D. Sudholt, "On the runtime analysis of stochastic ageing mechanisms," in *Proc. of GECCO 2014*, 2014, pp. 113–120.
- [46] D. Corus, P. S. Oliveto, and D. Yazdani, "On the runtime analysis of the Opt-IA artificial immune system," in *Proc. of GECCO 2017*, 2017, pp. 83–90.
- [47] C. Horoba, T. Jansen, and C. Zarges, "Maximal age in randomized search heuristics with aging," in *Proc. of GECCO 2009*, 2009, pp. 803–810.
- [48] P. S. Oliveto, P. K. Lehre, and F. Neumann, "Theoretical analysis of rank-based mutation - combining exploration and exploitation," in *Proc. of CEC 2009*, 2009, pp. 1455–1462.
- [49] D. Corus, P. S. Oliveto, and D. Yazdani, "On inversely proportional hypermutations with mutation potential," in *Proc. of GECCO 2019*, 2019, p. 215–223.
- [50] P. S. Oliveto and C. Witt, "On the runtime analysis of the simple genetic algorithm," *Theoretical Computer Science*, vol. 545, pp. 2–19, 2014.
- [51] —, "Improved time complexity analysis of the simple genetic algorithm," *Theoretical Computer Science*, vol. 605, pp. 21–41, 2015.
- [52] D. Whitley, "The genitor algorithm and selection pressure: Why rank-based allocation of reproductive trials is best," in *Proc. of the Third International Conference on Genetic Algorithms*, 1989, pp. 116–121.
- [53] F. Neumann, P. S. Oliveto, and C. Witt, "Theoretical analysis of fitness-proportional selection: landscapes and efficiency," in *Proc. of GECCO 2009*, 2009, pp. 835–842.
- [54] D. Corus, D.-C. Dang, A. V. Eremeev, and P. K. Lehre, "Level-based analysis of genetic algorithms and other search processes," *IEEE Transactions on Evolutionary Computation*, vol. 22, no. 5, pp. 707–719, 2017.
- [55] D. Corus and P. S. Oliveto, "On the benefits of populations for the exploitation speed of standard steady-state genetic algorithms," *Algorithmica*, vol. 82, pp. 3676–3706, 2020.
- [56] S. Böttcher, B. Doerr, and F. Neumann, "Optimal fixed and adaptive mutation rates for the leadingones problem," in *Proc. of PPSN'10*. Springer, 2010, pp. 1–10.
- [57] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "How the duration of the learning period affects the performance of random gradient selection hyper-heuristics," in *Proc. of AAAI 2020*, 2020, pp. 2376–2383.
- [58] W. Feller, *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, 1968.
- [59] D. Corus, P. S. Oliveto, and D. Yazdani, "Fast artificial immune systems," in *Proc. of PPSN 2018*, 2018, pp. 67–78.
- [60] A. Lissovoi, P. S. Oliveto, and J. A. Warwicker, "On the time complexity of algorithm selection hyper-heuristics for multimodal optimisation," in *Proc. of AAAI 2019*, 2019, pp. 2322–2329.
- [61] A. Bendahi, B. Doerr, A. Fradin, and J. F. Lutzeyer, "Speeding up hyper-heuristics with markov-chain operator selection and the only-worsening acceptance operator," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, ser. IJCAI '25, 2025.
- [62] J. L. Doob, *Stochastic Processes*. John Wiley & Sons, 1953.
- [63] J. Cervantes and C. R. Stephens, "Rank based variation operators for genetic algorithms," in *Proc. of GECCO 2008*, 2008, pp. 905–912.
- [64] M. S. Krejca and C. Witt, "A flexible evolutionary algorithm with dynamic mutation rate archive," in *Proc. of GECCO '24*, 2024, p. 1578–1586.
- [65] A. Rajabi and C. Witt, "Self-adjusting evolutionary algorithms for multimodal optimization," *Algorithmica*, vol. 84, no. 6, pp. 1694–1723, 2022.
- [66] —, "Stagnation detection with randomized local search," *Evolutionary Computation*, vol. 31, no. 1, pp. 1–29, 2023.
- [67] D. Corus, A. Lissovoi, P. S. Oliveto, and C. Witt, "On steady-state evolutionary algorithms and selective pressure: Why inverse rank-based allocation of reproductive trials is best," *ACM Transactions on Evolutionary Learning and Optimization*, vol. 1, no. 1, 2021.
- [68] D. Corus, P. S. Oliveto, and F. Zheng, "Hybrid selection allows steady-state evolutionary algorithms to control the selective pressure in multimodal optimisation," in *Proc. of GECCO 2025*, 2025, pp. 881–889.
- [69] T. Jansen and C. Zarges, "On benefits and drawbacks of aging strategies for randomized search heuristics," *Theoretical Computer Science*, vol. 412, no. 6, pp. 543–559, 2011.
- [70] —, "Aging beyond restarts," in *Proc. of GECCO 2010*, 2010, pp. 705–712.

VIII. BIOGRAPHY SECTION

If you include a photo:



Michael Shell Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

If you will not include a photo:

John Doe Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.

Difference with conference version

The original extended abstract of this paper analysed the advantages of using an exponential decay of the mutation rate since this provides the largest speed-ups for unimodal functions in ideal conditions where the optimum is known (see Section III). However, a great disadvantage of the exponential decay is that large mutations hardly ever occur in practice. Indeed no superiority of Inversely Proportional Hypermutations over Standard Bit Mutation operators was provided. To this end, in this paper we analyse the use of the linear decay operator and provide examples where the IPH indeed considerably outperforms SBM providing a difference between polynomial and superpolynomial expected runtimes. Sections 4 and 5 of the conference version have been removed from this manuscript and been replaced with two completely new sections (V and VI). Overall, Section IV has been thoroughly revised to analyse the linear decay operator, Section V requires a much more sophisticated proof for a completely different algorithm and Section VI is completely new.



On Inversely Proportional Hypermutations with Mutation Potential

Dogan Corus
Department of Computer Science
University of Sheffield
Sheffield
d.corus@sheffield.ac.uk

Pietro S. Oliveto
Department of Computer Science
University of Sheffield
Sheffield
p.oliveto@sheffield.ac.uk

Donya Yazdani
Department of Computer Science
University of Sheffield
Sheffield
dyazdani1@sheffield.ac.uk

ABSTRACT

Artificial Immune Systems (AIS) employing hypermutations with linear static mutation potential have recently been shown to be very effective at escaping local optima of combinatorial optimisation problems at the expense of being slower during the exploitation phase compared to standard evolutionary algorithms. In this paper, we prove that considerable speed-ups in the exploitation phase may be achieved with dynamic inversely proportional mutation potentials (IPM) and argue that the potential should decrease inversely to the distance to the optimum rather than to the difference in fitness. Afterwards, we define a simple (1+1) Opt-IA that uses IPM hypermutations and ageing for realistic applications where optimal solutions are unknown. The aim of this AIS is to approximate the ideal behaviour of the inversely proportional hypermutations better and better as the search space is explored. We prove that such desired behaviour and related speed-ups occur for a well-studied bimodal benchmark function called TwoMAX. Furthermore, we prove that the (1+1) Opt-IA with IPM efficiently optimises a second multimodal function, CLIFF, by escaping its local optima while Opt-IA with static mutation potential cannot, thus requires exponential expected runtime in the distance between the local and global optima.

CCS CONCEPTS

• **Computing methodologies** → *Bio-inspired approaches*.

KEYWORDS

Hypermutations, Artificial Immune Systems, Runtime Analysis.

ACM Reference Format:

Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2019. On Inversely Proportional Hypermutations with Mutation Potential. In *Genetic and Evolutionary Computation Conference (GECCO '19)*, July 13–17, 2019, Prague, Czech Republic. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3321707.3321780>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '19, July 13–17, 2019, Prague, Czech Republic

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-6111-8/19/07...\$15.00
<https://doi.org/10.1145/3321707.3321780>

1 INTRODUCTION

Artificial Immune Systems (AIS) for optimisation are generally inspired by the clonal selection principle [2]. For this reason they are also often referred to as clonal selection algorithms [1]. In the literature, two key features of clonal selection algorithms have been identified [9]:

- (1) The proliferation rate of each immune cell is proportional to its affinity with the selective antigen: the higher the affinity, the higher the number of offspring generated (clonal selection and expansion).
- (2) The mutation suffered by each immune cell during reproduction is inversely proportional to the affinity of the cell receptor with the antigen: the higher the affinity, the smaller the mutation (affinity maturation).

Indeed, well-known clonal selection algorithms employ mutation operators applied to immune cells (i.e., candidate solutions) with a rate that decreases with their similarity to the antigen (i.e., global optima) during affinity maturation. Often such operators are referred to as *inversely proportional hypermutations* (IPH). Popular examples of such clonal selection algorithms are Clonalg [10] and Opt-IA [8].

The ideal behaviour of the IPH operator is that the mutation rate is minimal in proximity of the global optimum and should increase as the difference between the fitness of the global optimum and that of the candidate solution increases. However, achieving such behaviour in practice may be problematic because the fitness of the global optimum is usually unknown. As a result, in practical applications some information about the problem is used to identify bounds on (or estimates of) the value of the global optimum¹. Thus, the closer is the estimate to the actual value of the global optimum, the closer should the behaviour of the IPH operator be to the desired one. On the other hand, if the bound is much higher (for a maximisation problem) than the true value, then there is a risk that the mutation rate is too high in proximity of the global optimum, i.e., the algorithm will struggle to identify the optimum.

Previous theoretical analyses, though, have highlighted various problems with IPH operators even when the fitness value of the global optimum is known. Zarges analysed the effects of mutating candidate solutions with a rate that is inversely proportional to their fitness for the ONEMAX problem [26]. She considered two different rates for the decrease of the mutation rate as the fitness increases: a linear decay (i.e., each bit flips with probability $\text{ONEMAX}(x)/\text{opt}$ where *opt* is the optimum value) and an exponential decay (i.e.,

¹Alternatively, the fitness of the best candidate solution is sometimes used and the mutation rate of the rest of the population is inversely proportional to the best.

each bit flips with probability $e^{-\rho \frac{\text{ONEMAX}(x)}{\text{opt}}}$ where ρ is called the *decay* parameter). The motivation behind these choices are that the former operator flips in expectation exactly the number of bits that maximises the probability of reaching the optimum in the next step while the latter is the operator used in the Clonalg algorithm. She showed that if the optimum of ONEMAX is known, then an algorithm employing such a mutation operator will require exponential time to optimise ONEMAX with overwhelming probability² (w.o.p.) in both cases of linear or exponential decays of the mutation rate. The reason is that the initial random solutions that have roughly half the fitness of the global optimum (i.e., fitness values of $n/2$ and n , respectively), have very high mutation rates. Such rates do not allow the algorithm to make any progress with any reasonable probability even if the decay parameter ρ is chosen very carefully (i.e., $\rho = \ln n$ leads to reasonable mutation rates between $1/n$ and $1/2$ for every possible fitness value of ONEMAX). Hence, the desired behaviour of the mutation operator, achieved by assuming the optimum is known, leads to very inefficient algorithms even for the simple ONEMAX function.

On the other hand, the Opt-IA clonal selection algorithm, that also uses very high mutation rates (called hypermutations with mutation potential), has been proven to be efficient by employing a selection strategy called *stop at first constructive mutation* (FCM). This strategy evaluates fitness after each bit is flipped and interrupts the hypermutation immediately if an improvement is detected. Indeed, the operator using a static mutation potential (i.e., a linear number of bits are flipped unless an improvement is found along the way) has been proven to be very effective at escaping local optima of standard multimodal benchmark functions [3, 7] and at finding arbitrarily good approximations for the Number Partitioning NP-Hard problem [4, 5] at the expense of being slower at hillclimbing during exploitation phases (i.e., it is up to a linear factor slower than standard bit mutation for easy functions such as ONEMAX and LEADINGONES). Hence, differently from other clonal selection algorithms (such as Clonalg), hypermutations with mutation potential coupled with FCM can cope with the desired behaviour of inversely proportional mutation rates.

In this paper, we consider whether Opt-IA may become faster during exploitation phases if inversely proportional mutation potentials are applied rather than static ones. The reason to believe this is that hypermutations waste many fitness function evaluations during exploitation because as the optimum is approached, the probability of flipping bits that lead to an improvement decreases. Hence, with high probability the operator flips wrong bits at the beginning of a hypermutation and ends up wasting a linear number of fitness function evaluations. On the other hand, if the mutation potential decreases as the optimum is approached, then the amount of wasted evaluations should decrease accordingly.

A previous runtime analysis of the inversely proportional hypermutations with mutation potential used by Opt-IA for ONEMAX has shown that the mutation rate is always in the range of $[(c/2)n, cn]$ where $M = cn$ is the highest mutation potential. Hence, it does not decrease inversely proportional to the optimum as desired [16].

In this paper we first show that considerable speed-ups in exploitation phases may be achieved compared to static hypermutations, if mutation rates decrease appropriately with either the fitness or the distance to the optimum. To show this, we analyse the IPH operators for standard unimodal benchmark functions where the performance of static mutation potentials is well-understood [3]. Through the analysis, we show what speed-ups may be hoped for by analysing the operators in the ideal situation where the location of the optimum is known. A result of this first analysis is that a mutation potential that increases exponentially with the Hamming distance to the optimum is the most promising out of three considered IPMs since it provides the largest speed-ups. Furthermore, using the Hamming distance rather than fitness as measure to quantify proximity to the optimum makes the operator robust to fitness function scaling. Hence, we consider this operator which is called M_{expoHD} in the rest of the paper.

Afterwards, we propose an AIS called (1+1) Opt-IA which uses IPH and ageing to be applied in practical applications where the optimum is not known³. The algorithm uses the best solution it has seen to estimate the mutation rates. In the literature it has been shown that ageing allows algorithms to escape from local optima by identifying a new slope of increasing fitness or to completely restart the optimisation process if it cannot escape [3, 4, 14, 15]. The idea behind the algorithm is that the more the search space that is explored, the better the ideal behaviour of the IPH is approximated through the discovery of better and better local optima.

We use the CLIFF benchmark function to show that the IPH can escape local optima when coupled with ageing. Static hypermutations cannot optimise the function efficiently because once the local optimum is escaped, the high mutation rates force the algorithm to jump back to the local optima with high probability. Our analysis though reveals that in general such a strategy does not produce the desired effect. Since the mutation rate decreases with the distance to the best found local optimum, the algorithm may encounter difficulties at identifying new promising optima. In particular, if the algorithm identifies some slope that leads away from the previous local optimum, then the mutation rate will increase as the new optimum is approached. Firstly, this makes the new optimum hard to identify. Secondly, the high mutation rates can lead to high wastage of fitness function evaluations defeating our main motivation of reducing such wastage compared to static mutation potentials. We rigorously prove this effect for the well-studied TwoMAX bimodal benchmark function where the expected runtime does not improve compared to the runtime of static hypermutations to optimise each slope of the function (i.e., $\Theta(n^2 \log n)$). To this end, we define a *Symmetric* IPH operator that decreases the potential with respect to the distance to the best seen local optimum and uses the same rate of decrease in all other directions. We prove the effectiveness of our strategy and subsequent speed-ups over static hypermutations for the TwoMAX benchmark function while showing that the local optima of CLIFF can still be escaped by the Symmetric IPH operator.

Due to space limitations, some proofs are omitted⁴.

²We say events occur “with overwhelming probability” (w.o.p.) when they occur with probability at least $1 - o(1/\text{poly}(n))$.

³The well-known Opt-IA AIS uses both operators in combination [8].

⁴A complete version of the paper with all the proofs is available in [6].

2 PRELIMINARIES

The original IPH potential proposed for Opt-IA was $M = \lceil (1 - f_{OPT}/v) \rceil cn$ for minimisation problems, where f_{OPT} is the best known fitness and v is the fitness of the individual. In an analysis for ONEMAX, such a mutation potential was shown not to decrease below $cn/2$, with cn being the highest possible mutation potential [16]. In this section, we introduce three different IPH operators that will be analysed in this paper. Two have been already considered in the literature while the third one is newly proposed by us based on the performance of the first two.

2.1 Hamming Distance Based Linear Decrease

Zarges analysed an IPH operator where the probability of flipping each bit increases linearly with the Hamming distance to the optimum (or its best available estimate) [26]. Precisely, this operator flips each bit with probability $\min\{H(x, best)\}/n$ where n is the size of the problem and $\min\{H(x, best)\}$ is the minimum Hamming distance of the current point to a best individual. Here, we consider the mutation potential version of this operator. As the expected number of bit-flips is $\min\{H(x, best)\}$ during each execution of the mutation operator, the mutation potential would be

$$M_{linHD}(x) := \min\{H(x, best)\}. \quad (1)$$

2.2 Fitness Difference Based Exponential Decrease

In Clonalg's IPH operator, the mutation rate decreases as an exponential function of the fitness of the current solution [10]. Precisely, each bit flips with probability $e^{-\rho \cdot v}$ where v is the normalised fitness value and ρ is a decay parameter that regulates the speed at which the mutation rate decreases. Since we consider only maximisation problems, we use $v = \frac{f(x)}{f(best)}$ as suggested by [26] where $best$ is the best known fitness value. Using this mutation operator as a mutation potential gives $M = n \cdot e^{-\rho \cdot v}$. According to both practical and theoretical results [26], a reasonable value for ρ is $\ln n$. We call this mutation potential $M_{expoF(x)}$ and define it as

$$M_{expoF(x)}(x) := \left\lceil n^{1 - \frac{f(x)}{f(best)}} \right\rceil. \quad (2)$$

2.3 Hamming Distance Based Exponential Decrease

Since it is well understood that using differences in fitness values makes randomised search heuristics unstable towards the scaling of fitness functions [17, 20, 21, 25], we also consider a mutation potential which is similar to $M_{expoF(x)}$ with the exception that it uses the normalised Hamming distance to the best estimate rather than the normalised fitness. We call this mutation potential M_{expoHD} and define it as $M_{expoHD} = n \cdot e^{-\rho \cdot \frac{n-H(x, best)}{n}}$ where n is the maximum Hamming distance from any search point to the optimum (which is always at most n), $H(x, best)$ is the Hamming distance to the best known solution and ρ is the decay of the mutation potential. For the choice of $\rho = \ln n$, we get

$$M_{expoHD}(x) := \left\lceil n^{\frac{H(x, opt)}{n}} \right\rceil. \quad (3)$$

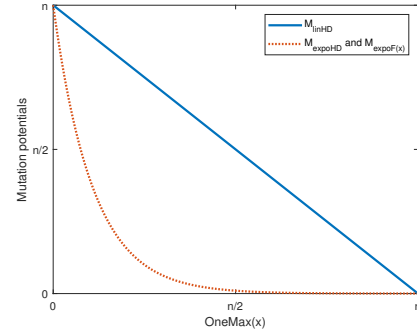


Figure 1: Different mutation potentials for ONEMAX.

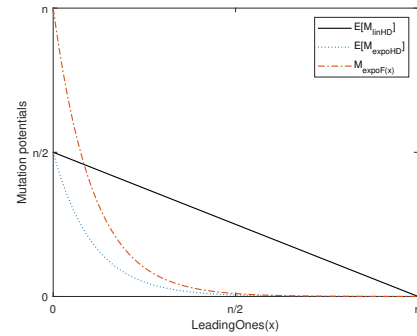


Figure 2: Mutation potentials for LEADINGONES. Expected values are shown for those using Hamming distance.

3 IDEAL BEHAVIOUR ANALYSIS

In this section, we evaluate the performance of the different inversely proportional mutation potentials, assuming the optimum is known (i.e., $best = opt$). Under this assumption, the operators exhibit their ideal behaviour (i.e., the mutation potential decreases with the desired rate as the optimum is approached). Our aim is to evaluate what speed-ups can be achieved in ideal conditions compared to the well-studied static mutation potentials. To achieve such comparisons we perform runtime analyses of the $(1+1)$ IA^{hyp} using the IPHs on the ONEMAX and LEADINGONES unimodal benchmark functions for which the performance of the same algorithm using static mutation potentials is known [3]. The simple to define ONEMAX function counts the number of 1-bits in the bit string and is normally used to evaluate the hill climbing ability of algorithms. On the other hand, LEADINGONES is a more complicated unimodal problem which counts the consecutive number of 1-bits at the beginning of the bit string before the first 0-bit.

The pseudo-code of the $(1+1)$ IA^{hyp} is given in Algorithm 1. It simply uses one candidate solution in each iteration to which inversely proportional hypermutations are applied. The results proven in this section and comparisons with static hypermutations are summarised in Table 1. As previously mentioned, a *constructive mutation* in Algorithm 1 is a mutation step where the evaluated bit string is at least as fit as its parent.

Table 1: Comparison of the expected runtimes obtained by different hypermutation schemes for ONEMAX and LEADINGONES. The original IPH operator of Opt-IA [8] has the same expected runtimes as the $(1 + 1)$ IA^{hyp} using static hypermutations [16].

Algorithms	ONEMAX	LEADINGONES
$(1 + 1)$ IA ^{hyp} using M_{linHD}	$\Theta(n^2)$	$\Theta(n^3)$
$(1 + 1)$ IA ^{hyp} using $M_{\text{expoF}(x)}$	$O(n^{(3/2)+\epsilon} \log n), \Omega(n^{3/2-2\epsilon})$	$O(n^3 / \log n), \Omega(n^{5/2+\epsilon})$
$(1 + 1)$ IA ^{hyp} using M_{expoHD}	$O(n^{(3/2)+\epsilon} \log n), \Omega(n^{3/2-2\epsilon})$	$O(n^{\frac{5/2+\epsilon}{\ln n}}), \Omega(n^{9/4-\epsilon})$
$(1 + 1)$ IA ^{hyp} with $M = n$ [3]	$\Theta(n^2 \log n)$ [3]	$\Theta(n^3)$ [3]

Algorithm 1 $(1 + 1)$ IA^{hyp}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: while termination condition is not satisfied do
3:   create  $y$  by flipping at most  $M := \text{IPM}(x)$  distinct bits of
    $x$  selected uniformly at random one after another until a
   constructive mutation happens;
4:   if  $f(y) \geq f(x)$  then
5:      $x := y$ .
```

Figures 1 and 2 show how the studied mutation potentials ideally decrease during the run of the algorithm when optimising ONEMAX and LEADINGONES, respectively.

3.1 ONEMAX

The following theorems derive the expected runtimes of the three variants of IPM for $\text{ONEMAX}(x) := \sum_{i=1}^n x_i$. By decreasing the potential linearly with the decrease of the Hamming distance, a logarithmic factor may be shaved off from the expected runtime of the $(1 + 1)$ IA^{hyp} compared to the expected runtime achieved by the same algorithm using static hypermutation potentials.

THEOREM 3.1. *The $(1 + 1)$ IA^{hyp} using M_{linHD} optimises ONEMAX in $\Theta(n^2)$ expected fitness function evaluations.*

PROOF. Considering i as the number of 0-bits in the candidate solution, the probability of improvement in the first step is i/n . Knowing that at most n improvements are needed to find the optimum and in case of failure, $H(x, \text{opt}) = i$ fitness function evaluations would be wasted, the total expected time to optimise ONEMAX is at most $\sum_{i=1}^n \frac{n}{i} \cdot i = O(n^2)$.

In order to prove a lower bound, we use the Ballot theorem [7, 12]. By Chernoff bounds, the number of 0-bits in the initialised solution is at least $n/3$ w.o.p. Considering the number of 0-bits as $i = q$ and the number of 1-bits as $n - i = p$, the probability of an improvement is at most $1 - (p - q)/(p + q) = 1 - (n - 2i)/n = 2i/n$ by the Ballot theorem, where $i = H(x, \text{opt})$. This means that we need to wait at least $n/(2i)$ iterations to see an improvement and each time the mutation operator fails to improve the fitness, i fitness function evaluations will be wasted. Considering that at least $n/3$ improvements are needed, the expected time to optimise ONEMAX is larger than $\sum_{i=1}^{n/3} \frac{n}{2i} \cdot i = \Omega(n^2)$. $\square$

The following theorem shows that a greater speed up may be achieved if the potential decreases exponentially rather than linearly. Its proof deviates from the proof of Theorem 3.2 only in the

amount of wasted evaluations. Note that for ONEMAX the Hamming distance of a solution to the optimum and its difference in fitness are the same. The proof is omitted due to lack of space.

THEOREM 3.2. *The $(1 + 1)$ IA^{hyp} using either $M_{\text{expoF}(x)}$ or M_{expoHD} optimises ONEMAX in $O(n^{3/2+\epsilon} \log n)$ and $\Omega(n^{3/2-\epsilon})$ expected fitness function evaluations for any arbitrarily small constant $\epsilon > 0$.*

3.2 LEADINGONES

In the previous section, it was shown that decreasing the mutation rate linearly with the Hamming distance to the optimum gives a logarithmic factor speed-up for ONEMAX compared to using static hypermutations. The following theorem shows that no improvement over static mutation potentials are achieved for LEADINGONES: $\sum_{i=1}^n \prod_{j=1}^i x_j$. The main reason for the lack of asymptotic improvement is that a linear mutation potential is sustained until at least a linear number of improvements are achieved.

THEOREM 3.3. *The $(1 + 1)$ IA^{hyp} using M_{linHD} optimises LEADINGONES in $\Theta(n^3)$ expected fitness function evaluations.*

PROOF. The probability of improvement in each step is at least $1/n$ which is the probability of flipping the leftmost 0-bit. As at most n improvements are needed and each failure in improvement yields $\frac{n-i-1}{2} + 1 + \epsilon n$ wasted fitness function evaluations in expectation (i.e., the expected $H(x, \text{opt})$ value), the expected time to find the optimum is at most $\sum_{i=1}^n n \cdot \left(\frac{n-i-1}{2} + 1 + \epsilon n \right) = O(n^3)$.

The proof for the lower bound is the same as the proof of Theorem 3.6 in [3] for the expected runtime of the $(1 + 1)$ IA^{hyp} with the exception that the amount of wasted fitness function evaluations in case of failure is $H(x, \text{opt})$ instead of n now⁵.

Considering i as the number of leading 1-bits, we denote the expected number of fitness function evaluations until an improvement happens by $E(f_i)$. Any such candidate solution has i leading 1-bits with a 0-bit following, and then $n - i - 1$ other bits which are distributed uniformly at random [11]. We take into account three possible events E_1, E_2 and E_3 that can happen in the first bit flip: E_1 is the event of flipping a leading 1-bit which happens with probability i/n , E_2 is the event of flipping the first 0-bit which happens with probability $1/n$, and E_3 is the event of flipping any other bit which happens with probability $(n - i - 1)/n$. So we get $E(f_i|E_1) = H(x, \text{opt}) + E(f_i)$, $E(f_i|E_2) = 1$, and $E(f_i|E_3) = 1 + E(f_i)$. Knowing that the bits

⁵ $(1 + 1)$ IA^{hyp} is a simple $(1+1)$ EA algorithm using static hypermutation operator (i.e., $M = n$) instead of standard bit mutation. Its selection mechanism accepts equally fit solutions.

after the left most 0-bit are distributed uniformly at random [11], the value of $H(x, \text{opt})$ is not less than $1 + \frac{(n-i-1)}{2} - \epsilon n$ w.o.p. by Chernoff bounds. Hence, by the law of total expectation, the expected number of fitness function evaluations for every improvement is $E(f_i) = \frac{1}{n} \left(1 + \frac{n-i-1}{2} - \epsilon n + E(f_i) \right) + \frac{1}{n} \cdot 1 + \frac{n-i-1}{n} (1 + E(f_i))$. Solving the equation for $E(f_i)$ gives us $E(f_i) = \frac{in-i^2-i-i\epsilon n}{2}$.

As we are computing the lower bound, we have to take into account the probability of skipping any level i . To do this, we need to calculate the expected number of consecutive 1-bits that follow the leftmost 0-bit (called free riders [11]). The expected number of free riders is $\sum_{i=1}^{n-1} i \cdot 1/2^{i+1} < \frac{1}{2} \cdot \sum_{i=0}^{\infty} i \cdot (1/2)^i = \frac{1}{2} \cdot \frac{1/2}{(1-1/2)^2} = 1$. This means that the probability of not skipping a level i is $\Omega(1)$. On the other hand, with probability at least $1 - 2^{-n/2}$, the initial solution does not have more than $n/2$ leading 1-bits. Thus, we obtain a lower bound of $(1 - 2^{-n/2}) \sum_{i=n/2}^n E(f_i) = \Omega(1) \sum_{i=n/2}^n \frac{in-i^2-i-i\epsilon n}{2} = \Omega(n^3)$ on the expected number of fitness function evaluations. $\square$

Theorem 3.5 shows that exponential fitness-based mutation potential gives us at least a logarithmic and at most a $\sqrt{n}$ factor speed-up compared to static mutation potentials. Before proving the main results, we introduce the following lemma that will be used in the proof of upcoming theorems.

LEMMA 3.4. *For large enough n and any arbitrarily small constant ϵ , $n^{1/n^\epsilon} = (1 + \frac{\ln n}{n^\epsilon})(1 \pm o(1))$.*

PROOF. By raising $\left(1 + \frac{\ln n}{n^\epsilon}\right)$ to the power of $\frac{n^\epsilon}{\ln n} \cdot \frac{\ln n}{n^\epsilon}$ we have $\left(1 + \frac{\ln n}{n^\epsilon}\right)^{\frac{n^\epsilon}{\ln n} \cdot \frac{\ln n}{n^\epsilon}} = (1 \pm o(1)) e^{\frac{\ln n}{n^\epsilon}} = (1 \pm o(1)) n^{1/n^\epsilon}$. $\square$

THEOREM 3.5. *The $(1+1)$ IA^{hyp} using $M_{\text{expoF}(x)}$ optimises LEADINGONES in $O(n^3/\log n)$ and $\Omega(n^{5/2+\epsilon})$ expected fitness function evaluations for any arbitrarily small constant $\epsilon > 0$.*

PROOF. As already shown in the proof of Theorem 3.3, the expected number of leading 1-bits is less than 1 in the initialised bit string. The probability of improvement in the first step is $1/n$. In case of failure in improving at the first step, at most $n \cdot \frac{n-i}{n}$ fitness function evaluations would get wasted where i is the number of leading 1-bits. Therefore, the total expected time to find the optimum is $E(T) \leq \sum_{i=1}^{n-1} n \cdot n^{(n-i)/n} = O(n^3/\log n)$ considering that $\sum_{i=1}^{n-1} n^{i/n} = \sum_{i=2}^n (n^{1/n})^{i-1}$ which is less than $\sum_{i=1}^n (n^{1/n})^{i-1} = \frac{1-n}{1-n^{1/n}}$ that gets in turn lower bounded by $n^2/(\log n)$ using Lemma 3.4.

The proof for the lower bound is similar to the proof of Theorem 3.3, except in the calculation of $E(f_i)$ when we want to consider the amount of wasted fitness function evaluations in case of E_1 happening. Here we have $E(f_i) = \frac{1}{n} \left(n^{(n-i)/n} + E(f_i) \right) + \frac{1}{n} \cdot 1 + \frac{n-i-1}{n} (1 + E(f_i))$. Solving it for $E(f_i)$ gives us $E(f_i) = i \cdot n^{(n-i)/n} + n - i$. Hence, the expected time to optimise LEADINGONES is $(1 - 2^{-n/2}) \sum_{i=n/2}^n E(f_i) = \Omega(1) \sum_{i=n/2}^n \left(i \cdot n^{(n-i)/n} + n - i \right) = \Omega(1) \left(\sum_{i=n/2}^n (n-i) + \sum_{i=n/2}^n \left(i \cdot n^{(n-i)/n} \right) \right)$. Evaluating the second sum in the interval $i \in [n/2 + \frac{\epsilon n}{2}, n/2 + \epsilon n]$, we get $\epsilon n/2 \cdot (n/2 + \epsilon n) \cdot n^{1/2-\epsilon/2} = \Omega(n^{5/2-\epsilon})$. $\square$

An advantage of Hamming distance-based exponential decays of the mutation potential compared to fitness-based ones are provided by the following theorem for LEADINGONES. The reason can be seen in Fig. 2. While the initial fitness is very low, the potential is very high, but the actual number of bits that have to be flipped to reach the optimum (i.e., the Hamming distance), is much smaller. More precisely, fitness-based potentials suggest to flip n bits when only $n/2$ bits have to be flipped in expectation to reach the optimum in one step. M_{expoHD} exploits this property, thus wastes less fitness evaluations than $M_{\text{expoF}(x)}$. The proof of the following theorem is similar to that of the two previous theorems and is omitted due to lack of space.

THEOREM 3.6. *The $(1+1)$ IA^{hyp} using M_{expoHD} optimises LEADINGONES in $O(\frac{n^{5/2+\epsilon}}{\ln n})$ and $\Omega(n^{9/4-\epsilon})$ expected fitness function evaluations for any arbitrarily small constant $\epsilon > 0$.*

Given that M_{expoHD} provides larger speed-ups compared to the other IPH operators and is stable to the scaling of fitness functions, we will use it in the remainder of the paper.

4 INEFFICIENT BEHAVIOUR IN PRACTICE

In this section we consider the usage of M_{expoHD} in realistic applications where the optimum is unknown. To this end, the best found solution will be used by the operator rather than the unknown optimum. We combine M_{expoHD} with hybrid ageing, as in the Opt-IA AIS [8], and call it $(1+1)$ Opt-IA. Its pseudo-code is provided in Algorithm 2. Ageing has been shown to enable algorithms to escape from local optima either by identifying a gradient leading away from it or by restarting the whole optimisation process.

Our aim is that the greater the number of local optima that are identified by the algorithm, the better M_{expoHD} approximates its ideal behaviour. However, we will show that this is not the case, e.g., for the well-studied bimodal benchmark function TwoMAX (Eq. ??) [13, 19, 23, 24]: $\text{TwoMAX} := \max \{ \sum_{i=1}^n x_i, n - \sum_{i=1}^n x_i \}$. TwoMAX is usually used to evaluate the global exploration capabilities of evolutionary algorithms, i.e., whether the population identifies both optima of the function. Our analysis shows that once the $(1+1)$ Opt-IA escapes from one local optimum, the mutation rate will increase as the algorithm climbs up the other branch. As a result, the algorithm struggles to identify the other optimum and wastes more and more fitness function evaluations as it approaches it. Thus, the whole purpose behind IPH is defeated.

On the bright side, we will show that M_{expoHD} combined with ageing can escape from local optima. We will use the well known CLIFF function (Eq. 4) for this purpose where static hypermutations are inefficient. Due to the high mutation rates of static hypermutations, even if they escape the local optima of CLIFF, they jump back with high probability.

$$\text{CLIFF}_k(x) := \begin{cases} \sum_{i=1}^n x_i & \text{if } \sum_{i=1}^n x_i \leq n-k, \\ \sum_{i=1}^n x_i - k + 1/2 & \text{otherwise.} \end{cases} \quad (4)$$

Both functions are illustrated in Fig 3.

Among the different variants of ageing, hybrid ageing has been shown to be very efficient at escaping local optima [3, 4, 18]. This operator assigns an initial *age* = 0 to individuals. During each iteration, the age increases by 1 and is passed to the offspring

Algorithm 2 $(1 + 1)$ Opt-IA with M_{expoHD}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: set  $x.\text{age} := 0$  and  $\text{best} := x$ ;
3: while termination condition is not satisfied do
4:    $x.\text{age} := x.\text{age} + 1$ ;
5:   create  $y$  by flipping at most  $M := M_{\text{expoHD}}(x)$  distinct bits
     of  $x$  selected uniformly at random one after another until a
     constructive mutation happens;
6:   if  $f(y) > f(x)$  then
7:      $y.\text{age} := 0$ ;
8:   else
9:      $y.\text{age} := x.\text{age}$ ;
10:  if  $f(y) \geq f(\text{best})$  then
11:    then set  $\text{best} := y$ ;
12:  for  $w \in \{x, y\}$  do
13:    if  $w.\text{age} \geq \tau$  then
14:      with probability  $1/2$ , reinitialise  $w$  uniformly at random
      with  $w.\text{age} = 0$ ;
15:  Set  $x = \arg \max_{z \in \{x, y\}} f(z)$ ;
```

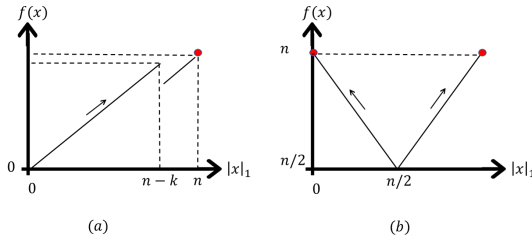


Figure 3: (a) CLIFF and (b) TwoMAX test functions. $|x|_1$ shows the number of 1-bits in the bit string.

if the offspring does not improve over its parent's fitness. If the offspring is fitter than the parent, then its age is set to 0. At the end of each iteration, any individual with age larger than a threshold τ is removed with probability $1/2$ and in case there is no other individual left, a new individual is initialised uniformly at random.

The following theorem shows that after the algorithm escapes from the local optimum, the mutation rate increases as the algorithm climbs up the opposite branch. This behaviour causes a large waste of fitness evaluations defying the objectives of IPH.

THEOREM 4.1. *The expected runtime of $(1+1)$ Opt-IA with M_{expoHD} for optimising TwoMAX is $O(n^2 \log n)$ with $\tau = \Omega(n^{1+\epsilon})$ for some constant $\epsilon > 0$.*

PROOF. Let x_t be the current solution at the beginning of the iteration t . Immediately after the initialisation, the best seen solution is the current individual x_1 itself and the mutation potential is $M = M_{\text{expoHD}} = n^{H(x_1, x_1)/n} = n^0 = 1$. Note that $x_t \neq x_{t+1}$ if and only if either $f(x_{t+1}) \geq f(x_t)$ or x_{t+1} is reinitialised after x_t is removed from the population due to ageing. Thus, the mutation operator flips a single bit at every iteration until ageing is triggered for the first time. The improvement probability will be at least $1/n$ until either 1^n or 0^n is sampled. Given that the ageing threshold

τ is at least $n^{1+\epsilon}$ for some constant $\epsilon > 0$, the probability that the current solution will not improve τ times consecutively is at most $(1 - \frac{1}{n})^{n^{1+\epsilon}} = e^{-\Omega(n^\epsilon)}$. Hence, the first optimum will be found before ageing is triggered w.o.p. Given that the ageing operator is not triggered, the expected time to find the first optimum is at most $O(n \log n)$ with similar proof to that of the standard $(1+1)$ RLS⁶. After finding the first optimum, no single-bit flip can yield an equally fit solution. The individual then reaches age τ and the ageing reinitialises the current solution. Thus, with $(1 - 2^{-n}) \cdot (1 - e^{-\Omega(n^\epsilon)})$ probability a new solution will be initialised in $\tau + n$ steps. Now, the current best is the first discovered optimum.

Let the first and second branch denote the subsets of the solution space which consist of solutions with less than and more than Hamming distance $n/2$ to the first discovered optimum respectively. We first consider the case when the new solution is initialised on the first branch. Given that the current solution has fitness i , the distance to the best seen is $n - i$ and the mutation potential is $M = n^{\frac{n-i}{n}}$. Since the first constructive mutation ensures that the probability of improvement is always at least $1/n$, w.o.p. the ageing will not be triggered until either optimum is discovered. Thus, given that the current solution is always on the first branch, the proof of Theorem 3.2 carries over and the expected time to find the first discovered optimum once again is at most $O(n^{3/2} \log n)$ in expectation. When the first discovered optimum is sampled again, w.o.p. the current solution is reinitialised with the first discovered optimum as the best seen solution in at most $\tau + n$ iterations.

Now, we consider the case where the current solution is on the second branch. A lower bound on the probability that the current solution will reach the optimum of the second branch before sampling an improving solution in the first branch will conclude the proof since the expected time to do so is $O(n^{3/2} \log n)$ given that the current solution does not switch branches before.

We will start by bounding the mutation potential for a solution in the second branch with fitness value $n - k$. Since the M_{expoHD} is always smaller than M_{linHD} , we can assume that no more than $n - k$ bits will be flipped.

Without losing generality, let the second branch be the branch with more 0-bits. For a hypermutation operation with mutation potential N , the N th solution that will be sampled has the highest probability of finding a solution with at least $n - k$ 1-bits (i.e., switching to the first branch). The current solution has $n - k$ 0-bits and k 1-bits. If more than $k/2$ 1-bits are flipped, then the number of 1-bits in the final solution after $n - k$ mutation steps is less than $n - k$ since the number of 0-bits flipped to 1-bits is less than $n - k - k/2$ and the number of remaining 1-bits is less than $k/2$. The event that at most $k/2$ 1-bits are flipped is equivalent to the event that in a uniformly random permutation of n bit positions at least $k/2$ 1-bits are ranked in the last k positions of the random permutation. We will now bound the probability that exactly $k/2$ 1-bits are in the last k position since having more 1-bits has a smaller probability. Each particular outcome of the last k positions has the equal probability of $\prod_{i=0}^{k-1} (n - i)^{-1}$. There are $\binom{k}{k/2}$ different equally likely ways to choose $k/2$ 1-bits and $k!$ different permutations of the last k positions, thus the probability of having exactly $k/2$ 1-bits in the

⁶ $(1 + 1)$ RLS₁ or random local search flips exactly one bit at each iteration.

last k positions is $\prod_{i=0}^{k-1} \frac{1}{n-i} \cdot \binom{k}{k/2} \cdot k! = \prod_{i=0}^{k-1} \frac{1}{n-i} \cdot \left(\frac{k!}{(k/2)!}\right)^2 < \left(\frac{k}{n-k}\right)^k$. For any $k \in [4, \frac{n}{2} - \Omega(\sqrt{n})]$ this probability is in the order of $\Omega(1/n^4)$. Using a union bound over the probabilities of having more than $k/2$ 1-bits and the probabilities of improving before the final step, we get $\frac{k}{2} \cdot (n-k) \cdot \Omega(1/n^4) = \Omega(1/n^2)$. Since the new solutions are uniformly sampled, the number of bits in the solutions are initially distributed binomially with parameters n and $1/2$ which has a variance in the order of $\Theta(\sqrt{n})$ and implies that with constant probability k is less than $\frac{n}{2} - \Omega(\sqrt{n})$ in the initial solution. Given that the expected time in terms of generations is in the order of $O(n \log n)$, the total probability of switching branches while $k \in [k, \frac{n}{2} - \Omega(\sqrt{n})]$ is at most $(1 - 1/n^2)^{O(n \log n)} = 1 - \Omega(1)$. Finally, we will consider the cases of $k \in \{1, 2, 3\}$ separately. When $k = 1$, the probability of flipping less than $k/2$ 1-bits is equivalent to the probability of not flipping the single 1-bit which happens with probability $1/n$. For $k = 2$, similarly we have to flip at most one 1-bit with probability $O(1/n)$. For $k = 3$, it is necessary that at least two 1-bits are not flipped which happens with probability at most $O(1/n^2)$. Thus, the probability of switching branches when $k < 4$ is at most $O(1/n)$, which gives a lower bound on the total probability of switching branches in the order of $1 - \Omega(1)$ given that the initial solution has at least $k > \frac{n}{2} - \Omega(\sqrt{n})$ 1-bits (which also occurs with at least $\Omega(1)$ probability). Given that no switch occurs, the expected time to find the second optimum is $\sum_{i=1}^{n/2+\epsilon n} \frac{n}{i} \cdot n^{\frac{n-i}{n}} = O(n^2 \log n)$. $\square$

Now we show that differently from static hypermutations, M_{expoHD} combined with ageing can escape from the local optima of CLIFF, hence optimises the function efficiently.

THEOREM 4.2. *The $(1 + 1)$ Opt-IA with M_{expoHD} and $\tau = \Omega(n^{1+\epsilon})$ for an arbitrarily small constant ϵ , optimises CLIFF $_k$ with $k < n(\frac{1}{4} - \epsilon)$ and $k = \Theta(n)$ in $O(n^{3/2} \log n + \tau n^{1/2} + \frac{n^{7/2}}{k^2})$ expected fitness function evaluations.*

PROOF. The analysis will follow a similar idea to the proof of Theorem 4.1. After initialisation, the initial mutation potential is $M = 1$ since the current solution is the best seen solution. With single bit-flips it takes in expectation at most $O(n)$ to find a local optimum of the cliff (i.e., a search point with $n - k$ 1-bits) as the improvement probability is always at least $(n - k)/n = \Omega(1)$. Since the local optima cannot be improved with single bit flips, in τ generations after it was first discovered the ageing will be triggered and in the following n steps the current solution will be removed from the population due to ageing with probability at least $1 - 2^{-n}$. The Hamming distance of the reinitialised solution will be distributed binomially with parameters n and $1/2$ and w.o.p. will be smaller than $n/2 + n^{2/3}$, yielding an initial mutation potential of $M = O(n^{1/2})$. We pessimistically assume that the mutation potential will not decrease until a local optima is found again, which implies that the expected time will be at most $O(n^{3/2} \log n + \tau n^{1/2})$ since each iteration will waste an extra $O(n^{1/2})$ fitness function evaluations. After finding a local optima again, the mutation potential will be $M = 1$ since it will replace the previously observed local optima as the best seen. The process of reinitialisation and reaching the local optima will repeat itself until the following event happens.

If the local optima produces an offspring with $n - k + 1$ bits with probability k/n and if this solution survives the ageing operator (with probability $(1 - p_{\text{die}})$), then the reinitialised solution will be rejected since its fitness value is less than $n - k$ w.o.p. The Hamming distance of this new solution to the best seen will be exactly one since it is created via a single bit-flip, thus its mutation potential will be $M = 1$. Moreover, if the surviving offspring improves again (with probability $(k - 1)/n$) in the next iteration, it will reset its age to zero and will have Hamming distance of at least two to any local optima. In expected $O(n/\log n)$ generations, this solution will reach the global optimum unless a solution with less or equal $n - k$ 1-bits is sampled before. Initially, this is impossible since for at least $\omega(1)$ steps we have $M = 1$, and later $M < 3$ holds as long as the distance to the last seen local optima is at most $n/\ln n$ since $n^{\frac{n/\ln n}{n}} = e$. Note that the number of 1-bits does not always reflect the actual Hamming distance since more than one bit can be flipped in an accepted offspring. We will pessimistically assume that all improvements have increased the Hamming distance by three until the total Hamming distance reaches $n/\ln n$, which implies that there have been $n/(3 \ln n)$ accepted solutions. As shown in the proof of Theorem 3.1, the Ballot theorem [7, 12]] implies that sampling a solution that is at least as good as the parent (which are the only solutions that are accepted) has probability at most $2i/n$ where i is the number of 0-bits in the solution. Since the probability of improving in the first step is at least i/n , we can conclude that the conditional probability that an accepted offspring is a strict improvement is at least $1/2$. Thus, when the Hamming distance to the local optima reaches $n/\ln n$, in expectation, the current solution will have at least $n/(6 \ln n)$ extra 1-bits compared to the local optima and at least $n^{3/5}$ extra 1-bits w.o.p. by Chernoff bounds. The Hamming distance to the local optima can be at most $2k$ since both the local optima and the current solution have less than k 0-bits. Since $k < n/4$, the mutation potential is at most $\sqrt{n}$. Thus, no hypermutation can yield a solution with less than $n - k + 2$ 1-bits. Therefore, once a solution with $n - k + 2$ bits is added to the population, the algorithm finds the optimum w.o.p. in $O(n \log n)$ iterations and in at most $O(n^{3/2} \log n)$ fitness function evaluations since the mutation potential is at most $\sqrt{n}$. The probability of obtaining a solution with $n - k + 2$ 1-bits at the end of each cycle of reinitialisation and removal of the local optima due to ageing is $(1 - p_{\text{die}})^2 \cdot (k/n) \cdot ((k - 1)/n)$. Since each such cycle takes $O(n^{3/2})$ fitness evaluations, our claim follows. $\square$

5 AN EFFICIENT OPT-IA WITH IPH

In the previous section, we observed in the analyses of both CLIFF and TwoMAX that towards the end of the optimisation process the mutation potential may increase as the current solution approaches an undiscovered, potentially promising optimum. This behaviour is against the design intentions of the inversely proportional mutation potential since in the final part of the optimisation process it gets harder to find improvements and high mutation potentials lead to many wasted fitness function evaluations. The underlying reason of this behaviour in both the CLIFF and TwoMAX landscapes is the necessity to follow a gradient that leads away from the local optimum to find the global one. Considering that this necessity would be ubiquitous in optimisation problems, we propose a new method to control mutation potentials in this section.

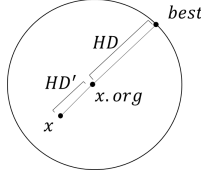


Figure 4: The geometric representation of Symmetric M_{exp0HD} . The mutation potential is determined according to the ratio of the Hamming distance between the current solution (x) and its origin ($x.\text{org}$) and the Hamming distance between its origin and the best seen solution.

Algorithm 3 (1+1) Opt-IA with Symmetric M_{exp0HD}

```

1: Initialise  $x \in \{0, 1\}^n$  uniformly at random;
2: set  $x.\text{origin} := x$ ,  $x.\text{age} := 0$ , and  $\text{best} := x$ ;
3: while termination condition not satisfied do
4:    $x.\text{age} := x.\text{age} + 1$ ;
5:   Create  $y$  by flipping at most  $M := \text{Symmetric } M_{\text{exp0HD}}(x)$ 
     distinct bits of  $x$  selected uniformly at random one after
     another until a constructive mutation happens;
6:    $y.\text{origin} := x.\text{origin}$ ;
7:   if  $f(y) > f(x)$  then
8:      $y.\text{age} := 0$ ;
9:     if  $f(y) \geq \text{best}$  then
10:       $\text{best} := y$ ;
11:   else
12:      $y.\text{age} := x.\text{age}$ ;
13:   for  $w \in \{x, y\}$  do
14:     if  $w.\text{age} \geq \tau$  then
15:       with probability  $1/2$ , reinitialise  $w$  uniformly at random
       with  $w.\text{age} = 0$ ;
16:       set  $w.\text{origin} = w$ ;
17:   Set  $x = \arg \max_{z \in \{x, y\}} f(z)$ ;
```

Algorithm 3 uses a mutation potential inversely proportional to the current solution's Hamming distance to its origin, where the origin is defined as the ancestor of the current bitstring after the last removal of a solution due to ageing (Fig. 4). We call the newly proposed mutation potential *Symmetric M_{exp0HD}* and define it as

$$\text{Symmetric } M_{\text{exp0HD}} := \max \left\{ \left\lfloor n \frac{H(\text{best}, x.\text{org}) - H(x, x.\text{org})}{n} \right\rfloor, 1 \right\}. \quad (5)$$

This mutation potential reliably decreases (at the same rate it would decrease if it was approaching the currently best seen local optimum) as the current solution improves and moves away from its origin up until it starts doing local search and finds a local optimum. Every time a local optimum is found, ageing is triggered after approximately τ steps and then both surviving and reinitialised individuals reset their origin to their own bitstring.

The following theorem shows that once one optimum of TwoMax has been identified, the value of the Symmetric M_{exp0HD} potential decreases as both optima are approached as desired and the wished for speed-up in the runtime is achieved.

THEOREM 5.1. *The (1 + 1) Opt-IA using Symmetric M_{exp0HD} with $\tau = \Omega(n^{1+\epsilon})$ for any arbitrarily small constant $\epsilon > 0$, needs $O(n^{3/2} \log n)$ fitness function evaluations in expectation to optimise TwoMax.*

PROOF. The expected time until the first branch is optimised is $O(n \log n)$ since the best seen search point is the current best individual and consequently the mutation potential is $M = 1$. Since the improvement probability is at least $1/n$ and $\tau = \Omega(n^{1+\epsilon})$, the ageing operator does not trigger before finding one of the optima w.o.p. Once one of the optima is found, ageing reinitialises the individual while the first discovered optima stays as the current best seen search point. For the randomly reinitialised solution, the Hamming distance to the best seen is binomially distributed with parameters n and $1/2$. Using Chernoff bounds, we can bound the distance to the previously seen optima by at most $n/2 + n^{2/3}$ w.o.p. This Hamming distance implies an initial mutation potential of $M < n^{\frac{n/2 + n^{2/3}}{n}} = n^{\frac{1}{2} + \frac{1}{n^{1/3}}}$ which decreases as the individual increases its distance to the origin and can never go above its initial value where the distance is zero. Pessimistically assuming that the mutation potential will be $n^{\frac{1}{2} + \frac{1}{n^{1/3}}} = O(n^{1/2})$ throughout the run, we can obtain the above upper bound by summing over all levels and using coupon collector's argument [22]. $\square$

The following theorem shows that Symmetric M_{exp0HD} is also efficient for CLIFF.

THEOREM 5.2. *The (1 + 1) Opt-IA using Symmetric M_{exp0HD} with $\tau = \Omega(n^{1+\epsilon})$ optimises CLIFF_k with $k < n(\frac{1}{4} - \epsilon)$ and $k = \Theta(n)$ in $O(n^{3/2} \log n + \tau n^{1/2} + \frac{n^{7/2}}{k^2})$ expected fitness function evaluations.*

PROOF. The proof is almost identical to the proof of Theorem 4.2. The most important distinction is that once a solution with $n - k + 2$ 1-bits is created, its mutation potential remains as $M = 1$ until the global optimum is found. The reason is that when ageing is triggered, the surviving solutions all reset their origin to themselves, i.e., start doing randomised local search. $\square$

6 CONCLUSION

We have presented an analysis of inversely proportional hypermutations (IPH). Previous theoretical studies have shown disappointing results concerning the IPH operators from the literature. In this paper we proposed a new IPH operator based on Hamming distance and exponential decay. We have shown its effectiveness in isolation for unimodal functions compared to static hypermutations in the ideal conditions when the optimum is known. Furthermore, we have provided a symmetric version of the operator for the complete Opt-IA AIS to be used in practical applications where the optimum is usually unknown. We have proved its efficiency for two well-studied multimodal functions. Future work should evaluate the performance of the proposed algorithm for combinatorial optimisation problems with practical applications.

Acknowledgments: This work has received funding from the EPSRC under grant agreement number EP/M004252/1.

REFERENCES

- [1] Jason Brownlee. 2007. *Clonal Selection Algorithms*. Technical Report. Complex Intelligent Systems Laboratory (CIS), Swinburne University of Technology.
- [2] Frank M. Burnet. 1959. *The Clonal Selection Theory of Acquired Immunity*. Cambridge University Press.
- [3] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2017. On the Runtime Analysis of the Opt-IA Artificial Immune System. In *Proc. of GECCO 2017*. 83–90.
- [4] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2018. Artificial Immune Systems Can Find Arbitrarily Good Approximations for the NP-hard Partition Problem. In *Proc. of PPSN 2018*. 16–28.
- [5] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2019. Artificial Immune Systems Can Find Arbitrarily Good Approximations for the NP-Hard Number Partitioning Problem. *To appear in Artificial Intelligence* (2019). <https://doi.org/10.1016/j.artint.2019.03.001>
- [6] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2019. On Inversely Proportional Hypermutations with Mutation Potential. In *arXiv*. –, <https://arxiv.org/abs/1903.11674>.
- [7] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. 2019. When Hypermutations and Ageing Enable Artificial Immune Systems to Outperform Evolutionary Algorithms. *To appear in Theor. Comp. Sci.* (2019). <https://doi.org/10.1016/j.tcs.2019.03.002>
- [8] Vincenzo Cutello, Giuseppe Nicosia, Mario Pavone, and Jonathan Timmis. 2007. An Immune Algorithm for Protein Structure Prediction on Lattice Models. *IEEE Trans. Evol. Comp.* 11 (2007), 101–117.
- [9] Leandro N. de Castro and Jonathan Timmis. 2002. *Artificial Immune Systems: A New Computational Intelligence Approach*. Springer Science and Business Media.
- [10] Leonardo N. de Castro and Fernando J. Von Zuben. 2002. Learning and Optimization Using the Clonal Selection Principle. *IEEE Trans. Evol. Comp.* 6, 3 (2002), 239–251.
- [11] Stefan Droste, Thomas Jansen, and Ingo Wegener. 2002. On the Analysis of the (1+1) Evolutionary Algorithm. *Theor. Comp. Sci.* 276, 1-2 (2002), 51–81.
- [12] William Feller. 1968. *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons.
- [13] Tobias Friedrich, Pietro S. Oliveto, Dirk Sudholt, and Carsten Witt. 2009. Analysis of Diversity-Preserving Mechanisms for Global Exploration. *Evolutionary Computation* 17, 4 (2009), 455–476.
- [14] Christian Horoba, Thomas Jansen, and Christine Zarges. 2009. Maximal Age in Randomized Search Heuristics with Aging. In *Proc. of GECCO 2009*. 803–810.
- [15] Thomas Jansen and Christine Zarges. 2011. On the Role of Age Diversity for Effective Aging Operators. *Evolutionary Intelligence* 4, 2 (2011), 99–125.
- [16] Thomas Jansen and Christine Zarges. 2011. Variation in Artificial Immune Systems: Hypermutations with Mutation Potential. In *Proc. of ICARIS 2011*. 132–145.
- [17] Frank Neumann, Pietro S. Oliveto, and Carsten Witt. 2009. Theoretical Analysis of Fitness-proportional Selection: Landscapes and Efficiency. In *Proc. of GECCO 2009*. 835–842.
- [18] Pietro S. Oliveto and Dirk Sudholt. 2014. On the Runtime Analysis of Stochastic Ageing Mechanisms. In *Proc. of GECCO 2014*. 113–120.
- [19] Pietro S. Oliveto, Dirk Sudholt, and Christine Zarges. 2018. On the Benefits and Risks of Using Fitness Sharing for Multimodal Optimisation. *Theor. Comp. Sci.* (2018), –. <https://doi.org/doi.org/10.1016/j.tcs.2018.07.007>
- [20] Pietro S. Oliveto and Carsten Witt. 2014. On the Runtime Analysis of the Simple Genetic Algorithm. *Theor. Comp. Sci.* 545 (2014), 2–19.
- [21] Pietro S. Oliveto and Carsten Witt. 2015. Improved Time Complexity Analysis of the Simple Genetic Algorithm. *Theor. Comp. Sci.* 605 (2015), 21–41.
- [22] Pietro S. Oliveto and Xin Yao. 2011. Runtime Analysis of Evolutionary Algorithms for Discrete Optimization. In *Theory of Randomized Search Heuristics*, Anne Auger and Benjamin Doerr (Eds.). World Scientific, 21–52.
- [23] Edgar Covantes Osuna and Dirk Sudholt. 2017. Analysis of the Clearing Diversity-preserving Mechanism. In *Proc. of FOGA 2017*. 55–63.
- [24] Dirk Sudholt. 2019. The Benefits of Population Diversity in Evolutionary Algorithms: A Survey of Rigorous Runtime Analyses. In *Theory of Randomized Search Heuristics in Discrete Search Spaces*, Benjamin Doerr and Frank Neumann (Eds.). Springer, –. <https://arxiv.org/abs/1801.10087>.
- [25] Darrell Whitley. 1989. The Genitor Algorithm and Selection Pressure: Why Rank-based Allocation of Reproductive Trials Is Best. In *Proc. of the Third International Conference on Genetic Algorithms*. 116–121.
- [26] Christine Zarges. 2008. Rigorous Runtime Analysis of Inversely Fitness Proportional Mutation Rates. In *Proc. of PPSN X*. 112–122.

Reply To Reviewers

We are grateful to the Associate Editor and the reviewers for their helpful comments which have indeed allowed us to improve the paper.

Reviewer: 1

Overall comment:

Sections 1, 2, and 3 already exist in the conference version, while Sections 4 and 5 contain the new theorems and results. There are some awkward sentences and some errors/ambiguities in the algorithm definition/pseudocode, but the theoretical results seem valid to me as far as I could follow (I got lost at several points due to presentation issues and writing ambiguity). Please revise the paper once again, more carefully, to improve clarity and to ensure the paper is fully consistent.

We have revised the paper once again to improve clarity and remove inconsistencies.

Some detailed issues:

Page 1, second column, line 13 to 15: refs [28], [29], [30] are from 2015, 2022 and 2025 (not in order). The first two are 7 years apart, and then the third is 3 years apart from the second one. If you have additional references that clearly support the claim of “increasing popularity”, add them; otherwise remove “increasing popularity”.

We have removed increasing popularity and just kept “are popular” justifying this with the survey paper and some recent theory and experimental papers.

Page 2, first column, line 45–46: consider using a dash instead of nested parentheses. **DONE**

Page 2, second column, line 41: there is an extra “allowed”. **DONE**

Page 2, second column, line 33–36: “However, an exponential decay with the distance to the optimum also implies that high mutation rates would occur only when the current solution is prohibitively far away from the optimum.” Clarify why this is a disadvantage here in the same sentence.

Done

Page 3, column 2, line 47: “With this operator we remember to update the best to the last seen individual of highest fitness ...”. Do you mean “last” or “best”?

We mean “last seen best”: we have better emphasized this now.

Page 4, Algorithm 4, lines 17–21: the “with probability p_{die} ...” block reads like it has a dangling structure. It looks like the middle if clause may have been pasted into the wrong place (?). As written, the most important part of the pseudocode is hard to read.

It is correct: we intend to update best to the “last seen best” before reinitialising the individual in the following line – We have improved the presentation on the algorithm to remove any ambiguities.

Page 4, Algorithm 4, line 17: am I missing something, or was p_{die} never introduced before it is used?

Right. We have now explained more clearly that the probability of an individual dying after reaching age τ is $p_{\text{die}}=1/2$.

Page 17, first column, line 22: awkward sentence: “Although our analysis used standard benchmark functions with significant structures to practical optimisation ...”. I think you mean that the benchmark functions have structures that resemble aspects of practical optimisation. Personally, I have not seen many practical optimisation problems that resemble these benchmarks. I would rather frame it as “resembling aspects of standard combinatorial optimisation problems / capturing specific structural challenges”, and avoid implying direct practical realism.

Thanks we have rephrased as “capturing specific structures that may appear in combinatorial optimisation problems”

Reviewer: 2

I have the following comments:

- Page 2, column 2, line 41: “practice” **DONE**
- Table 1: The asterisk is not defined – I presume it’s referring to the value of beta in the caption, but this is not explicit.

It was referring to the fact that we do not provide a formal analysis for the result. We now clarify this.

“For all results, the optimum is unknown...” (typo “is”) **DONE**

- Page 6, column 2: The large block of text is difficult to follow, I would separate it into some smaller paragraphs. **DONE**
 - Theorem 5, proof: The first line has semicolons – I’m not sure the meaning. This appeared a few times in the appendix as well. **DONE**
 - Page 9, column 1, line 49: “To use an ageing operator, an age ...” (missing comma) **DONE**
 - Page 9, column 2, line 50: “Hamming” (missing capital) **DONE**
 - Figure 3: I would suggest cropping off the x-axis past the global optimum, and labelling the local and global optimum. **DONE**
- It would also be useful to see a diagram for the “LeadingTrapJump” function, since it is not a common benchmark. **DONE**

- Corollary 4: What is the meaning of the term “e” in “ n^e ”? I guess it’s a typo for “4” but please check. **Yes, done!**
 - Section VI: It is mentioned that for LeadingTrapJump (LTJ), low mutations are more effective, but if the position of the global and local optima swap, then high mutations are more effective.
- Can you provide some quick result for the analysed IPH in this setting (i.e., swapped local and global optima)? Presumably it is similar to the one from LeadingOnes, but formalising this would show that it can be effective in either case.

Theorem 11 already covers this scenario: the proof shows that both optima are identified within the claimed runtime. We now state this clearly in the theorem statement.

- Similarly, are there other algorithms that perform well on LTJ? Since there exist a

number of algorithms that perform well on Onemax, LeadingOnes and Cliff, highlighting that these algorithms are poor on LTJ explicitly would be a useful thing to mention, and highlighting any that are good on LTJ would also be interesting. E.g., can rank-based mutation also be effective? If so, how is it on the first three problems.

The general point is that mostly AIS algorithms are analysed, but it is not explicitly clear how they fare compared to other approaches across all four functions.

We have added a discussion regarding the performance of other related algorithms at the end of the section.

Reviewer: 3

1) In general, I think that AIS are an interesting engineering-based concept, but I do not see the profit from using such hypermutations (especially FCM) instead of just flipping a random number of bits, as in standard bit mutation with various mutation rate. Why should not we instead of flipping up to M bits sequentially make up to M individuals by flipping one random bit, two random bits, ..., M random bits. It allows a better exploration of the search space, while not decreasing the probability of success compared to FCM. I assume that it might be useful when we can re-calculate fitness of Hamming neighbors efficiently, but I did not see such motivation in the paper.

This may seem more natural, yet whether this allows a better exploration of the search space is questionable. This is, by the way, similar what is done in the Fast AIS and to some extent what the power law operator of the Fast-GA does, which is shown in [3] to have very similar behaviour. Yet, in both those cases no attempt at inversely proportional mutations to the optimum are attempted. We have now cited these works in context.

Moreover, I believe that the $(1 + 1)$ EA can perform not worse (often better) than the considered AIS, except for Cliff. But on Cliff the performance gain is not thanks to the hyper-mutations, it is thanks to the ageing mechanism (well, also thanks to the AIS not being able to flip more than one bit after falling off the cliff, but my argument holds when we replace the $(1 + 1)$ EA with RLS). To summarize this comment, I need to see more motivation for this study.

The $(1+1)$ EA is inefficient on Cliff *and* on LTJ. Thus it only succeeds on the considered unimodal functions which is not surprising. RLS also fails on LTJ. The point is to have an algorithm that can escape local optima with ageing *as well as* with high mutation rates.

2) Almost all proofs in Section III are "loose". The non-matching lower and upper bounds could easily be tightened by some stronger (or just different) arguments. I will list my suggestions on improving the proofs below.

3) I am not fully convinced by the proofs of the result for LeadingTrapJump. Namely, I have doubts about Lemma 3 for the FirstBest mechanism (details are below).

We reply to both comments below.

4) In general, there are many small issues that have to be addressed by the authors before the paper is publishable. It seems to me that some of them were caused by the adaptation of the sections from the original conference paper to the journal version or by shortening some sections.

Given these issues, I recommend a *major revision* of this paper.

****Tightening the proofs.****

Theorem 2. The upper bound can be reduced by a logarithmic factor by splitting the sum in line 24 (page 6, left column) into three (after taking the factor n out of the sum): $\sum_{i \in [1..n/\ln(n)]} n^{i/n}$ (here $n^{i/n}$ is at most e , so the sum is bounded by $\log n$), $\sum_{i \in [n/\ln(n)..n/\ln(n) + \delta n]} n^{i/n}$ for some small constant δ , e.g., $\delta = 1/4$ (here $n^{i/n}$ is at most n^δ , so the sum is bounded by $n^\delta \log n = o(\sqrt{n})$), and for the rest of $\sum_{i \in [n/\ln(n) + \delta n..n/2 + n^{2/3}]} n^{i/n}$ (here $n^{i/n}$ is bounded by $1/\delta$, so the sum is bounded by a geometric progression, and it is easy to compute that it is approximately $\sqrt{n}/\log(n)$). For the lower bound, first of all you can say that with probability at least $1/2$ we start with $i \geq n/2$. Therefore (together with the Ballot theorem), the expected runtime is at least $\frac{1}{4} \sum_{i = n/4}^{n/2} \frac{n}{i} n^{i/n}$ (note that we excluded levels $i < n/4$, since in the upper bound we saw that they do not contribute a lot). Note that $\frac{n}{i} n^{i/n} \geq 2$, therefore, we can bound this sum with the geometric progression, which is $\sqrt{n}/\log(n)$ (similar to the upper bound). So the resulting bound is $\Theta(n^{3/2} / \log(n))$.

Thanks, we have done this.

Theorem 4. Why do you restrict yourself for the interval of fitness values from $n/2 + \epsilon n/2$ to $n/2 + \epsilon n$? With probability $1/2$ we start with $i = 0$ and hence we can sum over all $i \in [0..n-1]$. The sum of $\sum_{i=0}^{n-1} n^{i/n}$ can be estimated via standard series $\sum_{i=0}^{\infty} x^i$ (since $x = n^{-1/n} < 1$), which will give us $n^3 / \log(n)$.

Thanks. We note that the correct tight bound is $\Theta(n^3 / \log^2 n)$ (not $n^3 / \log n$ as suggested): the extra log factor arises from $(1-x)^2$ in the denominator, which contributes $(\ln n/n)^2$. The theorem statement and proof have been updated accordingly.

Theorem 5. In line 43 (page 7, right column) you can use $n^{2/3}$ instead of ϵn , it will allow to drop the ϵ from the resulting bounds. The final sum for the lower bound again struggles from being taken only over a limited range of i . Similar to Theorem 4, it is possible to get a tight lower bound.

Thanks, done.

Theorems 6 and 7. These proofs can be significantly simplified by observing that before the first time when ageing is triggered, the algorithm behaves exactly as RLS (with 1-bit mutation). So let T be the time that it takes RLS to optimize function f , and let A be an event when the RLS cannot improve the solution for τ consecutive iterations in a row (for the Opt-IA it would mean that ageing is triggered). Then we have $E[T] = E[T|A] \Pr[A] + E[T|\neg A] \Pr[\neg A]$, which is known to be at most $n \log n$ for OneMax and n^2 for LeadingOnes. Let T' be the runtime of the Opt-IA, then we can write an equation $E[T'] \leq (n E[T] + 1) \Pr[A] + E[T|\neg A] \Pr[\neg A]$, from which it is easy to show that $E[T']$ is at most by a factor $(1 + o(1))$ larger than $E[T]$. For the lower bound, one can show that $E[T|A] \leq E[T] + \tau$ (we spend at most $E[T] + \tau$ iterations before A occurs, and after A occurs we just continue with RLS, and it takes at most $E[T]$ iterations to get to the optimum), hence the contribution of the term $E[T|A] \Pr[A]$ to $E[T]$ is negligible,

and hence $E[T'] \geq E[T | \text{not } A] \Pr[\text{not } A] = (1 - o(1))E[T]$. Probably, it makes sense to unite these two theorems into one with a shorter proof.

We have unified the repeated arguments in Theorems 6 and 7 by adding a Lemma (RLS Reduction) that formalises reviewer's observation.

****Problem with Lemma 3.****

First of all, in the end of this section you say that LastBest can jump off the cliff in two steps, so I assume that Lemma 3 was supposed to be for the FirstBest.

It is thanks – DONE

But in this case I do not understand how we can get such a good probability of falling off the cliff near the first best in phases after the first one. In the first phase everything is similar to the LastBest: we reach the cliff top, our potential is one, so we can only flip one bit and we do not move from this point until ageing is triggered.

right

However, after the first ageing we reach the cliff top in a random point (it is not obvious, but it seems that there should be no bias to the first best point).

At this point the solution at a random point will create an offspring with exactly Hamming distance 2 from its parent. Then the parent will die and become the new reference genotype. Thus, the surviving solution has HD=2 from the new reference as we have flipped a 0→1 and a 1→0. Furthermore, the potential is M=1, and since no improvement has occurred ageing will trigger again with p_{die} probability in the next step.

Therefore, when ageing is triggered, even if we fall off the cliff, we still have a huge potential which makes it likely to get back to the cliff before going away from it. So am I missing something, or does this proof not work for the first best mechanism?

Whenever a solution that ties with the *best* dies it becomes the new reference. In the next step a solution at the bottom of the Cliff is created (0→1) with HD =1 from the reference and ageing is triggered again. If the parent dies, and the offspring survives, the potential is still M=1 and creating an improving offspring leads to the age being set to 0.

****Detailed comments.****

The rest of my review are the detailed comments to the manuscript. They are divided by pages, and each comment starting with either "l[number]" or "r[number]" refers to the line with that number in the left or right column respectively.

Page 1

- r43–45: in this sentence there should be no inversion: "the closer te estimate is to the..., the closer the behaviour of the IPH operator should be..." **DONE**
- r52: common practice → a common practice. **DONE**

Page 2

- l45 and r19: I feel like the term "hill-climbing" is used in different ways here: in the first case it means exploitation (the final phase when the algorithm converges to the global optimum), and in the second case it just means optimization of unimodal

functions, where there is always a bit that we can flip and increase the fitness. Could you explain what you mean by "hill-climbing"?

What we mean by hillclimbing is actually searching similarly to a local search. This can be useful all the time (e.g., OneMax) or at various stages of the process. A Good example could be the second slope of Cliff: where it is productive to perform local search up the slope while high mutation rates are counterproductive as they risk jumping to other areas of the search space interrupting the climb up.

- r22: missing comma after "To show this". [DONE](#)
- r36: missing comma after "For instance". [DONE](#)

Page 3

- Table 1 caption: it is not clear at this moment, what c is (in the definition of β). [DONE](#)
- Table 1, Opt-IA (static), LTJ runtime: what does the star mean there? [DONE](#) – see [response to previous reviewer](#).
- r51: $M_{th} \rightarrow M_{-th}$. [DONE](#)
- r54: is it assumed that $f_{\{OPT\}}$ is non-negative or even positive? In this case, due to rounding up, is not it always just c_n ?

We fixed the typo where we left the c_n out of the ceiling function.

- r56–60: I guess, for OneMax a different potential was used: if we **minimize** OneMax, then $f_{\{OPT\}} = 0$ and the potential is always c_n .

In the original paper this was circumvented by defining a new function $\text{ZeroMin}(x) := n + 1 - \text{Onemax}(x)$ – we have now specified this.

Page 4

- l58: floor command is missing. [DONE](#)
- r21: simple to define $\rightarrow$ simple-to-define. [DONE](#)
- r26: I would extend the sentence: "before the first 0-bit or the end of the string". [DONE](#)
- r27: I would say 1-bit instead of k -bit, since for general k it is not true (when k is always 2, the expected runtime is infinite). If you disagree with my notation, at least k should be in math mode (k -bit instead of k -bit).

We have removed the k

- r46: My criticism of FCM arose here. On such benchmarks as OneMax, the improvements are possible only in the first bit flip, since all other flips can only lead to finding an individual that is as fit as the parent. So it seems like wasting fitness evaluations. Probably, it should be explained here that such hypermutations are not designed for easy hill-climbing at all, and their benefits can only be seen on such benchmarks as LTJ, where after several bit flips that decrease the fitness it is possible to find a fitness that is even better than the fitness of the initial parent with only one flip.

The idea of randomised search heuristics is not to be exceptionally fast on unimodal benchmark functions. Typically in multimodal optimisation slow downs in unimodal phases are absorbed by the time required to escape from local optima. Yet, there is no reason not to try and also improve the performance of the algorithms in the exploitation phases, which is the idea behind inversely proportional hypermutations (and of the clonal selection principle from which they are inspired).

Page 5

– Table 2: I am not sure that I understand the difference between "using static hypermutation" and "traditional Opt-IA". Is it about using ageing?

Yes, the complete Opt-IA algorithms uses both hypermutations and Ageing.

– l39: $\$q\$ \rightarrow \$Q\$$. DONE

– Theorem 1, lower bound: note that the fitness cannot be improved by more than one (it is essential argument, and you do not skip it in Theorem 2). DONE

– r52: there is an extra space between the bracket and the comma. DONE

– r55: either add a factor $\$(1 - o(1))\$$ in front of the sum, or add a note: "(w.o.p, thus it is also $\$\Omega(n^2)\$$ unconditionally)". DONE

Page 6

– l28: Although I suggest tightening this roof, in your proof you probably meant that w.o.p. we start with $\$i \geq n/2 - \epsilon/2\$$ (not just $\$\epsilon\$$). By the way, I find $\$\epsilon\$$ a better choice over $\$\epsilon\$$, but it is a matter of taste.

(resolved automatically by the rewritten Theorem 2 proof, which no longer uses this Chernoff cutoff).

Page 7

– l9: it is not a trivial conclusion about the probability to skip a level. Could you add more arguments here or add a reference? I assume, it should be in the first paper on LeadingOnes by Rudolph (1997), but I have not checked.

Expanded the argument: "the probability of not skipping level i is $\Omega(1)$. This holds because the expected number of free-riders (consecutive 1-bits following the leftmost 0-bit) is at most $\sum_{j \geq 1} j \cdot 2^{-(j+1)} < 1$, so the level advances by exactly 1 with probability at least $\frac{1}{2}$; see also [DJW02]."

– l22: add some transition to Theorem 4, otherwise it is hard for the reader to follow the storyline. Say something like "We now proceed with the analysis of the exponential-based potential". DONE

– l47–53: this paragraph repeats the last computation in the previous paragraph. I guess, it is an artifact of shortening the proof.

Removed (resolved by the rewritten Theorem 4 proof).

– r16: it is not important in the light of my suggestions above but you did not introduce ϵ in this proof. It is in the statement, but it would be nice to start this paragraph with "For any arbitrary small $\epsilon > 0$,..."

Resolved by the rewritten Theorem 4 proof, which does not use ϵ as a free parameter.

– r36: it is a strange way to start the proof with such an expression. I again assume that it is an artifact from the previous version of the paper.

Resolved by the rewritten Theorem 5 proof, which opens with a properly introduced Taylor expansion sentence rather than a bare display equation.

Page 8

- l15: missing comma after "Overall". [DONE](#)
- l22: missing period in "e.g." [DONE](#)
- l42: missing comma after "In this section". [DONE](#)
- l50, l54: you say that the age of individuals is initially zero twice. [DONE](#)
- r15: Here you use smallcaps in the index (for "LIN"), but everywhere else you do not do this. [DONE](#)

Page 9

- Algorithms 3 and 4, lines 17 and 18 correspondingly: I assume that there should be a check if the re-initialized individual is better than the best, even though it is not likely to happen (otherwise we might lose it). [Thanks, DONE](#)
- l30: theorem should work even for $\tau \geq Cn \ln n$ for some constant C that is large enough. [DONE](#)
- l35: due $\rightarrow$ due to. [DONE](#)
- r32: for consistency with the rest of the text, I recommend to also use "w.o.p." here instead of "with overwhelming probability". [DONE](#)

Page 10

- l38: if I understand correctly, it should be $\ell < 2n/3$ (instead of $\ell \leq 2n/3$). [DONE](#)
- l48: it should be $m/(n-1)$ instead of m/n .

[Corrected; text now explains that conditional on not hitting the improving bit, the first flip is uniform over the remaining \$n-1\$ positions.](#)

- l52: actually, not **at least** once, but **exactly** once [DONE](#)
- l53: "best" is a name of a variable, so there should be no "the" before it. I actually recommend to rename it to x_{best} .

[DONE, best is now typeset in italic throughout the affected proof](#)

- r41: missing comma after "In this section". [DONE](#)

Page 11

- l25: it should be the same ϵ as for c .

[We disagree here since the epsilon in \$c\$ and the cliff match while the epsilon in \$\tau\$ is an arbitrarily small constant](#)

- l30: specify that the "main result" is Theorem 9. [DONE](#)
- l56: r_0 and the state description of the form $(r, \text{HD}(r))$ have not been introduced. They are clear from the context, but it is better to avoid ambiguity.

[Added an explicit introduction of the indexing convention and notation before Lemma 3.](#)

- Lemma 4: specify that you mean the return in one hypermutation (if I understand it correctly). [DONE](#)
- r59: I would swap the fraction and $K(\lambda)$ like this: $\frac{1 - \rho}{1 - \rho e^{2\lambda}} \approx K(\lambda)$.

[All three definition sites now present the closed-form expression first, followed by \$= K\(\lambda\)\$.](#)

Page 12

- l9: specify that c for Θ is from the potential.

We have now relaxed the claim to $O()$ instead of $\Theta()$ and noted that the extra n is from the linear potential.

- l16: it is not the phase MGF, it is the MGF of the buffer growth per phase. **DONE**
- l45: not the buffer itself, but $e^{-\lambda B_t}$. **DONE**
- r48: inspired from $\rightarrow$ inspired by the. **DONE**
- r50: population based $\rightarrow$ population-based. **DONE**

Page 13

- l11: notation $x_{i > r}$ has not been introduced. **DONE**
- l20, l26: emphasize "block-jump" and "deep-prefix plateau". **DONE**
- l21: local optimum $\rightarrow$ local optimum $0^{r-1}1^{n-r}$. **DONE**
- Theorem 11: I do not understand, why should we introduce r ? Is not it enough to just replace it with k everywhere? I do not see the reason for prefix of zeros in B_k to be of an odd length

The $2k+1$ is the legacy of the original definition of the function, we introduced the shorthand r to avoid $2k+1$ in the exponents throughout

- l47: I am not sure I understand what you mean by "epoch starting at J ". Each epoch starts with a random individual, and only after that the algorithm goes to J , does not it? Same comment goes to Lemma 12. Probably, you denote by "ageing epoch" something different than I think, so it should be defined.

Both Lemma statements now read "an ageing epoch that reaches the junction J " (instead of "begins at"), and a sentence before the proof clarifies that epochs start with random reinitialisation and ascend to J with high probability.

- r3-r7: but initially there are $L/2$ one-bits, not L . **DONE**
- r10: why $q^* < 1/4$?

Added: "choosing ξ close to 0 gives $q^* \approx 2c(1-p)$; since $p > 4/5$ we have $1-p < 1/5$, so $2c(1-p) < 2 \cdot (5/8) \cdot (1/5) = 1/4$."

- r40: I am not sure that I understood the coupon-collector effect here and why we need both times

"Since B and D are mutually exclusive within a single epoch, they must be discovered in different epochs. By a coupon-collector argument with two coupons, the expected number of epochs until both have been seen is $O(1/p_B + 1/p_D)$."

Page 14

- l7: I would argue that these algorithms are still far from practice, since you do not use populations.

We don't say that the algorithms are sophisticated: we say that they can be applied to real problems where the optimum is unknown vs the algorithms of the previous section that use the location of the optimum, thus not applicable in a realistic optimisation problem.

- l15-16: there is no good example when such a re-initialization helps

We use all three strategies in this paper: non elitism for Cliff and both hypermutations and restarts for LTJ.

The automated restart is well known in the literature: with constant probability the algorithm restarts the optimisation once trapped on a local optima. There are several example problems in the literature where ageing makes algorithms efficient, including the partition problem discussed straight after. There, simple restarts via ageing (or sufficient parallel (1+1)EA runs) suffice to guarantee the approximations. We now provide citations. It makes more sense to restart automatically due to a lack of improvement than according to a predefined schedule.

- l18-21: do you mean FirstBest here?

Yes, thanks.

- l22-34: with no details in the paper it should be formulated as a *conjecture*.

We are not making a conjecture. Concerning algorithms using ageing the result follows trivially from [39]. Concerning algorithms using IPH alone, we state an open problem.